

EXTENSE LLC

DITA Training Guide: From First Topic to AI-Powered Delivery

A Progressive Learning Path — Content Strategy,
Authoring, Development, and Intelligent Delivery

Comprehensive Training Guide

Preface: Who This Guide Is For

A progressive training guide that takes you from first DITA topic to production publishing and AI-powered delivery — with hands-on tutorials, real examples, and measurable outcomes for both technical writers and software engineers.

Who should read this guide?

This guide serves two primary learning tracks — you can follow one or both:

- **Technical writers** — Master DITA authoring, information typing, reuse, metadata, and publishing workflows. You will write real topics, build maps, and publish multi-channel output.
- **Software engineers** — Master DITA-OT architecture, plugin development, XSLT customization, CI/CD pipelines, Docker builds, and AI integration. You will build plugins, automate publishing, and connect content to RAG pipelines.

The guide is also valuable for content strategists planning modernization initiatives, documentation managers evaluating structured content ROI, and information architects designing content models.

What you will learn

The guide follows a progressive learning path — each part builds on the previous one. By the end, you will be able to:

1. **Explain** why unstructured content creates real problems for your team — duplication, inconsistency, scaling failure — and how structured content solves them
2. **Set up** a DITA authoring environment with DITA-OT, an XML editor, and project scaffolding
3. **Author** concept, task, and reference topics using proper information typing, and apply reuse patterns (conref, keyref, conkeyref)
4. **Design** topic hierarchies, controlled vocabulary, and maps that scale
5. **Enrich** content with metadata, taxonomy, and Schema.org markup for both human findability and AI readability
6. **Publish** HTML5, PDF, and multi-channel output with DITAVAL filtering and automated pipelines
7. **Develop** DITA-OT plugins, customize HTML5 transforms with XSLT, and deploy via Docker and CI/CD
8. **Connect** enriched DITA content to RAG pipelines, vector stores, and chatbot delivery
9. **Measure** the ROI and impact of intelligent content across your organization

How this guide is organized

The guide has 10 parts organized in a progressive learning path. Parts build on each other, so work through them in order. XML basics and inline explanations are provided where needed — no prior DITA or XML experience is required.

Part	Covers	Level	Best for
I — Why Intelligent Content	Problems your team faces today with unstructured content; what intelligent content is; AI readiness maturity model; unstructured format limitations	Beginner	Everyone
II — Content Preparation	DITA fundamentals and XML syntax primer; install DITA-OT; configure your editor; project setup and quick start	Beginner	Everyone
III — Authoring DITA Content	Topic types and authoring foundations; hands-on WidgetPro tutorial with inline tips; information typing basics; information typing for AI; reuse and variant management	Intermediate	Everyone
IV — Information Architecture	Topic-based architecture; taxonomy and controlled vocabulary; progressive map design	Intermediate	Writers, Architects
V — Making Content Intelligent	Metadata enrichment; Schema.org integration; migration from unstructured XML (step-by-step); AI-ready authoring guide; before-and-after examples	Intermediate	Everyone
VI — Publishing & Automation	HTML5 and PDF builds; multi-channel output; DITA project files; CI/CD publishing pipelines; troubleshooting	Intermediate	Everyone
VII — DITA-OT Development	OT mental model and preprocessing pipeline; plugin creation; Docker containers; GitHub Actions CI/CD; debugging	Advanced	Engineers
VIII — XSLT Customization	Hands-on labs: XSLT overrides, sidebar TOC, element styling, production patterns, plugin packaging	Advanced	Engineers
IX — AI Pipeline Integration	RAG pipeline architecture; chunking and embedding; retrieval and generation	Advanced	Engineers
X — Benefits & Impact	ROI measurement; utilizing intelligent features; impact on final products	Beginner	Everyone

Each part includes transition sections ("What's Next") that explain how the current material connects to what follows, so you always know where you are in the learning path.

Contents

Preface: Who This Guide Is For.....	iii
Part I: Why Intelligent Content Matters.....	9
What Is Intelligent Content?.....	10
The Problem with Unstructured Content.....	12
AI-Readiness Maturity Model for Content.....	15
Why Unstructured Formats Are Not Enough: A Detailed Analysis.....	16
Part II: Content Preparation.....	21
Quick Start Guide.....	22
DITA at a Glance.....	28
DITA Deep Dive.....	36
Installing DITA-OT.....	42
Setting Up Authoring Tools.....	44
Part III: Authoring DITA Content.....	47
Authoring DITA Topics.....	48
Writing Your First DITA Project.....	56
Information Typing: Matching Content to Purpose.....	60
Information Typing for AI: Intent Routing and Answer Formatting.....	63
Reuse, Keys, and Variant Management.....	66
Part IV: Information Architecture.....	75
Topic-Based Information Architecture for Training.....	76
Taxonomy and Controlled Vocabulary for AI Classification.....	78
Part V: Making Content Intelligent.....	83
Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step.....	84
Before and After: Complete Transformation Examples.....	88
Metadata Enrichment: Adding the AI Layer to Every Topic.....	94
AI-Ready Content Authoring Guide.....	97
DITA to Schema.org Type Mapping.....	100
Part VI: Publishing & Automation.....	103
Publishing with DITA-OT.....	104
Multi-Channel Output Formats.....	112
DITA Project Files.....	115
Output Customization Without Plugins.....	118
CI/CD Publishing Pipelines.....	121
Troubleshooting DITA Builds.....	125

Part VII: DITA-OT Development..... 131

DITA-OT Architecture and DevOps.....	132
Architecture and Preprocessing.....	132
How DITA-OT Is Built and Its Core Architecture.....	132
Preprocessing Deep Dive.....	133
Map-First Preprocessing (preprocess2) in 4.x.....	135
Extension Points and Plug-In Fundamentals.....	135
Customization and Plug-Ins.....	136
Safe Customization Strategy: XSLT-Import Overrides.....	136
Writing Robust XSLT Customizations.....	137
Adding Pipeline Steps: Pre/Post Preprocess and Legacy Hooks.....	137
A Minimal Developer Plug-In Template.....	138
DevOps and Operations.....	139
Developing DITA-OT from Source.....	139
Docker for DITA-OT: Local Builds and Reproducible Toolchains.....	140
Build a Custom Docker Image with Plug-Ins Pre-Installed.....	141
docker-compose for Doc Builds and Local Preview.....	141
GitHub Actions: CI/CD Publishing and Workflow Examples.....	142
Debugging Playbook.....	144
Repository Layout and Build Variants.....	145
Key References.....	146

Part VIII: Advanced XSLT Customization..... 147

XSLT Customization Tutorial.....	148
Concepts & Foundations.....	149
What you're customizing (HTML5 transform architecture).....	149
Two safe strategies for HTML5 customization.....	150
Prerequisites: what you need to know before overriding.....	150
DITA-OT 4.x setup: sidebar TOC + chunking baseline.....	151
Filtering with DITaval and build variants (audience/platform).....	152
TOC grouping with topichead/topicgroup (non-clickable headings).....	153
Plugin Development.....	153
Create a minimal HTML5 customization plug-in.....	153
Write your first override XSLT correctly.....	154
Production patterns for all topic types + common elements.....	155
Extended "common element" XSLT overrides (hooks + data-attributes).....	156
Sidebar TOC customization.....	158
Passing parameters cleanly.....	159
Operations & Reference.....	160
Debugging workflow (iterate fast, safely).....	160
Hardening for production (CI, quality gates, upgrade plan).....	160
Exercises and labs (hands-on).....	161
Troubleshooting.....	164
Appendix: starter repo integration guide.....	165
Appendix: quick checklists.....	166
Technical Reference: XSLT Patterns and Glossary.....	166

Part IX: AI Pipeline & Chatbot Integration..... 169

RAG Pipeline Architecture: From DITA to Chatbot Answers.....	170
Section-Aware Chunking and Embedding Strategy.....	172
Retrieval, Reranking, and Answer Generation.....	174
Part X: Benefits, Utilization & Impact.....	179
Measurable Benefits and ROI of Intelligent Content.....	180
Utilizing Intelligent Content Features: A Practical Guide.....	181
Impact on the Final Product: How Intelligence Changes Everything.....	184
Appendices — Reference Material.....	187
DITA-OT Architecture Diagram.....	188
Glossary.....	190
Complete Plugin Reference.....	202
Appendix: Complete AI-Ready Metadata Field Reference.....	204
From Unstructured XML to Intelligent DITA: Detailed Comparison and Improvements.....	207
Index.....	213

Part



Why Intelligent Content Matters

Topics:

- [What Is Intelligent Content?](#)
- [The Problem with Unstructured Content](#)
- [AI-Readiness Maturity Model for Content](#)
- [Why Unstructured Formats Are Not Enough: A Detailed Analysis](#)

What Is Intelligent Content?

Intelligent content is structurally rich, semantically categorized, and machine-readable — enabling AI systems to discover, classify, retrieve, and deliver it without human intervention.

How is intelligent content defined?

Intelligent content has five defining characteristics that distinguish it from ordinary documentation:

1. **Structurally rich:** Content is organized into typed, modular units (concepts, tasks, references) with semantic markup — not flat text with visual formatting.
2. **Semantically categorized:** Every content unit carries metadata that describes what it is about, who it is for, how difficult it is, and what user intent it serves.
3. **Discoverable:** Metadata, taxonomy, and controlled vocabulary enable precise search, filtering, and retrieval — by both humans and AI systems.
4. **Reusable:** Content units are authored once and assembled into multiple outputs, audiences, and delivery channels without duplication.
5. **Adaptable:** Conditional processing and metadata-driven filtering enable personalized delivery based on user role, platform, product version, or context.

When content has these properties, AI systems — chatbots, search engines, recommendation engines, and RAG pipelines — can work with it effectively. Without them, AI is limited to brute-force keyword matching on unstructured text.

Why is DITA the ideal foundation for intelligent content?

DITA (Darwin Information Typing Architecture) was designed from the ground up for the properties that intelligent content requires:

Property	How DITA provides it	How AI uses it
Structural typing	<code><concept></code> , <code><task></code> , <code><reference></code> — each with specific child elements	AI can format answers differently: prose for concepts, step-lists for tasks, tables for references
Prolog metadata	<code><prolog></code> with audience, category, difficulty, dates, custom <code><data></code> elements	Enables metadata-filtered retrieval: "show me advanced developer content about XSLT"
Section granularity	ID'd <code><section></code> elements with titles inside topic bodies	Section-aware chunking produces focused, retrievable units for vector search
Controlled vocabulary	Subject scheme maps with <code><subjectdef></code>	Normalizes classification values — AI knows "developer" and "engineer" mean the same audience

Property	How DITA provides it	How AI uses it
Content reuse	Conref, keyref, conkeyref mechanisms	Single-source truth — AI retrieves the authoritative version, not duplicates
Conditional processing	DITAVAL filtering on audience, platform, product attributes	AI can serve role-specific or product-specific content variants

The shift from documents to content components

The fundamental shift in intelligent content is moving from a **document-centric** model to a **component-centric** model:

Document-centric (Word/flat XML/PDF)	Component-centric (DITA)
A "file" is the unit of authoring	A "topic" is the unit of authoring — typed by purpose
Content is organized by document structure	Content is organized by information model and assembled by maps
Formatting is embedded in content	Formatting is applied at publish time via transforms
Metadata is an afterthought (YAML front matter at best)	Metadata is a first-class citizen in the prolog
Reuse requires copy-paste	Reuse is built into the architecture (conref, keyref)
One output per document	One source, many outputs (HTML5, PDF, JSON, chatbot)

Related tasks

[Quick Start Guide](#) on page 22

This quick start guide helps you navigate the documentation collection and begin working with DITA immediately.

Related reference

[Glossary](#) on page 190

Comprehensive DITA and DITA-OT terminology used throughout the Extense training documentation. Each term includes definitions, abbreviations, usage guidance, and scope notes for chatbot knowledge retrieval.

The Problem with Unstructured Content

Unstructured content — flat XML, Word, wikis, flat HTML — creates real problems for your team today (duplication, inconsistency, scaling failure) and prevents AI systems from working with your content effectively.

Problems your team faces today

Before we talk about AI, consider the problems that unstructured content creates for **your team right now**:

- **Copy-paste duplication:** The same warning, procedure, or product description lives in multiple files. When it changes, someone has to find and update every copy — and they always miss one.
- **Inconsistent terminology:** One writer calls it "the admin panel," another says "the management console," and a third writes "the dashboard." Users (and search engines) cannot tell if these are the same thing.
- **No single source of truth:** Product names, version numbers, and URLs are hard-coded throughout documents. A rebrand or version bump means a global find-and-replace with no safety net.
- **Variant headaches:** You need a Windows version and a Linux version of the same guide. With flat files, you maintain two nearly identical documents — or resort to fragile conditional comments.
- **Scaling failure:** A 20-page guide in a flat XML file is manageable. At 200 pages with multiple products and audiences, the lack of structure makes authoring, review, and publishing painful.
- **No structured review:** Reviewers cannot filter by topic type, audience, or difficulty — they must read everything linearly, even the parts that are not relevant to them.

These are authoring and governance problems that structured content solves *independent of AI*. But they also have a compounding effect: the same lack of structure that hurts your team also prevents AI systems from working with your content.

What do AI systems need from content?

When a user asks a chatbot "How do I create a custom HTML5 plugin for DITA-OT?", the AI system must:

1. **Find** the right content — not just keyword-matching "plugin" but understanding the user wants a how-to guide about HTML5 customization
2. **Filter** by relevance — the developer-facing task topic, not the manager-facing overview
3. **Extract** a focused answer — the step-by-step instructions, not the entire document
4. **Format** appropriately — showing numbered steps for a task, not a wall of prose
5. **Cite** the source — linking back to the authoritative topic

Each of these steps requires **semantic signals** that unstructured content cannot provide.

How does unstructured content fail AI systems?

Consider a typical unstructured XML training file:

```
<article>
  <h1>DITA Reuse and Variants</h1>

  <h2>Overview</h2>
  <p>Content reuse is a key benefit of DITA. You can reuse content
  at multiple levels using different mechanisms.</p>

  <h2>Conref</h2>
  <p>The conref mechanism lets you pull content from one topic into
  another. It works like a symlink for content.</p>

  <h2>Keyref</h2>
  <p>Keys provide indirect references. Define a key in your map,
  then reference it in topics.</p>

  <h2>DITaval Filtering</h2>
  <p>Use DITaval files to include or exclude content based on
  attributes like audience, platform, or product.</p>

  <h2>Best Practices</h2>
  <ul>
    <li>Always use keys for product names</li>
    <li>Prefer keyref over conref when possible</li>
    <li>Test all variants before publishing</li>
  </ul>
</article>
```

This content is technically accurate — but it gives AI **zero semantic signals**:

AI need	What unstructured XML provides	Impact on chatbot quality
Content type	Nothing — is this a concept? task? reference?	AI cannot format the answer appropriately
Audience	Nothing — who is this written for?	AI cannot filter by user role; shows expert content to beginners
Difficulty	Nothing — is this beginner or advanced?	AI cannot match content level to user expertise
Intent	Nothing — is this explaining, instructing, or troubleshooting?	AI cannot route "how do I..." questions to task content
Domain	Nothing — what subject area is this?	AI retrieves content from wrong domains
Chunk boundaries	Headings only — no section IDs, no semantic markers	Chunking is fragile; chunks lack addressable IDs for deep-linking
Freshness	No dates — when was this last updated?	AI cannot rank newer content higher

AI need	What unstructured XML provides	Impact on chatbot quality
Relationships	No prerequisite chains or learning paths	AI cannot suggest "read this first" or next steps

What are the real-world consequences?

When an organization feeds unstructured content into a chatbot or RAG pipeline, they experience:

- **Poor retrieval precision:** The chatbot finds vaguely related content instead of the exact answer, because it relies solely on keyword/vector similarity with no metadata boosting.
- **Wrong audience matching:** A beginner asks "What is DITA?" and gets an expert-level architecture deep-dive because all content is at the same "level."
- **Broken intent routing:** A user asks "How do I configure keys?" (wants a how-to task) but gets a conceptual overview because the AI cannot distinguish content types.
- **Duplicate/conflicting answers:** Copy-pasted content across multiple documents produces multiple retrieval hits with slightly different wording, confusing the AI's generation step.
- **No measurement or improvement:** Without metadata, you cannot analyze which content domains are underperforming, which audiences are underserved, or where gaps exist.

What is the cost of staying unstructured?

Organizations that attempt AI-powered content delivery without structured content face compounding costs:

- **Prompt engineering escalation:** Teams spend weeks tuning system prompts and retrieval thresholds to compensate for poor content quality — a problem structured content solves at the source.
- **Hallucination risk:** When retrieval quality is poor, the LLM fills gaps with generated (and potentially wrong) information. Structured content with metadata-filtered retrieval dramatically reduces this risk.
- **Maintenance burden:** Duplicated content across files means updates must be made in multiple places — and missed updates create inconsistencies that degrade AI answers.
- **Scaling limits:** Flat-file content that works for 20 topics breaks down at 200 or 2,000 topics because there is no classification, filtering, or governance infrastructure.

AI-Readiness Maturity Model for Content

A five-level maturity model for evaluating how ready your content is for AI-driven delivery — from Level 1 (unstructured files) to Level 5 (fully intelligent, self-optimizing content).

The five levels of content AI-readiness

Level	Name	Content characteristics	AI capability
1	Unstructured	Flat files (Word, wiki pages, flat XML/HTML). No metadata. No typing. Visual formatting embedded in content.	Keyword search only. No filtering, no intent routing, no personalization.
2	Structured	Content organized into typed units (concepts, tasks, references). Semantic markup replaces visual formatting. Basic metadata present.	Vector search with section-aware chunking. Type-based answer formatting. Basic filtering by category.
3	Enriched	Full metadata on every topic: audience, difficulty, domain, intent, priority. Controlled vocabulary via taxonomy. Section IDs enabling deep linking.	Metadata-filtered retrieval. Intent-based answer routing. Audience-matched results. Priority-weighted ranking.
4	Connected	Content linked into learning paths, prerequisite chains, and relationship tables. Glossary with synonym resolution. Schema.org structured data output.	Follow-up suggestions. "Read this first" guidance. Term resolution. Web-standard structured data for search engines.
5	Intelligent	Analytics-instrumented output. Feedback loops. Content automatically improves based on usage data, retrieval performance, and user ratings.	Self-optimizing content delivery. Data-driven gap analysis. Automated quality scoring. Continuous improvement pipeline.

How to score your content's AI-readiness

Evaluate your content across ten dimensions, scoring each from 1 (absent) to 10 (excellent):

- Structural intelligence:** Are topics properly typed (concept, task, reference)? Do sections have IDs and titles? Is there a clear information hierarchy?
- Metadata completeness:** Does every topic have audience, difficulty, domain, intent, priority, dates, and keywords?
- AI/RAG pipeline readiness:** Is there an ingestion pipeline? Chunking strategy? Vector database? Retrieval configuration? (RAG — *Retrieval Augmented Generation* — is a pattern where an AI retrieves relevant content chunks from a database before generating an answer, rather than relying solely on its training data.)

4. **Taxonomy and controlled vocabulary:** Is there a formal subject scheme (a DITA file that defines the allowed metadata values — like a picklist for categories) or other classification system? Are values consistent across topics?
5. **Glossary and term resolution:** Is there a machine-readable glossary with synonyms, abbreviations, and usage guidance?
6. **Schema.org / structured data:** Do published outputs include JSON-LD (a lightweight format for embedding machine-readable metadata in HTML pages) or other structured data markup that search engines can consume?
7. **Retrieval optimization:** Are shortdescs written as chunk summaries? Section titles as questions? Content front-loaded?
8. **Reuse architecture:** Is content single-sourced via conref/keyref? Are there shared libraries?
9. **Consistency across corpus:** Is the metadata standard applied uniformly? Are there gaps or two-tier quality?
10. **Scalability and governance:** Can the content model scale to 10x the current corpus? Are there authoring guidelines and validation?

A score below 5/10 on any dimension indicates a critical gap that will degrade AI-driven delivery. The target for production chatbot deployment is 8+/10 on all dimensions.

Case study: The Extense training guide assessment

When we assessed the Extense DITA Training Guide against this model, the initial scoring was:

- **Overall: 7.2/10** — strong foundation, but a critical gap in metadata consistency
- **Structural intelligence: 9/10** — proper information typing throughout
- **Metadata completeness: 6/10** — 15 top-level topics had full metadata, but 34 sub-topics had bare-minimum metadata (author + indexterms only)
- **Consistency: 5/10** — the two-tier metadata gap meant 69% of topics were invisible to metadata-filtered retrieval

After enriching all 49 topics with full AI-ready prolog metadata:

- **Overall: 9.2/10** — production-ready for chatbot deployment
- **Metadata completeness: 9/10** — 100% of topics carry full AI-ready metadata
- **Consistency: 9/10** — uniform standard across all topic tiers

This guide documents every step of that transformation.

Why Unstructured Formats Are Not Enough: A Detailed Analysis

Unstructured XML, flat HTML, and document-centric formats lack the semantic structure, metadata system, and content typing that AI-driven delivery requires.

What do unstructured formats do well?

Unstructured formats have real strengths that should be acknowledged:

- **Low barrier to entry:** Anyone can write flat HTML or basic XML in a text editor
- **Familiar tooling:** Most teams already have XML or HTML editors and workflows in place
- **Fast drafting:** Minimal schema overhead for getting initial content created
- **Wide ecosystem:** Browsers, CMS platforms, and web frameworks all consume HTML natively

These strengths make unstructured formats useful for **initial content capture** — but the content must evolve into semantically rich XML to power AI-driven experiences.

The semantic poverty of unstructured content

Unstructured XML and flat HTML have basic structural elements — headings, paragraphs, lists, tables — but no **semantic meaning**. A `<div>` or `<section>` tells you nothing about the content's purpose.

Compare what unstructured XML can express versus what DITA can express:

Semantic concept	Unstructured XML / HTML	DITA	Why this matters for your team
"This content explains a concept"	Cannot express this	<code><concept></code>	Reviewers and AI can filter by content type — show only how-tos, only explanations, etc.
"This content teaches a procedure"	Cannot express this	<code><task></code> with <code><steps></code>	Steps are validated for completeness; readers always see prereqs, steps, and expected results in a consistent order.
"This is a prerequisite"	Just text under a heading	<code><prereq></code>	Prerequisites are always visible before the steps — readers never start a task they cannot finish.
"This is the expected result"	Just text at the end	<code><result></code> , <code><stepresult></code>	Users can verify they completed a step correctly; helps troubleshooting when something goes wrong.
"This content is for developers"	Cannot express this	<code><audience>developer</audience></code>	Content can be filtered so each audience sees only what is relevant to their role — fewer pages, less noise.
"This content is at advanced level"	Cannot express this	<code><data name="difficulty" value="advanced"/></code>	Beginners are not overwhelmed by expert content; teams can track coverage gaps by difficulty level.

Semantic concept	Unstructured XML / HTML	DITA	Why this matters for your team
"This section can be reused elsewhere"	Copy-paste the text	<code>conref="shared/common.dita#warnings/safety"</code>	Update once, propagate everywhere — no more inconsistent safety warnings or outdated procedure steps.

Before: A real unstructured XML training file

This is an actual file from training content before migration to DITA:

```
<article>
  <h1>DITA Introduction Guide</h1>

  <section>
    <h2>What is DITA?</h2>
    <p>DITA (Darwin Information Typing Architecture) is an XML-based standard for structuring, writing, and publishing technical content. Created by IBM in the early 2000s and now maintained by OASIS, DITA enables modular, reusable documentation.</p>
  </section>

  <section>
    <h2>Why use DITA?</h2>
    <ul>
      <li><b>Modularity</b> – Write once, reuse everywhere</li>
      <li><b>Consistency</b> – Enforce structure and style</li>
      <li><b>Multi-channel</b> – Publish to HTML, PDF, mobile</li>
      <li><b>Collaboration</b> – Multiple authors, component review</li>
    </ul>
  </section>

  <section>
    <h2>Key Concepts</h2>
    <ul>
      <li><b>Topics</b> – The fundamental unit of content</li>
      <li><b>Maps</b> – Organize topics into deliverables</li>
      <li><b>Specialization</b> – Extend DITA types for your domain</li>
      <li><b>Conditional processing</b> – Filter by audience/platform</li>
    </ul>
  </section>

  <section>
    <h2>Getting Started</h2>
    <ol>
      <li>Install DITA-OT</li>
      <li>Create your first topic</li>
      <li>Build a map</li>
      <li>Publish to HTML5</li>
    </ol>
  </section>
</article>
```

What the AI sees: Generic XML with headings and lists. No metadata, no typing, no audience signal, no difficulty level, no subject classification. The AI must guess everything from the text content alone.

After: The same content as an AI-ready DITA topic

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">
<concept id="dita-intro-guide" xml:lang="en-US">
  <title>DITA Introduction Guide</title>
  <shortdesc>A comprehensive introduction to DITA – what it is,
  why organizations adopt it, and the core concepts that make
  structured authoring powerful.</shortdesc>
  <prolog>
    <author>Extense LLC</author>
    <critdates>
      <created date="2026-02-10"/>
    </critdates>
    <metadata>
      <keywords>
        <indexterm>DITA introduction</indexterm>
        <indexterm>What is DITA</indexterm>
        <indexterm>Why use DITA</indexterm>
      </keywords>
      <category>taxonomy:getting-started</category>
      <audience>all</audience>
    </metadata>
    <resourceid appid="extense-training" appname="chatbot"
    id="dita-intro-guide"/>
    <data name="difficulty" value="beginner"/>
    <data name="duration" value="PT15M"/>
    <data name="content-domain" value="dita-fundamentals"/>
    <data name="content-intent" value="explain"/>
    <data name="chatbot-priority" value="high"/>
    <data name="service-line" value="dita-engineering"/>
    <data name="prerequisites" value="none"/>
  </prolog>
  <conbody>
    <section id="what-is-dita">
      <title>What is DITA?</title>
      <p>DITA (Darwin Information Typing Architecture) is an
      XML-based standard for structuring, writing, and publishing
      technical content...</p>
    </section>
    <section id="why-use-dita">
      <title>Why do organizations adopt DITA?</title>
      <!-- narrative content with semantic table -->
    </section>
    ...
  </conbody>
</concept>
```

What the AI now sees: A *concept* topic (not a task or reference). For *all audiences* at *beginner* level. In the *dita-fundamentals* domain. With *explain* intent. At *high* chatbot priority. With 3 sections, each with its own ID and title, each becoming a separately retrievable chunk. With a shortdesc that serves as the chunk summary. With indexterms that the vector search can match against.

What's next?

You have now seen why unstructured content fails for both teams and AI systems, and you understand the gap between generic XML/HTML and semantic DITA. In **Part 2: Content Preparation**, you will set up your DITA toolkit, configure your editor, and learn the fundamentals of DITA's topic-map-publish workflow — everything you need before writing your first structured topic.

Part



Content Preparation

Topics:

- [Quick Start Guide](#)
- [DITA at a Glance](#)
- [DITA Deep Dive](#)
- [Installing DITA-OT](#)
- [Setting Up Authoring Tools](#)

Quick Start Guide

This quick start guide helps you navigate the documentation collection and begin working with DITA immediately.

30-Minute Fast Track

Step 1: Understand What DITA Is (10 minutes)

Read: [DITA Quick Introduction Guide](#)

Key takeaways:

- DITA = structured XML content that's reusable and multi-format
- Topics are small, focused information units
- Maps assemble topics into deliverables
- Publishing happens through DITA-OT

Skip to: Section 2 (DITA mental model) and Section 4 (most important features)

Step 2: See a Working Example (10 minutes)

Explore: the `examples/dita-html5-sidebar-starter-v2/` directory in the starter repo

Actions:

1. Look at sample topics in `dita/topics/`
2. Open the map in `dita/maps/getting-started.ditamap`
3. Review the plug-in structure in `plugins/`

What you'll learn:

- What a DITA topic looks like
- How maps organize topics
- Basic project structure

Step 3: Try Your First Build (10 minutes)

Prerequisites: DITA-OT 4.x installed (dita-ot.org)

Build command:

```
cd examples/dita-html5-sidebar-starter-v2
./build/build.sh # or build.ps1 on Windows
```

View output: Open `out/html/index.html` in browser

What you'll learn:

- How to run a DITA build
- What HTML output looks like
- Where files are generated

What to Do Next (By Role)

I'm a Technical Writer

Your path:

1. Complete 30-minute fast track above
2. Read [DITA Authoring](#)
3. Write 3 practice topics (concept, task, reference)
4. Learn [Reuse and Variants](#)
5. Start creating real documentation

Time to productivity: 1-2 weeks

I'm a Developer/Engineer

Your path:

1. Skim [DITA Overview](#)
2. Read [DITA-OT Developer Handbook](#)
3. Work through [XSLT Customization Tutorial](#)
4. Set up Docker builds (Handbook section 9-11)
5. Configure CI/CD (Handbook section 12-13)

Time to productivity: 2-4 weeks

I'm a Documentation Manager

Your path:

1. Read [DITA Overview](#) (why DITA matters)
2. Review [Reuse and Variants](#) (strategy)
3. Study [Publishing with DITA-OT](#) (workflow)
4. Compare v1 vs v2 starter projects (scaling considerations)
5. Plan team training and rollout

Time to decision: 1 week

I'm Evaluating DITA

Your path:

1. Read [DITA Quick Introduction](#)
2. Review [Architecture Diagram](#)
3. Build v1 starter project
4. Compare with your current workflow
5. List pros/cons for your use case

Time to evaluation: 2-3 days

Documentation Roadmap

Level 1: Beginner (Getting Started)

Goal: Understand DITA concepts

- [x] DITA Quick Introduction (~30 min)
- [x] DITA Overview (~60 min)
- [x] Explore v1 starter project (~30 min)

Outcome: Can explain what DITA is and why it exists

Level 2: Intermediate (Core Skills)

Goal: Author and publish DITA content

- [x] DITA Authoring guide (~2 hours)
- [x] Write practice topics (~2 hours)
- [x] Reuse and Variants (~2 hours)
- [x] Publishing with DITA-OT (~2 hours)
- [x] Explore v2 starter (variants) (~1 hour)

Outcome: Can create and publish multi-variant documentation

Level 3: Advanced (Development)

Goal: Customize and extend DITA toolchain

- [x] Developer Handbook (~4 hours)
- [x] XSLT Customization Tutorial (~8 hours + labs)
- [x] Build custom plug-in (~4 hours)
- [x] Set up CI/CD (~2 hours)
- [x] Production deployment (~4 hours)

Outcome: Can build production DITA toolchains

Common Questions Answered

"What's the difference between DITA and Word/HTML/other formats?"

Answer: [Getting Started # Section 1](#)

DITA is structured, semantic, and designed for:

- Content reuse at scale
- Multiple output formats from one source
- Product/platform/audience variants
- Long-lived documentation with frequent updates

"How do I know if my team needs DITA?"

Answer: [DITA Overview # When to use DITA](#)

Consider DITA if you have:

- Multiple products/variants needing shared content
- Multiple output formats (HTML + PDF + help)
- Large documentation sets (100+ topics)
- Frequent releases and updates
- Multiple writers needing consistency

Skip DITA if you have:

- Single product, single format
- Small doc set (<20 topics)
- Infrequent updates
- No reuse requirements

"How long does it take to learn DITA?"

Answer: Depends on role and depth

- **Basic authoring:** 1-2 weeks
- **Publishing/variants:** 2-4 weeks
- **Development/customization:** 4-8 weeks

Use learning paths above to accelerate.

"What tools do I need?"

Answer: [Publishing Guide # Tools](#)

Required:

- DITA-OT (free, open-source)
- XML editor (Oxygen, VS Code, or text editor)
- Java 17+ (for DITA-OT)

Optional:

- CCMS (enterprise content management)
- Docker (for builds)
- Git (for version control)

"Can I migrate existing docs to DITA?"

Answer: Yes, common approaches:

1. **Manual conversion** (highest quality, most effort)
2. **Scripted conversion** (HTML/XML # DITA)
3. **Gradual migration** (new content in DITA)
4. **Hybrid approach** (convert high-value content first)

See Developer Handbook for migration strategies.

"How do I customize HTML output?"

Answer: [XSLT Customization Tutorial](#)

Two safe approaches:

1. **CSS only** (styling, no structure changes)
2. **XSLT-import overrides** (structure + styling)

Avoid modifying core DITA-OT files.

Training Resources by Format

Prefer Reading?

- [DITA Quick Introduction](#) (text)
- [DITA Overview](#) (comprehensive)
- All core topics (detailed guides)

Prefer Hands-On?

- Starter Project (multi-variant)
- v2 Starter Project (advanced)
- [XSLT Tutorial Labs](#)

Prefer Visual?

- [Architecture Diagram](#)
- Starter project folder structures
- Build output examples

Prefer Step-by-Step?

- This quick start guide
- XSLT Tutorial (18 sections)
- Developer Handbook (17 sections)

Success Milestones

Track your progress:

Milestone 1: Understand DITA

- ☐ Can explain what DITA is
- ☐ Understand topics vs maps
- ☐ Know when to use DITA vs alternatives

Verify: Explain DITA to a colleague

Milestone 2: Author DITA

- ☐ Can create concept topics
- ☐ Can create task topics
- ☐ Can create reference topics
- ☐ Can organize topics in a map
- ☐ Can use conref for reuse

Verify: Write 5 topics and publish to HTML

Milestone 3: Manage Variants

- ☐ Understand conditional processing
- ☐ Can create DITaval filters
- ☐ Can build multiple variants
- ☐ Can use keys for variable content

Verify: Build web and print variants from same source

Milestone 4: Customize Output

- ☐ Can modify CSS

- [] Can create XSLT overrides
- [] Can package as plug-in
- [] Can configure build parameters

Verify: Build custom-branded HTML output

Milestone 5: Production Ready

- [] Can set up CI/CD
- [] Can deploy to hosting
- [] Can troubleshoot build errors
- [] Can train team members

Verify: Automated builds deploying to production

Troubleshooting First Steps

Build fails?

1. Check DITA-OT installation (`dita --version`)
2. Verify Java version (`java -version` # 17+)
3. Review error message for specific issue
4. Check [Architecture Diagram](#) to identify stage

Content not appearing?

1. Check DITAVAL filtering (conditional processing)
2. Verify topic is referenced in map
3. Check chunking settings
4. Examine temp files in work directory

HTML looks wrong?

1. Verify CSS path in build properties
2. Check XSLT overrides are installed
3. Clear browser cache
4. Review generated HTML source

Need more help?

- [Troubleshooting section](#)
- [Debugging playbook](#)

Where to Get Help

Documentation Collection (This Repo)

- Section-specific README files
- Code comments in starter projects
- Troubleshooting sections

Official Resources

- DITA-OT Docs: dita-ot.org
- DITA Spec: docs.oasis-open.org/dita

Community

- DITA Users: groups.io/g/dita-users
- DITA-OT Slack: dita-ot.slack.com
- Stack Overflow: tag [dita](#)

Your Next Action

Based on your role, your very next step should be:

Technical Writer: # Read [DITA Authoring](#)

Developer: # Read [Developer Handbook Section 1-5](#)

Manager: # Review the starter project and build sample variants

Evaluator: # Build v1 starter project and review output

Ready to dive deeper? Use the bookmap navigation for complete access to all chapters.

Last Updated: February 2026 **Version:** 1.0

Related concepts

[What Is Intelligent Content?](#) on page 10

Intelligent content is structurally rich, semantically categorized, and machine-readable — enabling AI systems to discover, classify, retrieve, and deliver it without human intervention.

[DITA at a Glance](#) on page 28

DITA is designed for teams that need:

Related reference

[Glossary](#) on page 190

Comprehensive DITA and DITA-OT terminology used throughout the Extense training documentation. Each term includes definitions, abbreviations, usage guidance, and scope notes for chatbot knowledge retrieval.

DITA at a Glance

DITA is designed for teams that need:

Darwin Information Typing Architecture (DITA) is an **XML-based** standard for writing **topic-oriented, reusable** technical content and publishing it to multiple outputs (HTML, PDF, help systems, portals, etc.). It's maintained as an open standard by the **OASIS DITA Technical Committee**.

Why DITA exists (the problems it solves)

DITA is designed for teams that need:

- **Reuse at scale** (the same warning/procedure used across products and manuals)
- **Multiple deliverables from one source** (HTML + PDF + embedded help)
- **Product variants** (editions, platforms, regions, customer-specific builds)
- **Long-lived documentation** with frequent updates
- **Large teams** needing structure, consistency, and governance

The core idea: write small, semantically meaningful modules (topics), assemble them with maps, and publish through a repeatable pipeline.

The DITA mental model

Topics: small, typed units of information

DITA encourages authors to write content as **topics**: self-contained units that answer a user need. The most common base topic types are:

- **Concept** — explain “what/why”
- **Task** — explain “how”
- **Reference** — provide “facts/specs/options”

This style is called **information typing**: using different structures depending on the user intent.

Maps: assemble topics into deliverables

A **DITA map** is how you organize and publish:

- Defines the **table of contents**
- Controls **navigation structure**
- Can define **relationships/links**
- Defines **keys** (late-bound references)
- Often defines **publishing context** (for filtering/branding)

“Darwin”: specialization and evolution

DITA is built to **evolve** without breaking. You can extend (specialize) elements and constraints to fit your domain while keeping compatibility with tooling.

XML basics for DITA authors

DITA files are written in XML. If you have never worked with XML before, here are the five things you need to know before reading any DITA code:

1. **Elements** are the building blocks. They have an opening tag and a closing tag:

```
<p>This is a paragraph.</p>
<title>My Topic Title</title>
```

The text between the tags is the element's content. The tag name (like `p` or `title`) tells the processor what kind of content this is.

2. **Attributes** add extra information to an element. They go inside the opening tag as `name="value"` pairs:

```
<concept id="my-topic" xml:lang="en-US">
```

Here `id` and `xml:lang` are attributes of the `concept` element.

3. **Nesting** means putting elements inside other elements. Indentation shows the hierarchy:

```
<ul>
  <li>First item</li>
  <li>Second item</li>
</ul>
```

The `li` elements are nested inside the `ul` element. Not every element can go inside every other element — DITA's rules (defined by its DTD or schema) control what nesting is allowed.

4. **Self-closing tags** are for elements with no text content. Instead of `<tag></tag>`, write `<tag/>`:

```
<data name="difficulty" value="beginner"/>
<indexterm>DITA</indexterm>  <!-- has content, so NOT self-closing -->
```

5. **The DOCTYPE declaration** at the top of every DITA file tells the processor which topic type this file uses:

```
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">
```

You do not need to memorize this — copy it from a template. The key part is the word after `DOCTYPE` (concept, task, reference) which must match your root element.

That is all the XML you need to start reading and writing DITA. Every code example in this guide uses these five patterns.

Key building blocks in the spec (what you'll see in real projects)

Topic anatomy (common elements)

Most topics include:

- `<title>`: topic title
- Body element depends on type: `<conbody>`, `<taskbody>`, `<refbody>`
- Common inline structures: `<p>`, `<ph>`, `<uicontrol>`, `<codeph>`, etc.
- Lists/tables: `<ul>`, `<ol>`, `<simplatable>`, `<table>`

Task topics add specialized structure:

- `<prereq>` (before you start)
- `<context>` (why/when)
- `<steps>` with `<step><cmd>...</cmd></step>`
- `<result>` / `<postreq>`

Reference topics commonly use:

- `<refbody>` with sections or a reference table-like structure
- `<properties>` (property/value definitions) in some models

Maps: `topicref` and hierarchy

The map primarily uses `<topicref>` to point to topics:

```
<map>
  <title>Getting Started</title>
  <topicref href="concepts/what-is-dita.dita"/>
  <topicref href="tasks/install.dita"/>
</map>
```

DITA maps can also use:

- `<topichead>`: a heading without a linked topic
- `<topicgroup>`: grouping without appearing in TOC (tool-dependent behavior)
- Relationship tables (advanced linking)

Domains, constraints, and specialization (practical view)

In real implementations you may see:

- **Domains**: add sets of elements (e.g., UI domain, highlighting domain)
- **Constraints**: restrict what's allowed for a certain doc set
- **Specialization**: define new elements as specializations of existing ones

Rule of thumb: specialize only when you have a strong semantic or validation need and you can support it in publishing + authoring tools.

The most important DITA features you'll actually use

Reuse with `conref` (content reference)

`conref` pulls content from another location.

Use cases

- Shared warnings, notes, prerequisites
- Shared UI labels
- Shared steps or step fragments

Example (reuse a note):

```
<p conref="shared/common-notes.dita#common-notes/install-note"/>
```

Best practice

- Keep `conref` targets stable and well-named
- Treat shared snippets as “mini APIs” (don't casually restructure them)

Keys + `keyref` (late-bound reuse and linking)

Keys are defined in the map; topics reference them with `keyref`.

Why keys matter:

- Change a URL/product name once in the map # updates everywhere
- Reuse topics in different publications with different key definitions
- Avoid hard-coding product names and links in topics

Example (in map):

```
<keydef keys="product-name">
  <topicmeta>
    <keywords><keyword>Acme Writer Pro</keyword></keywords>
  </topicmeta>
</keydef>

<keydef keys="support-url" href="https://example.com/support"/>
```

In a topic:

```
<p>For help with <ph keyref="product-name"/>, see <xref keyref="support-url"/>.</p>
```

Conditional processing (profiling / filtering)

DITA supports filtering content based on attributes. Common attributes include:

- `audience`
- `platform`
- `product`
- `props` (generic)
- `otherprops` (legacy-ish patterns; depends on governance)

Typical use case: *one source, multiple editions* (e.g., Windows vs Linux).

In topics:

```
<p platform="windows">Click Start > Settings.</p>
<p platform="linux">Open the system settings panel.</p>
```

Then apply a filter (often using a **DITAVAL** file in DITA-OT) to include/exclude.

Governance tip: Define a controlled vocabulary early (what values are allowed) and enforce it.

A tiny working example (topics + map)

Concept topic (`concepts/what-is-dita.dita`)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">
<concept id="what-is-dita">
  <title>What is DITA?</title>
  <conbody>
    <p>DITA is an XML-based standard for writing topic-oriented content that can be
    reused and published to multiple formats.</p>
  </conbody>
</concept>
```

Task topic (`tasks/install.dita`)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA Task//EN" "task.dtd">
<task id="install">
  <title>Install the product</title>
  <taskbody>
```



```

<prereq>
  <p>Make sure you have administrator access.</p>
</prereq>
<steps>
  <step><cmd>Download the installer.</cmd></step>
  <step><cmd>Run the installer and follow the prompts.</cmd></step>
  <step><cmd>Verify installation by launching the application.</cmd></step>
</steps>
</taskbody>
</task>

```

Map (`guides/getting-started.ditamap`)

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN" "map.dtd">
<map>
  <title>Getting Started</title>
  <topicref href="../../concepts/what-is-dita.dita" type="concept"/>
  <topicref href="../../tasks/install.dita" type="task"/>
</map>

```

Publishing with DITA Open Toolkit (DITA-OT) (common approach)

DITA-OT is the most widely used open-source engine for publishing DITA.

Basic commands (typical)

Generate HTML5:

```
dita -i guides/getting-started.ditamap -f html5 -o out/html
```

Generate PDF (toolchain dependent—usually FO-based):

```
dita -i guides/getting-started.ditamap -f pdf -o out/pdf
```

Repeatable builds with project files (recommended)

As doc sets grow, use project files to store:

- inputs
- outputs
- filters
- parameters

This makes builds consistent across machines/CI.

Where customization lives

Common customization layers include:

- Parameters (simple switches)
- CSS/HTML templates (HTML output)
- PDF plugins/themes (PDF output)
- Custom XSLT and DITA-OT plugins (advanced)

Practical advice: start with configuration + small overrides, then grow into plugins only when needed.

Authoring workflow: how teams actually work with DITA

Folder structure (simple and scalable)

A common approach:

```
dita/
  maps/
    guides/
      getting-started.ditamap
  topics/
    concepts/
      what-is-dita.dita
    tasks/
      install.dita
    reference/
  reuse/
    common-notes.dita
  media/
    images/
  ditaval/
    windows.ditaval
    linux.ditaval
  out/
```

Content development lifecycle (CDLC)

Many teams use a lifecycle like:

1. Plan IA (maps, deliverables, variants)
2. Author topics (semantics and structure)
3. Reuse and link (conref, keys)
4. Validate (XML validity + broken refs)
5. Publish (local + CI)
6. Review (SME + editorial)
7. Release and maintain

CCMS vs file-based DITA

- **File-based:** Git + DITA files, fast for engineers/DevOps-heavy teams
- **CCMS:** workflow, reuse management, metadata, translations, governance at enterprise scale

DITA works in both models; the difference is management tooling.

Best practices (what prevents pain later)

Write small topics with a single purpose

- Don't write "chapters" as a single topic
- Prefer "one user goal = one topic"

Use keys for anything that changes across deliverables

- Product names, external URLs, reused references
- This keeps topics clean and portable

Be deliberate about conref

- Great for stable reusable fragments
- Overuse can make content hard to read and maintain
- Consider “reuse boundaries”: warnings, UI strings, legal text, standard steps

Standardize attribute vocabularies

- Define allowed values for platform/audience/product
- Enforce via review and/or constraints

Establish naming conventions

- Topic IDs and file names should be stable
- Use predictable IDs for conref targets

Validation and build hygiene

- Always validate during CI:
 - well-formed XML - schema/DTD validation (as applicable) - broken links/conrefs/keys

Common pitfalls (and how to avoid them)

- **Giant topics** # split into focused topics; assemble via maps
- **Hard-coded product names/URLs** # use keys ([keyref](#))
- **Conref spaghetti** # limit conref usage; centralize shared content; document ownership
- **Uncontrolled filtering values** # standardize and govern values
- **Specialization too early** # start with base DITA; specialize only with clear requirements

Quick-start checklist

1. Decide your deliverables (HTML? PDF? both?)
2. Define a starter map set (Getting Started, User Tasks, Reference)
3. Author 5–10 topics using concept/task/reference
4. Introduce keys for product name + URLs
5. Add one reuse file ([reuse/common-notes.dita](#)) and use a couple of conrefs
6. Set up one filtering example (platform=windows/linux) + build it both ways
7. Automate publishing via a script or CI

Next steps (if you want to go deeper)

If you're aiming for “professional-level DITA” adoption, the next major skills are:

- Map architecture patterns (navigation, reuse strategy)
- Keys + key scopes for multi-publication reuse
- DITaval profiling strategy and governance
- Output customization (HTML/CSS, PDF theming, plugins)
- Translation workflows (conref/keyref stability matters a lot)

Analytics and Instrumentation

Track which introduction topics are most viewed and where users drop off. Use analytics to improve the onboarding experience.

Personalization and Conditional Processing

Use conditional content to tailor the introduction for different backgrounds (writer, developer, manager).

Integrating with Conversational AI and NLP

Enable users to ask a chatbot “What is DITA?” and receive context-aware answers from this topic.

Continuous Improvement and Feedback Loops

Add a prompt for users to rate the introduction and suggest improvements. Use analytics and feedback to refine onboarding content.

Related tasks

[Quick Start Guide](#) on page 22

This quick start guide helps you navigate the documentation collection and begin working with DITA immediately.

Related reference

[Glossary](#) on page 190

Comprehensive DITA and DITA-OT terminology used throughout the Extense training documentation. Each term includes definitions, abbreviations, usage guidance, and scope notes for chatbot knowledge retrieval.

DITA Deep Dive

If you've ever seen a documentation program struggle with:

What DITA is (in practical terms)

Darwin Information Typing Architecture (DITA) is a standard for writing content as **structured XML topics** that can be **reused**, **assembled**, and **published** to multiple formats (HTML, PDF, help portals, etc.). The “architecture” part is important: DITA is not just a tag set. It is a way to design content so it can evolve and scale.

If you've ever seen a documentation program struggle with:

- repeated copy#paste content,
- inconsistent terminology,
- many products/variants,
- many output formats,

- frequent releases and updates,
- multiple teams editing at once,

...DITA exists to reduce those problems by shifting from “write a big document” to “author modular components and assemble them.”

The DITA mindset: topics + maps + reuse + publishing

Almost everything in DITA makes sense if you keep this mental picture:

1. You author **topics** (small, reusable information units).
2. You assemble topics into deliverables using **maps** (a map is the publishing “spine”).
3. You reuse content using **references** (conref, keyref, conkeyref, etc.).
4. You publish through a toolchain (often DITA#OT, sometimes a CCMS with its own engine).
5. You can evolve the model through **specialization** (the “Darwin” aspect).

DITA is designed for teams who want their content to behave like well-engineered software:

- stable interfaces (IDs and reuse targets),
- modular design,
- controlled vocabulary,
- repeatable builds,
- automated validation.

Why “Darwin” is in the name

“Darwin” signals **adaptation**. You can extend DITA to match your domain without abandoning the ecosystem. In practice, this means you can:

- specialize elements to represent domain concepts,
- create constraints that enforce standards,
- keep the base semantics recognizable so tools can still process it.

This is not “invent your own XML from scratch.” It’s “extend a known model safely.”

Information typing: concept, task, reference

DITA popularized the idea that different user intents deserve different structures:

- **Concept** topics: explain what something is and why it matters.
- **Task** topics: explain how to do something step#by#step.
- **Reference** topics: list facts, fields, values, APIs, commands, parameters, tables.

Why this matters:

- It drives consistency (every task has prerequisites, steps, results).
- It helps readers find information faster (users know what to expect).
- It improves automation (transforms can format tasks differently than reference).

A DITA program that ignores information typing usually ends up with “generic topics” that are hard to maintain and publish consistently.

Core components you'll see in real DITA programs

Topics

Topics are files like:

- `install.dita` (task)
- `what-is-dita.dita` (concept)
- `cli-options.dita` (reference)

A topic typically includes:

- a stable `id`,
- a `<title>`,
- a body (taskbody/conbody/refbody),
- optional metadata in `<prolog>`.

Maps

Maps are files like:

- `getting-started.ditamap`
- `admin-guide.ditamap`
- `api-reference.ditamap`

A map:

- defines the publication structure (TOC),
- defines keys and sometimes scope,
- can define relationships,
- can define conditional processing rules indirectly (via build config).

Metadata

DITA supports structured metadata:

- authorship and versioning,
- product/platform/audience attributes,
- keywords and taxonomy,
- short descriptions and abstracts.

Metadata is not decoration. In an enterprise environment it drives:

- search relevance,
- portal faceting,
- translation segmentation,
- governance and review workflows.

Reuse mechanisms

DITA's reuse is not just “copy content.” It includes:

- **conref**: reuse content by referencing a source element.
- **keyref**: refer to keys defined in a map (late binding).
- **conkeyref**: pull content via keys (flexible reuse across contexts).
- **content references via maps** (topic reuse by including the same topic in multiple maps).

Conditional processing (filtering)

DITA supports publishing different variants:

- Windows vs Linux,
- Basic vs Pro,
- Internal vs External,
- Region#specific or customer#specific.

This is typically done using:

- attributes in the source (e.g., `platform="windows"`),
- filtering rules in build config (e.g., DITaval in DITA#OT).

Specialization and constraints

Advanced programs define:

- specialization to represent domain semantics,
- constraints to restrict the model and reduce variability.

Constraints are often more important than specialization because they control author behavior.

When DITA is a great fit

DITA shines when you have at least one of these:

- **Multiple deliverables** from a single source (HTML + PDF + portal).
- **High reuse** (warnings, procedures, UI strings, legal statements).
- **Complex products** with many settings and options.
- **Rapid releases** with continuous updates.
- **Multiple product variants** requiring conditional content.
- **Large author teams** where structure reduces inconsistency.

If you have a single small doc set with minimal reuse and only HTML output, DITA can be more effort than it's worth—unless your organization already runs a DITA platform and needs standardization.

DITA vs. alternatives (practical comparison)

DITA vs Unstructured Formats

Lightweight formats like Markdown or flat HTML are fast to write. DITA is structured and scalable.

Unstructured formats are often ideal for:

- developer docs close to code,
- small teams,
- fast iteration with minimal publishing complexity.

DITA is often ideal for:

- complex product documentation,
- multi#format deliverables,
- controlled reuse and variants,
- enterprise governance.

A hybrid strategy is common:

- engineers write in lightweight formats for dev-facing docs,
- tech pubs writes in DITA for user-facing manuals and regulated outputs,
- publishing pipelines unify the experience in a portal.

DITA vs DocBook

Both are XML-based publishing systems. DITA is topic-oriented and reuse-centric; DocBook is traditionally book/chapter oriented (though you can do modular structures). DITA's key systems and specialization model are a differentiator for large reuse programs.

DITA vs S1000D

S1000D is a defense/aerospace standard with strict structures, data modules, and business rules. DITA is more flexible and widely used across industries. Organizations sometimes map between them, but they are not interchangeable.

How DITA programs are organized (the “content system” view)

A successful DITA adoption is as much **operational design** as it is technology. Typical components:

1. Content model

- which topic types you use, - which domains are enabled, - what constraints exist, - how IDs and reuse targets are structured.

1. Information architecture

- map patterns, - navigation approach, - link strategy, - modularization and reuse boundaries.

1. Toolchain

- authoring (XML editor, CCMS), - validation and build automation, - publishing engine and customization, - review workflows.

1. Governance

- naming conventions, - controlled vocabularies for filtering, - content ownership for shared snippets, - change control for keys and reuse targets.

If you skip governance, DITA often becomes “XML chaos” instead of “structured content.”

A realistic first DITA project (what success looks like)

A good initial DITA project is not “convert the entire manual.” Instead, aim for a small deliverable with clear reuse potential:

- Build a **Getting Started** guide:
 - 10–25 topics, - a simple map, - a small reuse library (warnings, standard steps), - a key system for product names/URLs.

Then add complexity gradually:

- Introduce filtering for platform variants.
- Introduce project files (repeatable builds).
- Introduce PDF styling and HTML theming.

- Introduce more formal taxonomy (subject scheme).

Common misconceptions (and the reality)

“DITA is just XML tags”

DITA is an ecosystem: modeling, reuse, build pipelines, and specialization.

“DITA automatically makes content reusable”

DITA provides mechanisms, but reuse requires design:

- stable IDs,
- consistent chunking,
- disciplined authoring,
- governance.

“You must use a CCMS to use DITA”

You can start with files + Git + DITA#OT. A CCMS becomes valuable at scale for workflow, translation management, reuse governance, and metadata enforcement.

“Specialization is mandatory”

Most teams succeed using base DITA plus constraints and controlled vocabularies. Specialization is powerful but best introduced after you can publish and maintain reliably.

What “good DITA” looks like

Good DITA content is:

- modular (each topic has a single purpose),
- semantically consistent (use task for procedures),
- reusable (common content is referenced, not copied),
- publishable (builds are repeatable and validated),
- governed (attributes and keys are controlled).

A quick self-check:

- Can you reuse the same topic in two different maps without editing it?
- Can you publish two product variants using filtering?
- Can you change a product name in one place (keys) and have it update everywhere?
- Can your build fail fast when references are broken?

If the answer is “yes,” you’re doing DITA the way it was designed.

Analytics and Instrumentation

Track which overview topics are most viewed and where users drop off. Use analytics to improve the onboarding experience.

Personalization and Conditional Processing

Use conditional content to tailor the overview for different backgrounds (writer, developer, manager).

Integrating with Conversational AI and NLP

Enable users to ask a chatbot “What is DITA?” and receive context-aware answers from this topic.

Continuous Improvement and Feedback Loops

Add a prompt for users to rate the overview and suggest improvements. Use analytics and feedback to refine onboarding content.

Installing DITA-OT

Download, install, and verify DITA Open Toolkit 4.x on your machine.

After completing this chapter, you will be able to:

- Install DITA-OT 4.x on Windows, macOS, or Linux
- Verify the installation works correctly
- Run your first DITA build

Prerequisites:

- **Java 17 or later** — DITA-OT 4.x requires Java 17+. Check with `java -version`.
- **500 MB free disk space** for the toolkit and build artifacts.
- **Network access** to download the release archive.

DITA-OT is a free, open-source publishing engine that transforms DITA XML into HTML5, PDF, and other formats. It is the standard toolchain for DITA publishing.

1. Verify Java is installed and meets the minimum version.

```
java -version
```

You should see output like `openjdk version "17.0.x"` or higher. If Java is not installed or is too old:

- **macOS:** `brew install openjdk@17`
- **Ubuntu/Debian:** `sudo apt install openjdk-17-jdk`
- **Windows:** Download from adoptium.net

2. Download the latest DITA-OT release.

Visit dita-ot.org/download and download the zip archive, or use the command line:

```
# Replace 4.3.5 with the latest version
curl -LO https://github.com/dita-ot/dita-ot/releases/download/4.3.5/dita-ot-4.3.5.zip
```

3. Extract the archive to a permanent location.

```
# Linux / macOS
unzip dita-ot-4.3.5.zip -d /opt/
# The toolkit is now at /opt/dita-ot-4.3.5/

# Windows (PowerShell)
Expand-Archive dita-ot-4.3.5.zip -DestinationPath C:\dita-ot
```

4. Add the DITA-OT bin/ directory to your PATH.

```
# Linux / macOS – add to ~/.bashrc or ~/.zshrc
export DITA_HOME=/opt/dita-ot-4.3.5
export PATH="$DITA_HOME/bin:$PATH"

# Apply immediately
source ~/.bashrc
```

```
# Windows PowerShell – add to profile
$env:DITA_HOME = "C:\dita-ot\dita-ot-4.3.5"
$env:PATH = "$env:DITA_HOME\bin;$env:PATH"
```

5. Verify the installation.

```
dita --version
```

Expected output:

```
DITA-OT version 4.3.5
```

Also list installed plugins to confirm a working installation:

```
dita --plugins
```

You should see a list including [org.dita.html5](#), [org.dita.pdf2](#), and others.

6. Run a test build using the included samples.

```
# Build the included sample to HTML5
dita -i $DITA_HOME/docsrc/samples/sequence.ditamap -f html5 -o /tmp/dita-test-output
```

Open `/tmp/dita-test-output/index.html` in a browser to verify the output.

DITA-OT is installed and working. You can now build any DITA map into HTML5 or PDF output.

Troubleshooting:

- **dita: command not found** — Ensure `$DITA_HOME/bin` is on your PATH and restart your terminal.
- **JAVA_HOME is not set** — Set `JAVA_HOME` to your JDK installation directory.
- **Unsupported class file major version** — Your Java version is too old. DITA-OT 4.x requires Java 17+.
- **Build succeeds but output is empty** — Check that your map file path is correct and the map references valid topics.

Next step: [Set up your authoring tools](#), then move on to [Authoring DITA Topics](#).

Setting Up Authoring Tools

Install and configure Oxygen XML Editor for productive DITA authoring with schema validation, visual editing, and built-in DITA-OT publishing.

After completing this chapter, you will be able to:

- Install and configure Oxygen XML Editor for DITA authoring
- Use Author mode for WYSIWYG-style DITA editing
- Validate DITA topics against the schema in real time
- Publish directly from Oxygen using DITA-OT transformation scenarios
- Set up Git for version-controlled DITA projects

Prerequisites: [DITA-OT installed](#) and working.

Oxygen XML Editor is the industry-standard authoring tool for DITA. It provides schema-aware editing, WYSIWYG Author mode, a visual DITA map editor, and built-in DITA-OT integration — everything a technical writer needs in a single application.

Oxygen comes in two editions:

- **Oxygen XML Editor** — Full-featured edition with Text, Author, and Grid views plus XSLT/XQuery debugging. Ideal for developers and content engineers.
- **Oxygen XML Author** — Streamlined edition focused on Author (WYSIWYG) mode. Ideal for technical writers who prefer visual editing.

1. Download and install Oxygen XML Editor.

Go to oxygenxml.com and download the installer for your platform (Windows, macOS, or Linux). Run the installer and follow the prompts.

Oxygen includes a bundled JRE, so you do not need to install Java separately for the editor itself.

Tip

Oxygen offers a 30-day free trial. Academic and volume licenses are also available.

2. Configure DITA preferences.

Open **Options # Preferences # DITA** and verify:

- **DITA Open Toolkit:** Point to your installed DITA-OT directory (e.g., `~/dita-ot-4.2`). Oxygen bundles its own DITA-OT, but using your project's version ensures consistent output.
- **Default DITA-OT transtype:** Set to `html5` for quick preview builds.
- **Show DITA-OT console output:** Enable this to see build logs directly in Oxygen.

3. Switch to Author mode for visual DITA editing.

Open any DITA topic and click the **Author** tab at the bottom of the editor (next to **Text** and **Grid**).

Author mode renders your DITA content visually — similar to a word processor — while maintaining the underlying XML structure. Key features:

- **Content Completion Assistant:** Press `Enter` inside an element to see valid child elements. Oxygen only offers elements allowed by the DITA DTD at the cursor position.
- **Inline validation:** Invalid structure is underlined in red with a description of the problem.
- **Reuse actions:** Right-click # **Reuse** to insert conrefs, keyrefs, or content key references.
- **Attribute editor:** The Attributes panel (right side) shows all valid attributes for the selected element.

4. Create a new DITA topic using Oxygen templates.

Select **File # New** (or press `Ctrl+N`). In the New Document wizard:

- a. Expand **Framework templates # DITA**
- b. Choose a topic type: **Concept**, **Task**, or **Reference**
- c. Enter the file name and click **Create**

Oxygen generates a valid skeleton with the correct DOCTYPE, root element, title, and body element — ready for you to fill in.

5. Open your DITA map in the DITA Maps Manager.

Open your `.ditamap` file. Oxygen displays it in the **DITA Maps Manager** panel (usually on the left), showing the topic hierarchy as a tree.

From the Maps Manager you can:

- **Drag and drop** topics to reorder them
- **Right-click # Insert Reference** to add new topicrefs
- **Double-click** any topic to open it for editing
- **Validate** the entire map (check for broken references, missing topics)

6. Set up a transformation scenario for publishing.

With your ditamap open, click the **Configure Transformation Scenario(s)** button (wrench icon) or press `Ctrl+Shift+T`.

- a. Click **New # DITA-OT Transformation**
- b. Set **Transtype:** `html5`
- c. Set **Output directory:** `${pd}/out/html5` (`${pd}` is the project directory)
- d. Optionally add parameters: `nav-toc=full` for navigation
- e. Click **OK**, then **Apply Associated** to run the build

The output opens automatically in your browser. You can re-run the same scenario with `Ctrl+Shift+T # Apply Associated`.

7. Validate your topics and map.

Oxygen validates DITA automatically as you type. You can also run manual validation:

- **Single topic:** Press `Ctrl+Shift+V` to validate the current file against its DTD
- **Entire map:** In the DITA Maps Manager, click **Validate and Check for Completeness**. This checks all referenced topics for broken links, missing conrefs, and structural errors.

8. Initialize Git version control for your DITA project.

Oxygen includes a Git client plugin. Alternatively, use the command line:

```
cd myproject
git init
```

```
# Create a .gitignore for DITA projects
cat > .gitignore <<'EOF'
out/
*.log
.DS_Store
temp/
EOF

git add .
git commit -m "Initial DITA project structure"
```

In Oxygen, enable the Git client from **Window # Show View # Git Staging** to commit, push, and pull directly from the editor.

Your Oxygen XML authoring environment is configured. You have schema-aware DITA editing with Author mode, a visual map editor, transformation scenarios for publishing, validation, and Git version control.

Alternative editors:

- **Visual Studio Code** — Free, open-source editor with XML support via the Red Hat XML extension. Good for developers who prefer lightweight tooling. Requires manual snippet setup for DITA templates.
- **Any text editor** — DITA is plain XML. Vim, Emacs, Sublime Text, and Notepad++ all work; you lose auto-complete, inline validation, and Author mode.

Next step: [Authoring DITA Topics](#)

Part



Authoring DITA Content

Topics:

- [Authoring DITA Topics](#)
- [Writing Your First DITA Project](#)
- [Information Typing: Matching Content to Purpose](#)
- [Information Typing for AI: Intent Routing and Answer Formatting](#)
- [Reuse, Keys, and Variant Management](#)

Authoring DITA Topics

Writing DITA is not just writing in XML. It's writing with:

What “authoring in DITA” really means

Writing DITA is not just writing in XML. It's writing with:

- a **content model** (what elements are allowed where),
- a **purpose** (concept vs task vs reference),
- a **reuse strategy** (what stays local vs shared),
- a **publishing strategy** (HTML/PDF/portal),
- a **governance strategy** (consistent attributes, IDs, terminology).

When DITA authoring goes wrong, it's usually not because XML is hard. It's because the team never agreed on:

- topic size,
- naming conventions,
- metadata patterns,
- reuse boundaries,
- filtering rules.

This topic gives you a practical, “how real teams do it” guide.

Topic design: picking the right topic type

Concept topics (what/why)

Use concept topics to explain:

- definitions and mental models,
- background and rationale,
- architectural overview,
- decisions and tradeoffs,
- “when to use” and “when not to use.”

A good concept topic:

- answers a single question,
- avoids procedural steps,
- uses examples sparingly and purposefully.

Common elements:

- `<title>`
- `<shortdesc>` (optional but often valuable)
- `<conbody>` with `<p>`, lists, sections as needed

Task topics (how)

Use tasks to instruct:

- installation steps,
- configuration procedures,
- workflows and checklists,
- troubleshooting actions (when they are step-based).

A good task topic:

- begins with prerequisites and context,
- has clear, testable steps,
- distinguishes commands/UI actions,
- ends with results or next steps.

Common elements:

- `<taskbody>`
- `<prereq>`, `<context>`, `<steps>`, `<result>`, `<postreq>`

Reference topics (facts/options)

Use reference topics for:

- command syntax,
- configuration parameters,
- API endpoint specs,
- field definitions,
- error code tables.

A good reference topic:

- is scannable,
- uses tables consistently,
- avoids long narrative explanations,
- includes examples where needed.

The most important authoring decision: topic scope (“chunking”)

Why chunking matters

Chunking is how you decide what is:

- one topic,
- a section inside a topic,
- a set of topics assembled by a map.

Chunking affects:

- reuse,
- translation cost,
- search relevance,
- link stability,
- maintenance.

Practical chunking guidelines

Make a topic when:

- the content solves one user goal,
- the content can be reused elsewhere,
- the content could appear in search results on its own.

Make a section (not a new topic) when:

- the content is only meaningful in the same context,
- splitting would create “orphan topics” with unclear purpose.

Split a topic if:

- it has multiple distinct goals,
- it becomes too long to scan,
- it mixes concept and procedure heavily,
- it gets reused partially (a sign you need smaller building blocks).

A simple heuristic

If a topic title needs “and” or “then” often, consider splitting:

- “Install and Configure the System” # likely two tasks or a concept + task.
- “What It Is and How to Use It” # concept + task.

IDs, filenames, and stability (treat your content like an API)

Why IDs matter more than you think

DITA references rely on IDs:

- conref targets,
- xrefs,
- map references,
- CCMS reuse references.

If IDs change frequently, you break:

- reuse,
- links,
- downstream systems and portals.

Recommended patterns**File names**

- Use lowercase with hyphens: `install-windows.dita`
- Keep names stable over time
- Avoid “v2” or dates unless truly necessary

Topic IDs

- Match the filename or a stable semantic label:
- file: `install.dita` - topic id: `install`
- Avoid auto-generated IDs like `topic_1234` if you can

Element IDs for reuse targets

- Use meaningful IDs for reusable blocks:

- `install-note` - `admin-warning` - `prereq-java`

Stability rule

- Treat changing an ID like changing a public API:
- do it rarely, - communicate it, - update dependent references, - validate builds.

Writing style in DITA (structure first, prose second)

DITA is not a prose-first system. It's structure-first:

- the structure communicates intent to both humans and machines,
- the prose fills in the details.

Task writing best practices

- Use imperative verbs: "Click", "Select", "Run", "Verify".
- One action per step.
- Put expected results in `<result>` or a final step.
- Separate UI and command line procedures if they diverge significantly.
- Use `<uicontrol>`, `<menucascade>`, `<filepath>`, `<cmdname>` when available (domain-dependent).

Concept writing best practices

- Start with a crisp definition or summary.
- Use short paragraphs.
- Use examples and diagrams deliberately.
- Avoid hidden tasks ("Do X, do Y") unless it's truly conceptual.

Reference writing best practices

- Use consistent tables for parameters/options.
- Use stable ordering (alphabetical or logical grouping).
- Provide defaults, constraints, and examples.

Practical authoring templates (copy/paste patterns)

Concept template

```
<concept id="...">
  <title>...</title>
  <shortdesc>One-sentence summary for search and previews.</shortdesc>
  <conbody>
    <p>Definition and context.</p>
    <p>Key idea #1.</p>
    <p>Key idea #2.</p>
    <section>
      <title>Example</title>
      <p>Use an example to clarify.</p>
    </section>
  </conbody>
```

```
</concept>
```

Task template

```
<task id="...">
  <title>...</title>
  <shortdesc>What this procedure achieves.</shortdesc>
  <taskbody>
    <prereq>
      <p>Conditions and required permissions.</p>
    </prereq>
    <context>
      <p>When and why you do this.</p>
    </context>
    <steps>
      <step><cmd>Do step 1.</cmd></step>
      <step><cmd>Do step 2.</cmd></step>
      <step><cmd>Verify the result.</cmd></step>
    </steps>
    <result>
      <p>What success looks like.</p>
    </result>
  </taskbody>
</task>
```

Reference template

```
<reference id="...">
  <title>...</title>
  <shortdesc>What this reference covers.</shortdesc>
  <refbody>
    <section>
      <title>Syntax</title>
      <codeblock>command --option value</codeblock>
    </section>
    <section>
      <title>Options</title>
      <simplatable>
        <sthead>
          <stentry>Option</stentry>
          <stentry>Description</stentry>
        </sthead>
        <strow>
          <stentry><codeph>--help</codeph></stentry>
          <stentry>Displays help text.</stentry>
        </strow>
      </simplatable>
    </section>
  </refbody>
</reference>
```

Authoring for reuse: designing “reusable chunks”

Reuse is not free. It requires deliberate boundaries.

What should be reusable?

- warnings, cautions, safety notes,
- prerequisites and environment statements,
- legal/compliance text,

- UI labels and product naming,
- standard procedures used across many tasks,
- shared troubleshooting steps,
- consistent definitions.

What should usually NOT be reused?

- highly contextual narrative,
- content that changes often,
- content that's meaningful only in one location,
- deep cross-referenced paragraphs that assume surrounding text.

“Reusable chunk” rules

A reusable chunk should:

- make sense when read alone,
- avoid “above/below” references,
- avoid map-specific context,
- be stable over time.

If it depends on context, keep it local.

Metadata: the overlooked authoring power tool

Metadata improves:

- search relevance,
- portal navigation,
- translation handling,
- analytics and reporting,
- content governance.

Useful metadata patterns

- `<shortdesc>` for topic previews.
- `<prolog>` for author, version, review status.
- `<keywords>` for taxonomy and search.
- product/platform attributes for filtering.

Governance tip

Create a “metadata minimum standard”:

- every topic must have `<shortdesc>` and at least one keyword,
- every task must include prerequisites if relevant,
- every reference must include syntax and examples.

These standards reduce variability.

Images, UI screenshots, and media management

Image authoring best practices

- Keep images in a dedicated folder: `media/images/`.
- Use consistent naming: `ui-settings-panel.png`.
- Use alt text and/or text equivalents where required.
- Prefer vector (SVG) where your pipeline supports it and where clarity matters.

Screenshot governance

Screenshots are expensive to maintain. DITA doesn't solve that automatically. To reduce screenshot maintenance:

- minimize screenshot dependence,
- use callouts sparingly,
- document UI actions in text with `<uicontrol>` elements,
- standardize screenshot production workflow.

Translation and localization implications

DITA's structure can reduce translation cost, but only if you:

- reuse content instead of duplicating it,
- keep topics stable,
- avoid rewriting shared snippets frequently,
- keep IDs stable (for reuse references).

Practical localization tips

- Don't embed locale-specific text in shared snippets unless that's intentional.
- Use keys for product names and terms that differ by region.
- Avoid hard-coded date formats and culturally specific examples in globally reused topics.

Common authoring mistakes (and how to fix them)

Mistake 1: "Generic topics everywhere"

Fix: adopt information typing (concept/task/reference) and enforce it via review.

Mistake 2: "Giant topics"

Fix: split into smaller topics and assemble with maps.

Mistake 3: "Conref everywhere"

Fix: reuse only stable, context-independent chunks; document ownership and change control.

Mistake 4: "Uncontrolled attributes"

Fix: define a controlled vocabulary and enforce it (subject scheme + process).

Mistake 5: "IDs keep changing"

Fix: treat IDs as stable contracts; manage renames with care.

A practical authoring workflow for teams

Here's a workflow that works for both file-based and CCMS environments:

1. **Plan:** create map skeleton (TOC) and topic list.
2. **Draft:** create topic stubs with titles and shortdesc.
3. **Fill:** author content following templates.
4. **Review:** SME + editorial; check structure and semantics.
5. **Reuse pass:** consolidate repeated content into reuse library.
6. **Metadata pass:** keywords, attributes, taxonomy.
7. **Validate:** run builds and resolve warnings/errors.
8. **Publish:** produce HTML/PDF; confirm navigation and search behavior.
9. **Maintain:** version and update with stable IDs and reuse targets.

Action checklist (do this to get productive quickly)

- [] Choose topic types you will use (start with concept/task/reference).
- [] Define naming conventions for files and IDs.
- [] Create templates and enforce them (even if informally at first).
- [] Establish reuse boundaries and a reuse folder.
- [] Define attribute vocabulary for platform/audience/product.
- [] Run validation on every change (locally or in CI).
- [] Keep topics small and stable; assemble with maps.

Analytics and Instrumentation

Track authoring patterns and template usage with analytics. Monitor which templates are most used and where authors encounter issues. Use this data to improve authoring workflows and training.

Personalization and Conditional Processing

Use DITA conditional attributes to tailor authoring guidance for different roles or workflows. Example:

```
<section audience="editor">Editor-specific guidance here.</section>
```

Integrating with Conversational AI and NLP

Expose authoring best practices to chatbots or virtual assistants. Enable users to ask for authoring help and receive context-aware answers from this topic.

Continuous Improvement and Feedback Loops

Collect feedback from authors and editors on templates and workflows. Use analytics and feedback to refine authoring guidance and improve documentation quality.

Related tasks

[Reuse, Keys, and Variant Management](#) on page 66

For many organizations, multi-format output is helpful—but reuse is what changes the economics of documentation. Reuse reduces:

Related reference

[Glossary](#) on page 190

Comprehensive DITA and DITA-OT terminology used throughout the Extense training documentation. Each term includes definitions, abbreviations, usage guidance, and scope notes for chatbot knowledge retrieval.

Writing Your First DITA Project

Build a complete documentation project from scratch — create three topic types, a map with keys, a conref library, and publish to HTML5.

After completing this chapter, you will be able to:

- Create concept, task, and reference topics from scratch
- Build a DITA map with key definitions
- Use conref to share reusable content
- Use keyref for product names
- Build HTML5 output with sidebar navigation

Prerequisites:

- [DITA-OT installed](#) and on your PATH
- An XML editor configured ([Setting Up Authoring Tools](#))
- You have read the preceding [Authoring DITA Topics](#) chapter (or at least the templates section)

Time estimate: 45–60 minutes

This is a hands-on, end-to-end tutorial. You will create a small "Getting Started" guide for a fictional product called **WidgetPro**. By the end, you will have a working DITA project that demonstrates all core DITA concepts.

1. Create the project directory structure.

```
mkdir -p widgetpro-docs/{maps,topics/concepts,topics/tasks,topics/reference,reuse,out}
```

Your project tree:

```
widgetpro-docs/  
  maps/  
  topics/  
    concepts/  
    tasks/  
    reference/  
  reuse/  
  out/
```


2. Create a concept topic: topics/concepts/what-is-widgetpro.dita

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">
<concept id="what-is-widgetpro">
  <title>What Is <ph keyref="product-name"/>?</title>
  <shortdesc><ph keyref="product-name"/> is a widget management platform for teams.</shortdesc>
  <conbody>
    <p><ph keyref="product-name"/> helps you create, organize, and deploy widgets across multiple environments.</p>
    <section id="key-benefits">
      <title>Key benefits</title>
      <ul>
        <li>Centralized widget management</li>
        <li>Multi-environment deployment</li>
        <li>Version control built in</li>
        <li>Role-based access control</li>
      </ul>
    </section>
  </conbody>
</concept>
```

Tip

New syntax explained:

- `<!DOCTYPE concept ...>` — This line tells the XML processor which topic type this file uses. The word after DOCTYPE (concept, task, or reference) must match your root element. You can copy this line from a template.
- `<ph keyref="product-name"/>` — This is a **key reference**. Instead of typing "WidgetPro" directly, you reference a key defined in the map. If the product is rebranded, you change it once in the map — not in every topic.

Key learning: Notice `<ph keyref="product-name"/>` — this pulls the product name from the map. You never hard-code product names in topics.

3. Create a task topic: topics/tasks/install-widgetpro.dita

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA Task//EN" "task.dtd">
<task id="install-widgetpro">
  <title>Installing <ph keyref="product-name"/></title>
  <shortdesc>Download and install <ph keyref="product-name"/> on your machine.</shortdesc>
  <taskbody>
    <prereq>
      <note conref="../../reuse/shared-notes.dita#shared-notes/admin-warning"/>
      <ul>
        <li>Operating system: Windows 10+, macOS 12+, or Ubuntu 22+</li>
        <li>1 GB free disk space</li>
      </ul>
    </prereq>
    <context>
      <p>This procedure installs <ph keyref="product-name"/> using the command-line installer.</p>
    </context>
    <steps>
      <step>
        <cmd>Download the installer from <xref keyref="download-url"/>.</cmd>
      </step>
      <step>
```

```

    <cmd>Run the installer.</cmd>
    <info>
      <codeblock outputclass="language-bash">./widgetpro-install.sh</codeblock>
    </info>
  </step>
  <step>
    <cmd>Verify the installation.</cmd>
    <info>
      <codeblock outputclass="language-bash">widgetpro --version</codeblock>
      <p>Expected output: <codeph>WidgetPro 3.0.0</codeph></p>
    </info>
  </step>
</steps>
<result>
  <p><ph keyref="product-name"/> is installed and ready to use.</p>
</result>
</taskbody>
</task>

```

Key learning: Notice the `conref` pulling a shared warning and `keyref` for the download URL.

Tip

New syntax explained:

- `conref="../../reuse/shared-notes.dita#shared-notes/admin-warning"` — A **content reference**. This pulls content from another file by pointing to a specific element's `id`. The format is `filepath#topic-id/element-id`. Think of it as an "include" that always stays in sync with the source.
- `<xref keyref="download-url"/>` — A cross-reference resolved via a key. The actual URL is defined in the map, not in the topic.

4. Create a reference topic: topics/reference/cli-options.dita

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE reference PUBLIC "-//OASIS//DTD DITA Reference//EN" "reference.dtd">
<reference id="cli-options">
  <title><ph keyref="product-name"/> CLI Options</title>
  <shortdesc>Command-line options for <ph keyref="product-name"/>.</shortdesc>
  <refbody>
    <section id="syntax">
      <title>Syntax</title>
      <codeblock>widgetpro [command] [options]</codeblock>
    </section>
    <section id="commands">
      <title>Commands</title>
      <simplatable>
        <sthead>
          <stentry>Command</stentry>
          <stentry>Description</stentry>
        </sthead>
        <strow>
          <stentry><codeph>init</codeph></stentry>
          <stentry>Initialize a new widget project</stentry>
        </strow>
        <strow>
          <stentry><codeph>build</codeph></stentry>
          <stentry>Build widgets for deployment</stentry>
        </strow>
        <strow>
          <stentry><codeph>deploy</codeph></stentry>
          <stentry>Deploy widgets to target environment</stentry>
        </strow>
        <strow>
          <stentry><codeph>--version</codeph></stentry>

```

```

        <stentry>Display version information</stentry>
      </strow>
      <strow>
        <stentry><codeph>--help</codeph></stentry>
        <stentry>Display help text</stentry>
      </strow>
    </simplettable>
  </section>
</refbody>
</reference>

```

Key learning: Reference topics are scannable. Use tables consistently for parameters, commands, and options.

5. Create a reuse library: reuse/shared-notes.dita

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE topic PUBLIC "-//OASIS//DTD DITA Topic//EN" "topic.dtd">
<topic id="shared-notes">
  <title>Shared Notes Library</title>
  <body>
    <note id="admin-warning" type="warning">You must have administrator
      privileges to perform this operation.</note>
    <note id="backup-reminder" type="tip">Always back up your configuration
      before making changes.</note>
  </body>
</topic>

```

Key learning: Each reusable element has a stable `id`. Topics pull them with `conref="reuse/shared-notes.dita#shared-notes/admin-warning"`.

6. Create the map: maps/widgetpro-guide.ditamap

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN" "map.dtd">
<map id="widgetpro-guide">
  <title>WidgetPro Getting Started Guide</title>

  <!-- Key definitions – change these once, updates everywhere -->
  <keydef keys="product-name">
    <topicmeta><keywords><keyword>WidgetPro</keyword></keywords></topicmeta>
  </keydef>
  <keydef keys="download-url" href="https://example.com/widgetpro/download"
    scope="external" format="html"/>

  <!-- Table of contents -->
  <topicref href="../topics/concepts/what-is-widgetpro.dita"/>
  <topicref href="../topics/tasks/install-widgetpro.dita"/>
  <topicref href="../topics/reference/cli-options.dita"/>
</map>

```

Key learning: The map defines `product-name` and `download-url` as keys. Topics use `keyref` to resolve them. To rebrand for a different product, change the map — not the topics.

7. Build HTML5 output.

```

cd widgetpro-docs
dita -i maps/widgetpro-guide.ditamap -f html5 -o out/html5 --nav-toc=full

```

8. Open the output and verify everything works.

- a) Open `out/html5/index.html` in a browser.

- b) Verify the product name shows "WidgetPro" (from keyref).
- c) Verify the install topic shows the admin warning (from conref).
- d) Verify the reference topic has a formatted command table.
- e) Verify the sidebar TOC shows all three topics.

9. Try these experiments to deepen your understanding.

Experiment A — Change the product name:

- a. In the map, change `<keyword>WidgetPro</keyword>` to `<keyword>GizmoMax</keyword>`.
- b. Rebuild. Verify every page now says "GizmoMax" — without editing any topic file.

Experiment B — Add a second conref:

- a. Add the `backup-reminder` note from your reuse library to the install topic's `<postreq>`.
- b. Rebuild and verify it appears.

Experiment C — Add platform filtering:

- a. In the install topic, add two steps with `platform="windows"` and `platform="linux"`.
- b. Create a `build/windows.ditaval` that excludes `platform="linux"`.
- c. Build with `--filter=build/windows.ditaval` and verify only Windows content appears.

You have created a complete DITA project with three topic types, a map with key definitions, a conref reuse library, and HTML5 output. You have practiced the core DITA workflow: **author # organize # build # verify**.

What you learned:

- Concept, task, and reference topics have different structures for different purposes
- Maps define the TOC structure and key definitions
- Keys (`keyref`) make topics portable across deliverables
- Conref pulls shared content from a reuse library
- DITA-OT transforms XML into browsable HTML5

Next step: [Reuse, Keys & Variant Management](#) for advanced reuse patterns.

Information Typing: Matching Content to Purpose

DITA's three core topic types — concept, task, and reference — give each piece of content a clear purpose, making it easier to write, review, and maintain.

Why does information typing matter?

When you write content, every piece has a primary purpose. You are either:

- **Explaining** what something is or why it matters
- **Teaching** how to do something step by step
- **Providing** facts, specs, or options for lookup

DITA makes this purpose explicit by giving each topic type its own XML structure. This matters because:

- **Writers** get a template that guides what to include (prerequisites, steps, results) instead of guessing
- **Reviewers** can check that a task has all required steps, or that a concept actually explains the "why"
- **Readers** always know what to expect — a task topic always has steps, a reference topic always has lookup data
- **Tools** can validate, format, and filter content because the type is machine-readable

Concept topics: Explaining ideas

Use a concept topic when you need to answer "**what is...**" or "**why does...**" questions. Concepts use prose organized into sections.

Structure: `<concept>` # `<conbody>` # `<section>` elements with paragraphs, lists, and tables.

Example:

```
<concept id="what-is-reuse">
  <title>What is content reuse?</title>
  <shortdesc>Content reuse means writing a piece of content
    once and referencing it from multiple locations.</shortdesc>
  <conbody>
    <section id="why-reuse-matters">
      <title>Why does reuse matter?</title>
      <p>When the same safety warning appears in 12 manuals,
        a single-source approach ensures all 12 stay
        in sync when the warning changes.</p>
    </section>
  </conbody>
</concept>
```

When to use it: Background information, definitions, explanations of architecture or design decisions, "why we do it this way" content.

Task topics: Teaching procedures

Use a task topic when you need to answer "**how do I...**" questions. DITA tasks enforce a consistent structure so readers always know where to find prerequisites, steps, and expected results.

Structure: `<task>` # `<taskbody>` # `<prereq>` # `<context>` # `<steps>` # `<result>`

Example:

```
<task id="install-dita-ot">
  <title>Installing DITA-OT</title>
  <shortdesc>Download and verify DITA Open Toolkit.</shortdesc>
  <taskbody>
    <prereq>
      <ul><li>Java 17 or later installed</li></ul>
    </prereq>
    <steps>
      <step><cmd>Download from dita-ot.org.</cmd></step>
      <step><cmd>Unzip to your preferred location.</cmd></step>
      <step><cmd>Add bin/ to your PATH.</cmd></step>
    </steps>
    <result>
      <p>Run <code>dita --version</code> to
        confirm installation.</p>
    </result>
  </taskbody>
```

```
</task>
```

When to use it: Installation guides, configuration procedures, workflows, any "do these steps" content.

Reference topics: Lookup information

Use a reference topic when readers need to **look up** specific facts — parameter values, CLI options, error codes, configuration settings.

Structure: `<reference>` # `<refbody>` # tables, property lists, or syntax definitions.

Example:

```
<reference id="cli-options">
  <title>CLI Options Reference</title>
  <shortdesc>Command-line options and flags.</shortdesc>
  <refbody>
    <simpletable>
      <sthead>
        <stentry>Option</stentry>
        <stentry>Description</stentry>
      </sthead>
      <strow>
        <stentry><code>--version</code></stentry>
        <stentry>Display version information</stentry>
      </strow>
    </simpletable>
  </refbody>
</reference>
```

When to use it: API references, parameter tables, error code lists, glossaries, any "look this up" content.

Quick decision guide

When creating a new topic, ask yourself:

- Am I explaining **what** something is or **why** it matters? # **Concept**
- Am I teaching someone **how** to do something? # **Task**
- Am I providing **facts** for someone to look up? # **Reference**

If a single source file contains all three, split it into separate DITA topics — one for each purpose. This is how DITA achieves modularity.

Beyond authoring: How AI uses information typing

Information typing is valuable for writing and reviewing even without AI. But when your content feeds a chatbot or search system, the topic type becomes a machine-readable signal that lets the AI format answers appropriately — showing numbered steps for tasks, prose for concepts, and tables for references. This dimension is explored in depth in [Information Typing for AI](#).

Information Typing for AI: Intent Routing and Answer Formatting

How DITA's concept, task, and reference types give AI systems the semantic signal to route user intent and format answers appropriately.

Tip

This topic explores how AI systems use DITA's information typing for intent routing and answer formatting. If you are new to information typing, start with [Information Typing: Matching Content to Purpose](#), which covers the authoring fundamentals.

Why does information typing matter for AI?

When a user asks a chatbot a question, they have an **intent** — and different intents require different content types:

User question	User intent	DITA type	How AI formats the answer
"What is conref?"	Understand	Concept	Prose explanation with definition
"How do I install DITA-OT?"	Do something	Task	Numbered steps with prerequisites
"What parameters does the HTML5 transform accept?"	Look up	Reference	Parameter table with values
"What does this error mean?"	Troubleshoot	Reference	Problem/cause/solution format

Without information typing, the AI cannot distinguish these content types — it treats everything as undifferentiated text and formats all answers the same way.

Concept topics: Explaining ideas

A concept topic answers "what is..." and "why does..." questions. It uses `<conbody>` for narrative prose, organized into titled `<section>` elements.

Before (unstructured XML) — no type signal:

```
<article>
  <h1>Content Reuse in DITA</h1>
  <p>Content reuse is one of DITA's most powerful features.
  It allows you to write content once and reference it
  from multiple locations...</p>
</article>
```

After (DITA concept) — explicitly typed:

```
<concept id="dita-reuse-variants">
```

```

<title>Reuse, Keys, and Variant Management</title>
<shortdesc>How to write content once and assemble it into
multiple deliverables using conref, keyref, and DITAVAL
filtering.</shortdesc>
<conbody>
  <section id="why-reuse-matters">
    <title>Why does content reuse matter?</title>
    <p>Content reuse is one of DITA's most powerful
      features...</p>
  </section>
</conbody>
</concept>

```

AI benefit: The chatbot knows this is an explanation. It retrieves this topic for "what is..." questions and formats the answer as explanatory prose, not as steps.

Task topics: Teaching procedures

A task topic answers "how do I..." questions. DITA tasks enforce a specific structure: `<prereq> # <context> # <steps> # <result> # <postreq>`.

Before (unstructured XML) — steps look like any list:

```

<article>
  <h1>Installing DITA-OT</h1>

  <h2>Prerequisites</h2>
  <ul><li>Java 17 or later</li></ul>

  <h2>Steps</h2>
  <ol>
    <li>Download from dita-ot.org</li>
    <li>Unzip to your preferred location</li>
    <li>Add bin/ to your PATH</li>
    <li>Verify with <code>dita --version</code></li>
  </ol>
</article>

```

After (DITA task) — steps are semantically structured:

```

<task id="installing-dita-ot">
  <title>Installing DITA-OT</title>
  <shortdesc>Download, install, and verify DITA Open Toolkit
  4.x on your machine.</shortdesc>
  <taskbody>
    <prereq>
      <ul>
        <li><b>Java 17 or later</b> – check with
          <code>java -version</code></li>
      </ul>
    </prereq>
    <steps>
      <step>
        <cmd>Download the latest release from
          dita-ot.org</cmd>
        <info>Choose the .zip distribution</info>
      </step>
      <step>
        <cmd>Unzip to your preferred location</cmd>
        <stepresult>You should see a dita-ot-4.x/
          directory</stepresult>
      </step>
    </steps>
  </taskbody>
</task>

```



```

    <step>
      <cmd>Add the bin/ directory to your
        PATH</cmd>
    </step>
  </steps>
  <result>
    <p>Verify the installation:
      <codeph>dita --version</codeph> should
        print the version number.</p>
    </result>
  </taskbody>
</task>

```

AI benefit: The chatbot extracts structured steps. It can present prerequisites separately. It knows the expected result. It formats the answer as a numbered procedure with clear pre/post conditions.

Reference topics: Lookup information

A reference topic answers "what are the options for..." questions. It uses `<refbody>` with tables, property lists, and syntax references.

Before (unstructured XML) — a table that looks like any table:

```

<article>
  <h1>DITA-OT Parameters</h1>
  <table>
    <tr><th>Parameter</th><th>Description</th><th>Default</th></tr>
    <tr><td>args.input</td><td>Input file</td><td>--</td></tr>
    <tr><td>output.dir</td><td>Output directory</td><td>out</td></tr>
    <tr><td>transtype</td><td>Transform type</td><td>html5</td></tr>
  </table>
</article>

```

After (DITA reference) — semantically typed lookup:

```

<reference id="dita-ot-parameters">
  <title>DITA-OT Build Parameters Reference</title>
  <shortdesc>Complete reference of DITA-OT build parameters
    for the HTML5 and PDF transforms.</shortdesc>
  <refbody>
    <table>
      <tgroup cols="3">
        <thead>
          <row>
            <entry>Parameter</entry>
            <entry>Description</entry>
            <entry>Default</entry>
          </row>
        </thead>
        <tbody>
          <row>
            <entry><codeph>args.input</codeph></entry>
            <entry>Path to the input DITA map</entry>
            <entry>Required</entry>
          </row>
          ...
        </tbody>
      </tgroup>
    </table>
  </refbody>
</reference>

```

AI benefit: The chatbot knows this is lookup data. It formats the answer as a table or parameterized list. It can extract specific parameter values when users ask about individual parameters.

How information typing enables intent routing

With typed topics and the `content-intent` metadata value, the AI can build an intent-routing pipeline:

User query pattern	Detected intent	AI retrieval filter
"What is...", "Explain...", "Describe..."	explain	Filter for <code>content-intent="explain"</code>
"How do I...", "Steps to...", "Guide me..."	how-to	Filter for <code>content-intent="how-to"</code>
"List all...", "What are the options..."	reference	Filter for <code>content-intent="reference"</code>
"Error...", "Not working...", "Fix..."	troubleshoot	Filter for <code>content-intent="troubleshoot"</code>
"Best practice...", "Should I..."	best-practice	Filter for <code>content-intent="best-practice"</code>

This is impossible with unstructured XML — the AI must examine every document equally, regardless of whether it is a concept, task, or reference.

Reuse, Keys, and Variant Management

For many organizations, multi-format output is helpful—but reuse is what changes the economics of documentation. Reuse reduces:

Why reuse is the “real reason” many teams adopt DITA

For many organizations, multi-format output is helpful—but **reuse** is what changes the economics of documentation. Reuse reduces:

- duplicated editing effort,
- inconsistent content,
- translation cost,
- review cycles,
- release risk.

But reuse only works when it's designed, governed, and validated. This topic explains the core reuse mechanisms and how to use them without creating maintenance pain.

Reuse types: topic reuse vs content reuse vs reference reuse

Topic reuse

You reuse a whole topic by including it in multiple maps. This is the cleanest form:

- least fragile,
- easiest to understand,
- best for long-term maintenance.

Example use cases

- a “Safety overview” concept used in multiple manuals,
- a “Reset password” task reused across product editions,
- a “CLI options” reference topic used in both admin and developer docs.

Content reuse (partial reuse)

You reuse parts of topics:

- warnings,
- short paragraphs,
- shared steps,
- reusable tables.

This is powerful but can create complexity if overused.

Reference reuse (keys)

You reuse *references* (links, product names, terms) using keys so the same topic can adapt to different contexts without editing.

Conref: direct content reuse

What conref does

`conref` allows an element to pull content from another location. Think of it as “include this snippet here.”

Example

```
<p conref="reuse/common-notes.dita#common-notes/install-note"/>
```

This pulls the element with `id="install-note"` from the topic with `id="common-notes"` in `common-notes.dita`.

When conref is a great choice

- stable legal text,
- standardized safety notes,
- common prerequisites,
- consistent step fragments.

When conref causes problems

- when the reused content depends on local context,
- when authors “fix” shared snippets without coordinating,

- when the referenced snippet changes structure often,
- when the conref chain becomes too deep to understand.

Conref governance patterns (what professional teams do)

Pattern A: Shared library ownership

- Create a [reuse/](#) folder (or CCMS equivalent).
- Assign ownership to a small group.
- Use review requirements for changes to shared snippets.

Pattern B: “Reusable API” approach Treat each shared snippet like an API:

- stable ID,
- stable purpose,
- change logs,
- backwards compatibility mindset.

Pattern C: Shallow reuse Avoid deep conref chains:

- don't conref a snippet that contains conrefs that contain conrefs...
- shallow reuse is easier to debug and maintain.

Keys and keyref: the late-binding superpower

What keys are

Keys are map-defined labels that resolve to:

- links (hrefs),
- text,
- images,
- or other resources (depending on usage).

A topic uses [keyref](#) to point to a key, and the actual target is resolved based on the map at build time.

Why this matters in real life

Keys enable:

- brand-specific product naming,
- different support URLs per region,
- partner editions where links change,
- reuse of topics across multiple deliverables without editing.

Example: product name and support URL

In the map

```
<keydef keys="product-name">
  <topicmeta>
    <keywords><keyword>Acme Writer Pro</keyword></keywords>
  </topicmeta>
</keydef>

<keydef keys="support-url" href="https://example.com/support"/>
```

In the topic

```
<p>For help with <ph keyref="product-name"/>, see <xref keyref="support-url"/>.</p>
```

Change the key definition in the map, and the same topic publishes differently—without editing the topic.

Key design principles**1. Use keys for anything that varies by deliverable**

- product name, - URLs, - partner branding, - company names, - legal disclaimers by region (sometimes).

1. Keep key names stable

- Treat key names like public identifiers.

1. Group keys logically

- `product-*` keys
- `link-*` keys
- `img-*` keys

1. Document key ownership

- who can change keys, - how changes are reviewed.

Conkeyref: content reuse through keys**What conkeyref is**

`conkeyref` lets you pull content via a key instead of a direct file#id reference. This is powerful when:

- the *source* of reusable content varies by deliverable,
- the same topic must pull different snippets depending on map context.

Example scenario

You publish the same “Install” topic for:

- Product A,
- Product B,
- Partner Edition.

Each needs a slightly different legal notice. Instead of editing the topic or maintaining three variants, you define:

- `legal-install-note` key in each map,
- the topic uses `conkeyref="legal-install-note"`.

This is advanced, but it scales well when managed correctly.

Conkeyref example (complete)**Step 1 — Define the key in each map to point to a different snippet source:**

```
<!-- product-a.ditamap -->
<keydef keys="legal-install-note" href="reuse/product-a-legal.dita"/>

<!-- product-b.ditamap -->
```

```
<keydef keys="legal-install-note" href="reuse/product-b-legal.dita"/>
```

Step 2 — Create the snippet sources with matching IDs:

```
<!-- reuse/product-a-legal.dita -->
<topic id="legal-notices">
  <title>Legal</title>
  <body>
    <note id="install-legal">
      Product A is licensed under the Acme Standard License.
      See https://acme.example.com/license-a for details.
    </note>
  </body>
</topic>
```

```
<!-- reuse/product-b-legal.dita -->
<topic id="legal-notices">
  <title>Legal</title>
  <body>
    <note id="install-legal">
      Product B is licensed under the Acme Enterprise License.
      Contact your account manager for terms.
    </note>
  </body>
</topic>
```

Step 3 — Use conkeyref in the shared topic:

```
<!-- topics/install.dita (shared across both maps) -->
<task id="install-task">
  <title>Installation</title>
  <taskbody>
    <prereq>
      <note conkeyref="legal-install-note/install-legal"/>
    </prereq>
    <!-- ... steps ... -->
  </taskbody>
</task>
```

When you build with `product-a.ditamap`, the `conkeyref` resolves to the Product A legal note. Build with `product-b.ditamap`, and the same topic automatically shows the Product B text — no editing required.

When conkeyref is worth the effort

- many branded deliverables,
- partner ecosystems,
- product families with shared content but different key spaces.

Filtering and variant management: one source, many outputs

DITA filtering is often called **profiling** or **conditional processing**.

How filtering works

1. You tag content with attributes:

```
- platform="windows" - audience="admin" - product="pro"
```

1. Your build rules include/exclude based on those attributes.

Choosing attribute strategy (a big decision)

Common attribute dimensions

- platform: windows/linux/mac
- audience: user/admin/developer
- product/edition: standard/pro/enterprise
- region: us/eu/apac
- delivery: internal/external

Don't use everything Pick only the dimensions you truly need. Each added dimension multiplies complexity.

Controlled vocabulary (non-negotiable at scale)

If authors invent values freely, filtering becomes unreliable:

- `windows`, `Windows`, `win`, `win32` (all different!)

Professional programs define allowed values and enforce them.

Subject scheme maps (governance tool)

Many DITA ecosystems use subject scheme maps to:

- define allowed attribute values,
- provide taxonomy and classification,
- support consistent filtering.

Even if you don't implement subject scheme immediately, write down your vocabulary and enforce it in reviews.

Designing reusable and filterable content without chaos

Here are strategies that prevent "variant spaghetti."

Strategy A: Separate by topic when variants are large

If Windows and Linux procedures differ significantly:

- create separate task topics,
- assemble them differently per deliverable.

Use filtering only for minor differences, not completely different workflows.

Strategy B: Use filtering for small differences only

Filtering works well for:

- a few steps that differ,
- a paragraph that differs,
- a note that applies only to one platform.

If half the topic is conditional, split the topic.

Strategy C: Keep conditional blocks coarse

Instead of tagging every sentence:

- tag whole paragraphs,
- tag whole steps,
- tag whole sections.

This reduces maintenance.

Strategy D: Combine keys + filtering intelligently

Keys handle “context changes” (names/links). Filtering handles “content inclusion changes” (platform-specific steps). Use both; don’t force one mechanism to do everything.

Reuse + translation: how to save money and time

Reusable content reduces translation effort only if:

- you actually reuse (conref/topic reuse),
- content doesn’t change frequently,
- IDs remain stable,
- you avoid minor rewrites in shared snippets.

Translation best practices for reuse

- Keep shared snippets stable.
- Avoid embedding variable product names in shared snippets; use keys.
- Don’t conditionalize tiny phrases across many languages—use topic-level variants when needed.

Validation: how you keep reuse and variants reliable

A DITA program needs automated checks:

- broken conrefs,
- missing targets,
- unresolved keys,
- invalid attribute values (where possible),
- build warnings treated as failures in CI.

Practical quality gates

- Build must complete with zero unresolved references.
- Key space must resolve without ambiguity.
- Filtering must produce non-empty outputs for required deliverables.
- Link checking must pass for internal references (external link checking is optional, but useful).

A realistic “reuse plan” for a growing doc set

Start simple:

1. Topic reuse first

- Build a library of reusable concepts and tasks.

1. Conref for stable snippets

- Add a `reuse/common-notes.dita` library.

1. Keys for product names + URLs

- Keep topics portable across maps.

1. Filtering for platform/audience

- Choose 1–2 dimensions; don't overcomplicate.

Then scale:

- introduce key scopes if needed,
- introduce conkeyref for partner ecosystems,
- formalize subject schemes and vocabulary enforcement.

Common reuse and variant mistakes

Mistake: Conref everything

Fix: prefer topic reuse; conref only stable, context-independent snippets.

Mistake: Filtering every sentence

Fix: use topic splitting; conditionalize only where differences are small.

Mistake: No controlled vocabulary

Fix: define allowed values and enforce them.

Mistake: Hard-coded product names and URLs

Fix: use keys and `keyref`.

Mistake: Shared snippets without ownership

Fix: assign owners and add review rules.

Analytics and Instrumentation

To measure the impact of reuse, instrument your output with analytics. Track conref and keyref usage, and monitor which topics are reused most. Use analytics dashboards to identify content that could benefit from further reuse or consolidation.

Personalization and Conditional Processing

Use DITA conditional attributes (such as audience, product, platform) and DITAVAL files to personalize content for different users or outputs. Example:

```
<section audience="developer">Developer-specific content here.</section>
```

Integrating with Conversational AI and NLP

Expose this topic's content to chatbots or virtual assistants by making it available via API or structured export. Use semantic markup and metadata to help NLP systems extract relevant answers for user queries.

Continuous Improvement and Feedback Loops

Add feedback widgets to your published documentation and use analytics to monitor user suggestions. Regularly review feedback to refine reuse strategies and update content for clarity and effectiveness.

What's next?

You can now write typed topics, build projects, and apply reuse patterns. In **Part 4: Information Architecture**, you will learn how to organize topics at scale — designing map hierarchies, defining taxonomy and controlled vocabulary, and structuring content so it can grow from 10 topics to 1,000 without losing consistency.

Related concepts

[Authoring DITA Topics](#) on page 48

Writing DITA is not just writing in XML. It's writing with:

Related reference

[Glossary](#) on page 190

Comprehensive DITA and DITA-OT terminology used throughout the Extense training documentation. Each term includes definitions, abbreviations, usage guidance, and scope notes for chatbot knowledge retrieval.

Part

IV

Information Architecture

Topics:

- [Topic-Based Information Architecture for Training](#)
- [Taxonomy and Controlled Vocabulary for AI Classification](#)

Topic-Based Information Architecture for Training

How to design a modular, map-driven information architecture that supports progressive learning paths, role-based content delivery, and AI-powered navigation.

What are the architecture principles for AI-ready training content?

- 1. Topics are self-contained:** Every topic should be understandable without reading the surrounding content. The chatbot may retrieve a single section from a single topic — that section must make sense on its own.
- 2. Maps define context, not topics:** The sequence, hierarchy, and relationships between topics are defined in the map, not hardcoded into the topics. This allows the same topic to appear in multiple learning paths.
- 3. Progressive depth:** Organize content from foundational to advanced, with explicit prerequisite chains so the chatbot can guide users through an appropriate learning path.
- 4. Role-based navigation:** Use audience metadata and conditional processing to enable filtered views for different roles (technical writer, developer, manager).
- 5. Granularity for retrieval:** Each topic should cover one focused subject. Each section within a topic should address one specific question. Overly broad topics produce low-quality chunks.

How should topics be organized in the file system?

Organize topics by content domain, mapping to the taxonomy categories:

```
dita-topics/
### getting-started/           # Foundational, beginner content
#   ### dita-intro-guide.dita
#   ### installing-dita-ot.dita
#   ### setting-up-authoring-tools.dita
### core-topics/              # Core subject area content
#   ### dita-authoring.dita
#   ### dita-publishing.dita
#   ### dita-reuse-variants.dita
#   ### writing-first-dita-project.dita
### developer-guides/         # Advanced developer content
#   ### dita-ot-developer-handbook.dita (parent)
#   ### devhandbook-topics/      (sub-topics)
#       ### docker-basics.dita
#       ### github-actions-cicd.dita
#       ### ...16 files
### plugins-and-customization/ # Plugin development content
#   ### xslt-customization-tutorial.dita (parent)
#   ### xslt-topics/             (sub-topics)
#       ### create-html5-plugin.dita
#       ### debugging-workflow.dita
#       ### ...18 files
### intelligent-content/      # Meta: AI authoring guidelines
#   ### ai-ready-authoring-guide.dita
#   ### schema-org-mapping.dita
#   ### extense-taxonomy.subjectScheme
#   ### chatbot-config.json
#   ### chatbot-data/
### reference/                 # Lookup content
#   ### glossary.dita
#   ### architecture-diagram.dita
```

```
### plugin-reference.dita
```

Why this structure matters for AI: The directory organization mirrors the content domain taxonomy, making it easy to maintain classification consistency. The [chatbot-data/](#) directory keeps generated artifacts separate from source content.

How should the bookmap define learning paths?

The bookmap creates a progressive learning sequence where each chapter builds on the previous:

```
<bookmap id="extense-training-guide">
  <!-- Part 1: Foundation -->
  <part>
    <topicmeta><navtitle>Getting Started</navtitle></topicmeta>
    <chapter href="getting-started/dita-intro-guide.dita">
      <topicref href="getting-started/installing-dita-ot.dita"/>
      <topicref href="getting-started/setting-up-tools.dita"/>
    </chapter>
  </part>

  <!-- Part 2: Core skills -->
  <part>
    <topicmeta><navtitle>Core DITA Skills</navtitle></topicmeta>
    <chapter href="core-topics/dita-authoring.dita"/>
    <chapter href="core-topics/dita-reuse-variants.dita"/>
    <chapter href="core-topics/dita-publishing.dita"/>
  </part>

  <!-- Part 3: Advanced (developer audience) -->
  <part>
    <topicmeta><navtitle>Advanced Development</navtitle></topicmeta>
    <chapter href="developer-guides/dita-ot-developer-handbook.dita">
      <topicref href="developer-guides/devhandbook-topics/docker-basics.dita"/>
      <topicref href="developer-guides/devhandbook-topics/github-actions-cicd.dita"/>
      <!-- ...more sub-topics -->
    </chapter>
  </part>
</bookmap>
```

The map hierarchy gives the chatbot three critical signals:

- **Sequence:** Topics should be read in this order for maximum comprehension
- **Nesting:** Sub-topics are detailed elaborations of their parent topic
- **Grouping:** Parts group topically related content for navigation

How granular should topics and sections be?

The right granularity depends on the content purpose:

Level	Guideline	AI rationale
Topic	One focused subject per topic. 500–3000 words. 3–8 sections.	Topics that are too long produce too many chunks; too short topics waste metadata overhead.

Level	Guideline	AI rationale
Section	One question or concept per section. 100–1200 tokens (1–6 paragraphs).	Each section becomes a separately retrievable chunk. It must make sense on its own with the inherited topic metadata.
Shortdesc	One sentence, 25–35 words. Describe purpose, not content.	Prepended to every chunk as summary. Vague shortdescs degrade every chunk from that topic.
Section title	Phrase as a question when possible.	"How does conref work?" matches user queries better than "Conref Overview".

Related tasks

[Metadata Enrichment: Adding the AI Layer to Every Topic](#) on page 94

Step-by-step process for adding the complete AI-ready prolog metadata to DITA topics — the single highest-impact action for improving chatbot retrieval quality.

[Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step](#) on page 84

A complete, repeatable process for converting unstructured XML files into semantically rich, AI-ready DITA topics with full prolog metadata.

Related reference

[Appendix: Complete AI-Ready Metadata Field Reference](#) on page 204

Comprehensive reference for every metadata field used in the AI-ready DITA prolog — allowed values, purpose, AI pipeline usage, and examples.

[AI-Ready Content Authoring Guide](#) on page 97

Guidelines for writing DITA content that is optimized for the Extense chatbot, RAG retrieval, and AI-driven content delivery.

Taxonomy and Controlled Vocabulary for AI Classification

How a formal DITA subject scheme taxonomy provides the controlled vocabulary that AI systems need to classify, filter, and route content accurately.

Why does AI need a controlled vocabulary?

Without a taxonomy, authors use inconsistent terms for the same concepts. One author writes `audience="developers"`, another writes `audience="dev"`, a third writes `audience="engineers"`. The AI sees these as three different audiences and cannot match them in retrieval.

A **controlled vocabulary** defines the exact allowed values for each classification facet. A DITA **subject scheme** map is the standard mechanism for defining and enforcing this vocabulary.

How to design a content taxonomy for AI

A production taxonomy needs five classification facets at minimum:

Facet	Purpose	Example values	AI use
Audience	Who the content is written for	technical-writer, developer, build-engineer, manager, all	Filter results by user role; personalize recommendations
Difficulty	Knowledge level required	beginner, intermediate, advanced, expert	Match content level to user expertise; suggest simpler alternatives
Content domain	Subject area classification	dita-fundamentals, authoring, publishing, customization, xslt, devops	Domain-scoped retrieval; only search relevant subject areas
Content intent	What the content does for the reader	explain, how-to, reference, troubleshoot, best-practice, tutorial	Intent routing; match user question type to content type
Service line	Business service connection	dita-engineering, publishing-engineering, xml-engineering, training	Link content to service offerings; enable lead generation

How to implement a subject scheme in DITA

The subject scheme is a specialized DITA map that defines the taxonomy:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE subjectScheme PUBLIC "-//OASIS//DTD DITA Subject Scheme Map//EN"
  "subjectScheme.dtd">
<subjectScheme id="extense-taxonomy">
  <title>Extense Content Taxonomy</title>

  <!-- AUDIENCE TYPES -->
  <subjectdef keys="audience-type">
    <subjectdef keys="aud-technical-writer">
      <topicmeta><navtitle>Technical Writer</navtitle></topicmeta>
    </subjectdef>
    <subjectdef keys="aud-developer">
      <topicmeta><navtitle>Developer / Engineer</navtitle></topicmeta>
    </subjectdef>
    <subjectdef keys="aud-build-engineer">
      <topicmeta><navtitle>Build / DevOps Engineer</navtitle></topicmeta>
    </subjectdef>
    <subjectdef keys="aud-manager">
      <topicmeta><navtitle>Documentation Manager</navtitle></topicmeta>
    </subjectdef>
    <subjectdef keys="aud-all">
      <topicmeta><navtitle>All Audiences</navtitle></topicmeta>
    </subjectdef>
  </subjectdef>
```

```

<!-- DIFFICULTY LEVELS -->
<subjectdef keys="difficulty-level">
  <subjectdef keys="beginner"/>
  <subjectdef keys="intermediate"/>
  <subjectdef keys="advanced"/>
  <subjectdef keys="expert"/>
</subjectdef>

<!-- CONTENT DOMAINS -->
<subjectdef keys="content-domain">
  <subjectdef keys="dom-dita-fundamentals"/>
  <subjectdef keys="dom-authoring"/>
  <subjectdef keys="dom-content-reuse"/>
  <subjectdef keys="dom-publishing"/>
  <subjectdef keys="dom-customization"/>
  <subjectdef keys="dom-xslt"/>
  <subjectdef keys="dom-devops"/>
  <subjectdef keys="dom-architecture"/>
</subjectdef>

<!-- CONTENT INTENT -->
<subjectdef keys="content-intent">
  <subjectdef keys="explain"/>
  <subjectdef keys="how-to"/>
  <subjectdef keys="reference"/>
  <subjectdef keys="troubleshoot"/>
  <subjectdef keys="best-practice"/>
  <subjectdef keys="tutorial"/>
  <subjectdef keys="overview"/>
</subjectdef>
</subjectScheme>

```

This taxonomy is machine-readable, version-controlled, and can be shared across projects. The chatbot configuration references it for metadata validation.

How does the glossary support AI term resolution?

A DITA glossary serves as the chatbot's **term resolution engine**. Rich glossary entries enable the AI to:

- **Resolve synonyms:** User asks about "conditional processing" — the glossary maps this to "DITAVAL filtering"
- **Expand abbreviations:** User references "OT" — the glossary knows this means "DITA Open Toolkit"
- **Provide definitions:** User asks "What does conref mean?" — the glossary provides the canonical definition
- **Scope applicability:** The glossary notes that a feature applies to "DITA-OT 4.x only"

Example of an AI-optimized glossary entry:

```

<glossentry id="gl-conref">
  <glossterm>conref</glossterm>
  <glossdef>A DITA content reference mechanism that pulls
    content from one topic into another at build time,
    enabling single-source reuse.</glossdef>
  <glossBody>
    <glossSurfaceForm>content reference (conref)</glossSurfaceForm>
    <glossAlt>
      <glossAbbreviation>conref</glossAbbreviation>
    </glossAlt>
    <glossUsage>Use "conref" when referring to the pull
      mechanism. Use "content reference" in formal
      documentation.</glossUsage>
    <glossScopeNote>Available in all DITA versions. In

```



```
DITA 1.3+, conkeyref provides indirect conref via
keys.</glossScopeNote>
</glossBody>
</glossentry>
```

What's next?

You now have an information architecture and taxonomy that can organize content at scale. In **Part 5: Making Content Intelligent**, you will put this architecture to work — enriching topics with AI-ready metadata, migrating content from unstructured XML, and preparing your structured content for chatbot ingestion and intelligent delivery.

Related tasks

[Metadata Enrichment: Adding the AI Layer to Every Topic](#) on page 94

Step-by-step process for adding the complete AI-ready prolog metadata to DITA topics — the single highest-impact action for improving chatbot retrieval quality.

[Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step](#) on page 84

A complete, repeatable process for converting unstructured XML files into semantically rich, AI-ready DITA topics with full prolog metadata.

Related reference

[Appendix: Complete AI-Ready Metadata Field Reference](#) on page 204

Comprehensive reference for every metadata field used in the AI-ready DITA prolog — allowed values, purpose, AI pipeline usage, and examples.

[AI-Ready Content Authoring Guide](#) on page 97

Guidelines for writing DITA content that is optimized for the Extense chatbot, RAG retrieval, and AI-driven content delivery.

Part

V

Making Content Intelligent

Topics:

- [Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step](#)
- [Before and After: Complete Transformation Examples](#)
- [Metadata Enrichment: Adding the AI Layer to Every Topic](#)
- [AI-Ready Content Authoring Guide](#)
- [DITA to Schema.org Type Mapping](#)

Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step

A complete, repeatable process for converting unstructured XML files into semantically rich, AI-ready DITA topics with full prolog metadata.

Before starting migration, ensure you have:

- DITA-OT 4.x installed and working (`dita --version`)
- A taxonomy/subject scheme defining your controlled vocabulary (optional — if you do not have one yet, use the starter taxonomy below)
- Your complete set of unstructured XML or HTML source files
- A clear understanding of your target audience types

Tip

No taxonomy yet? You can start with this minimal subject scheme and refine it as you migrate more content. Save this as `subjectScheme.ditamap` in your project:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE subjectScheme PUBLIC "-//OASIS//DTD DITA Subject Scheme Map//EN"
"subjectScheme.dtd">
<subjectScheme>
  <subjectdef keys="difficulty">
    <subjectdef keys="beginner"/>
    <subjectdef keys="intermediate"/>
    <subjectdef keys="advanced"/>
  </subjectdef>
  <subjectdef keys="content-intent">
    <subjectdef keys="explain"/>
    <subjectdef keys="how-to"/>
    <subjectdef keys="reference"/>
    <subjectdef keys="troubleshoot"/>
  </subjectdef>
  <subjectdef keys="audience-type">
    <subjectdef keys="technical-writer"/>
    <subjectdef keys="developer"/>
    <subjectdef keys="manager"/>
  </subjectdef>
</subjectScheme>
```

A subject scheme is simply a DITA map that defines **allowed values** — like a picklist. It ensures every author uses the same vocabulary for difficulty, intent, and audience metadata. You will learn more about subject schemes in [Taxonomy and Controlled Vocabulary](#).

This migration process was developed and refined while converting the Extense training content from 20+ unstructured XML files to 76 AI-ready DITA topics. Each step produces a measurable improvement in AI readiness. The steps are ordered from foundational (structure) to advanced (AI optimization) so that each step builds on the previous one.

1. Audit your source content and classify each file by information type.

Read every source XML file and determine whether its primary purpose is to:

- **Explain** a concept # will become a `<concept>`
- **Teach** a procedure # will become a `<task>`
- **Provide** lookup/reference data # will become a `<reference>`

Create a spreadsheet mapping each source file to its DITA type:

```
dita_intro_guide.xml      # concept  (dita-intro-guide.dita)
installing_dita_ot.xml    # task      (installing-dita-ot.dita)
dita-ot-parameters.xml    # reference (dita-ot-parameters.dita)
dita_reuse_variants.xml    # concept  (dita-reuse-variants.dita)
```

You have a complete mapping of source files to target DITA types.

2. Convert the structural markup from source XML to DITA XML.

For each file, perform these structural conversions:

Source XML / HTML	DITA
<code><h1>Title</h1></code>	<code><title>Title</title></code>
<code><h2>Section Heading</h2></code>	<code><section id="..."><title>...</title></code>
<code><p>paragraph text</p></code>	<code><p>paragraph text</p></code>
<code>bullet item</code>	<code>...</code>
<code>numbered item</code>	<code>...</code> or <code><steps><step>...</step></steps></code>
<code><pre>code block</pre></code>	<code><codeblock outputclass="language-...">...</codeblock></code>
<code><code>inline code</code></code>	<code><codeph>...</codeph></code>
<code>bold text</code>	<code>...</code>
<code>text</code>	<code><xref href="...">text</xref></code>

Valid DITA XML files that pass structural validation (`dita --input file.dita -f html5`).

3. Write a `<shortdesc>` for every topic.

The shortdesc is **critical for AI** — it is prepended to every chunk from that topic as a summary. A good shortdesc:

- Is one clear sentence (25-35 words)
- Describes the topic's purpose, not just its title
- Contains key search terms
- Works as a standalone summary when shown in search results

Before (no shortdesc): Chunks from this topic have no summary context.

After (with shortdesc):

```
<shortdesc>How to write content once and assemble it into
multiple deliverables using conref, keyref, and DITAVAL
filtering.</shortdesc>
```

Every topic has a shortdesc that functions as a chunk summary.

4. Add `xml:lang="en-US"` to every root element.

This attribute enables NLP language detection in the RAG pipeline. Without it, the embedding model cannot confirm the content language.

```
<concept id="dita-intro-guide" xml:lang="en-US">
```

5. Add the full AI-ready prolog metadata to every topic.

This is the most impactful step. Each topic needs this complete `<prolog>` block:

```
<prolog>
  <author>Extense LLC</author>
  <critdates>
    <created date="2026-02-10"/>
  </critdates>
  <metadata>
    <keywords>
      <indexterm>Term 1</indexterm>
      <indexterm>Term 2</indexterm>
      <indexterm>Term 3</indexterm>
    </keywords>
    <category>taxonomy:content-domain</category>
    <audience>target-audience</audience>
  </metadata>
  <resourceid appid="extense-training"
    appname="chatbot" id="topic-id"/>
  <data name="difficulty" value="intermediate"/>
  <data name="duration" value="PT20M"/>
  <data name="content-domain" value="publishing"/>
  <data name="content-intent" value="explain"/>
  <data name="chatbot-priority" value="high"/>
  <data name="service-line" value="publishing-engineering"/>
  <data name="prerequisites" value="previous-topic-id"/>
</prolog>
```

Choose metadata values from the controlled vocabulary defined in the subject scheme taxonomy. Each field has a specific purpose for AI:

- **difficulty:** beginner | intermediate | advanced | expert — enables level-appropriate retrieval
- **duration:** ISO 8601 format (PT15M = 15 minutes) — helps users plan learning time
- **content-domain:** subject area for domain-filtered retrieval
- **content-intent:** explain | how-to | reference | troubleshoot | best-practice | tutorial — enables intent routing
- **chatbot-priority:** high | medium | low — weights retrieval ranking
- **service-line:** links content to business service offerings
- **prerequisites:** topic-id of prerequisite content — enables learning path sequencing

Every topic has complete, taxonomy-aligned metadata that the RAG pipeline can filter on.

6. Ensure every `<section>` has both an `@id` and a `<title>`.

The chunking pipeline splits content at section boundaries. Each section becomes a separately retrievable chunk. Sections without IDs cannot be deep-linked. Sections without titles produce chunks that lack context labels.

```
<!-- BAD: No id, no title -->
<section>
  <p>Conref allows you to...</p>
</section>

<!-- GOOD: id + title -->
<section id="conref-mechanism">
  <title>How does the conref mechanism work?</title>
  <p>Conref allows you to...</p>
</section>
```

Every section is addressable, titled, and produces a well-labeled chunk.

7. Add `outputclass="language- *"` to all codeblocks.

This enables syntax highlighting in HTML output and language detection in RAG chunks:

```
<codeblock outputclass="language-bash">dita --input map.ditamap -f html5</codeblock>
<codeblock outputclass="language-xml">&lt;concept id="..."&gt;</codeblock>
<codeblock outputclass="language-xslt">&lt;xsl:template match="..."&gt;</codeblock>
```

8. Build the DITA map that defines topic organization and learning paths.

Create a bookmark or map that organizes topics into a progressive learning sequence:

```
<bookmark id="training-guide">
  <chapter href="getting-started/dita-intro-guide.dita">
    <topicref href="getting-started/installing-dita-ot.dita"/>
    <topicref href="getting-started/setting-up-tools.dita"/>
  </chapter>
  <chapter href="core-topics/dita-authoring.dita">
    <topicref href="core-topics/dita-reuse-variants.dita"/>
    <topicref href="core-topics/dita-publishing.dita"/>
  </chapter>
  ...
</bookmark>
```

The map hierarchy tells the chatbot about content relationships — which topics belong together and in what order they should be learned.

A complete DITA map defining the information architecture and learning path.

9. Validate by building HTML5 output and testing the chatbot ingestion pipeline.

Run two validations:

- a. Build HTML5 to verify structural validity: `dita -i map.ditamap -f html5 -o output/`
- b. Run the chatbot ingestion script to verify metadata extraction and chunking: `python3 dita-to-chatbot-json.py --input dita-topics --output chatbot-data`

Inspect the `ingestion-report.json` for chunks missing metadata, short chunks, or other quality issues.

All topics build without errors and produce metadata-rich chunks in the chatbot pipeline.

After completing these steps, your content has been transformed from unstructured XML into AI-ready DITA topics. Every topic is:

- Semantically typed (concept, task, or reference)
- Metadata-enriched with audience, difficulty, domain, intent, and priority
- Section-structured for optimal RAG chunking
- Organized into maps for learning path sequencing
- Validated for both HTML output and chatbot ingestion

Once migration is complete, establish ongoing governance:

- Create a topic template with the full prolog metadata as a starting point for new topics
- Add metadata validation to your CI/CD pipeline to prevent metadata-lean topics from being published
- Schedule quarterly AI-readiness audits using the scoring framework from the maturity model

Related concepts

[Topic-Based Information Architecture for Training](#) on page 76

How to design a modular, map-driven information architecture that supports progressive learning paths, role-based content delivery, and AI-powered navigation.

[Taxonomy and Controlled Vocabulary for AI Classification](#) on page 78

How a formal DITA subject scheme taxonomy provides the controlled vocabulary that AI systems need to classify, filter, and route content accurately.

Related reference

[Appendix: Complete AI-Ready Metadata Field Reference](#) on page 204

Comprehensive reference for every metadata field used in the AI-ready DITA prolog — allowed values, purpose, AI pipeline usage, and examples.

[AI-Ready Content Authoring Guide](#) on page 97

Guidelines for writing DITA content that is optimized for the Extense chatbot, RAG retrieval, and AI-driven content delivery.

Before and After: Complete Transformation Examples

Side-by-side comparison of real training content showing the complete transformation from unstructured XML to AI-ready DITA, with annotations explaining why each change matters for chatbot quality.

Example 1: Concept topic — Explaining content reuse

BEFORE — Unstructured XML:

```
<article>
  <h1>DITA Reuse and Variants</h1>

  <section>
    <h2>Overview</h2>
    <p>Content reuse is a key benefit of DITA. You can reuse
```



```

    content at multiple levels using different mechanisms.</p>
</section>

<section>
  <h2>Conref</h2>
  <p>The conref mechanism lets you pull content from one
  topic into another. It works like a symlink for content.</p>
  <p>Example:</p>
  <pre>&lt;p conref="shared/warnings.dita#warnings/safety-warning"/&gt;</pre>
</section>

<section>
  <h2>Keyref</h2>
  <p>Keys provide indirect references. Define a key in
  your map, then reference it in topics.</p>
</section>

<section>
  <h2>DITAVAL Filtering</h2>
  <p>Use DITAVAL files to include or exclude content based
  on audience, platform, or product attributes.</p>
</section>

<section>
  <h2>Best Practices</h2>
  <ul>
    <li>Always use keys for product names</li>
    <li>Prefer keyref over conref when possible</li>
    <li>Test all variants before publishing</li>
  </ul>
</section>
</article>

```

AFTER — AI-Ready DITA:

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN"
  "concept.dtd">
<concept id="dita-reuse-variants" xml:lang="en-US">
  <title>Reuse, Keys, and Variant Management</title>
  <shortdesc>How to write content once and assemble it into
  multiple deliverables using conref, keyref, conkeyref,
  and DITAVAL filtering.</shortdesc>

  <prolog>
    <author>Extense LLC</author>
    <critdates>
      <created date="2026-02-10"/>
    </critdates>
    <metadata>
      <keywords>
        <indexterm>Content reuse</indexterm>
        <indexterm>Conref</indexterm>
        <indexterm>Keyref</indexterm>
        <indexterm>DITAVAL</indexterm>
        <indexterm>Conditional processing</indexterm>
      </keywords>
      <category>taxonomy:content-reuse</category>
      <audience>technical-writer</audience>
      <audience>developer</audience>
    </metadata>
    <resourceid appid="extense-training"
      appname="chatbot" id="dita-reuse-variants"/>
    <data name="difficulty" value="intermediate"/>
    <data name="duration" value="PT25M"/>
    <data name="content-domain" value="content-reuse"/>
  </prolog>
</concept>

```

```

<data name="content-intent" value="explain"/>
<data name="chatbot-priority" value="high"/>
<data name="service-line" value="dita-engineering"/>
<data name="prerequisites" value="dita-authoring"/>
</prolog>

<conbody>
  <section id="why-reuse-matters">
    <title>Why does content reuse matter?</title>
    <p>Reuse eliminates duplication...</p>
  </section>

  <section id="conref-mechanism">
    <title>How does conref work?</title>
    <p>The conref mechanism pulls content from one
      topic into another at publish time...</p>
    <codeblock outputclass="language-xml">
      <p conref="shared/warnings.dita#warnings/safety"/>
    </codeblock>
  </section>

  <section id="keyref-mechanism">
    <title>How do keys and keyref work?</title>
    <p>Keys provide indirect references...</p>
  </section>

  <section id="ditaval-filtering">
    <title>How does DITaval filtering work?</title>
    <p>DITaval files include or exclude content...</p>
  </section>

  <section id="reuse-best-practices">
    <title>What are the best practices for reuse?</title>
    <ul>
      <li>Always use keys for product names</li>
      <li>Prefer keyref over conref</li>
      <li>Test all variants before publishing</li>
    </ul>
  </section>
</conbody>
</concept>

```

What changed and why:

Change made	Why it was needed	AI/chatbot impact
File typed as <code><concept></code>	This content explains ideas, not procedures	AI formats answer as explanation, not steps
Added <code><shortdesc></code>	Serves as chunk summary in vector search	Every chunk inherits this summary for context
Added 5 <code><indexterm></code> s including "Conditional processing"	Users search by multiple terms for the same concept	Both "DITaval" and "conditional processing" queries find this topic
Set <code>content-domain="content-reuse"</code>	Classifies by subject area	Domain-filtered retrieval: only return reuse content when user asks about reuse

Change made	Why it was needed	AI/chatbot impact
Set <code>content-intent="explain"</code>	Declares this explains a concept, not instructs a task	"What is conref?" routes here; "How do I set up conref?" routes to a task topic
Section titles rewritten as questions	Matches how users ask the chatbot	"How does conref work?" query matches section title directly, boosting retrieval score
Each section has an <code>@id</code>	Enables deep-linking and section-level retrieval	Chatbot can link to <code>#conref-mechanism</code> specifically
Set <code>prerequisites="dita-authoring"</code>	Defines learning sequence	Chatbot can suggest "Read DITA Authoring first" to beginners

Example 2: Task topic — Installing DITA-OT

BEFORE — Unstructured XML:

```
<article>
  <h1>Installing DITA-OT</h1>

  <p>You need Java 17+ installed first.</p>

  <ol>
    <li>Go to https://www.dita-ot.org/download</li>
    <li>Download the latest .zip release</li>
    <li>Unzip to a directory (e.g., ~/dita-ot)</li>
    <li>Add ~/dita-ot/bin to your PATH</li>
    <li>Test: run <code>dita --version</code></li>
  </ol>

  <p>If you see the version number, you're all set!</p>
</article>
```

AFTER — AI-Ready DITA task:

```
<task id="installing-dita-ot" xml:lang="en-US">
  <title>Installing DITA-OT</title>
  <shortdesc>Download, install, and verify DITA Open Toolkit
  4.x on your machine.</shortdesc>
  <prolog>
    <!-- Full AI-ready metadata... -->
    <data name="difficulty" value="beginner"/>
    <data name="content-intent" value="how-to"/>
    <data name="chatbot-priority" value="high"/>
    <data name="prerequisites" value="none"/>
  </prolog>
  <taskbody>
    <prereq>
      <ul>
        <li><b>Java 17 or later</b> – Check with
          <code>java -version</code>.</li>
        <li><b>500 MB free disk space</b></li>
        <li><b>Network access</b> to download</li>
      </ul>
    </prereq>
  </taskbody>
</task>
```

```

    </ul>
  </prereq>
  <steps>
    <step>
      <cmd>Download the latest release from
        dita-ot.org/download</cmd>
      <info>Choose the .zip distribution for your
        platform.</info>
    </step>
    <step>
      <cmd>Unzip to your preferred directory.</cmd>
      <stepxmp>
        <codeblock outputclass="language-bash">
unzip dita-ot-4.2.zip -d ~/
        </codeblock>
      </stepxmp>
      <stepresult>A dita-ot-4.2/ directory is
        created.</stepresult>
    </step>
    <step>
      <cmd>Add the bin/ directory to your PATH.</cmd>
      <stepxmp>
        <codeblock outputclass="language-bash">
export PATH=~/.dita-ot-4.2/bin:$PATH
        </codeblock>
      </stepxmp>
    </step>
  </steps>
  <result>
    <p>Run <codeph>dita --version</codeph>. You should
      see: <codeph>DITA-OT version 4.2</codeph>.</p>
  </result>
</taskbody>
</task>

```

Key improvements:

- **Prerequisites are semantically separated** — AI can show them before the steps, and warn users if prerequisites are not met
- **Steps use <cmd>** — the imperative instruction is clearly separated from supporting info, examples, and results
- **<stepresult>** — AI can tell users what to expect after each step
- **<result>** — AI can describe the expected outcome to help users verify success
- **content-intent="how-to"** — "How do I install DITA-OT?" queries are routed directly here

Example 3: The metadata difference — same topic, two quality levels

This example shows the same DITA topic at two metadata quality levels, demonstrating the before/after of the metadata enrichment process:

BEFORE — Lean metadata (only author + indexterms):

```

<concept id="docker-basics">
  <title>Docker for DITA-OT</title>
  <shortdesc>Using the official DITA-OT Docker image
    for consistent builds.</shortdesc>
  <prolog>
    <author>Extense LLC</author>
    <metadata>
      <keywords>
        <indexterm>Docker</indexterm>

```

```

        <indexterm>Container</indexterm>
    </keywords>
</metadata>
</prolog>
...
</concept>

```

What the chatbot can do: Find this topic via keyword match ("Docker") or vector similarity. That's it. No filtering by audience, difficulty, domain, or intent.

AFTER — Full AI-ready metadata:

```

<concept id="docker-basics" xml:lang="en-US">
  <title>Docker for DITA-OT: Local Builds and Reproducible
    Toolchains</title>
  <shortdesc>Using the official DITA-OT Docker image for
    consistent, reproducible builds that eliminate environment
    differences across developer machines and CI.</shortdesc>
  <prolog>
    <author>Extense LLC</author>
    <critdates>
      <created date="2026-02-10"/>
    </critdates>
    <metadata>
      <keywords>
        <indexterm>Docker</indexterm>
        <indexterm>Container</indexterm>
        <indexterm>Reproducible builds</indexterm>
      </keywords>
      <category>taxonomy:devops</category>
      <audience>developer</audience>
      <audience>build-engineer</audience>
    </metadata>
    <resourceid appid="extense-training"
      appname="chatbot" id="docker-basics"/>
    <data name="difficulty" value="intermediate"/>
    <data name="duration" value="PT20M"/>
    <data name="content-domain" value="devops"/>
    <data name="content-intent" value="explain"/>
    <data name="chatbot-priority" value="high"/>
    <data name="service-line"
      value="publishing-engineering"/>
    <data name="prerequisites" value="dita-publishing"/>
  </prolog>
  ...
</concept>

```

What the chatbot can now do:

- Filter for `audience=developer` — show this only to developers, not content strategists
- Filter for `difficulty=intermediate` — don't show to absolute beginners
- Route `content-domain=devops` — only return when user asks about devops topics
- Match `content-intent=explain` — show for "what is..." questions, not "how to..." questions
- Apply `chatbot-priority=high` — rank above low-priority reference material
- Suggest `prerequisites=dita-publishing` — "You should read DITA Publishing first"
- Use `critdates` — rank fresher content higher

Summary: What changes at each transformation stage

Aspect	Unstructured XML (Level 1)	DITA with lean metadata (Level 2)	AI-ready DITA (Level 3+)
Content type signal	None	Root element type	Root element + content-intent metadata
Audience targeting	None	None	<code><audience></code> + taxonomy
Difficulty filtering	None	None	<code><data name="difficulty"></code>
Domain routing	Inferred from text	Inferred from text	Explicit <code>content-domain</code>
Chunk quality	Heading-split, no metadata	Section-split, basic metadata	Section-split with full metadata inheritance
Retrieval ranking	Vector similarity only	Vector + keyword	Vector + keyword + metadata boost + priority weighting
Learning sequences	None	Map ordering	Map ordering + prerequisite chains
Chatbot answer quality	Mediocre	Good structure, limited filtering	Precise, filtered, intent-matched, audience-appropriate

Metadata Enrichment: Adding the AI Layer to Every Topic

Step-by-step process for adding the complete AI-ready prolog metadata to DITA topics — the single highest-impact action for improving chatbot retrieval quality.

Metadata enrichment is the process of adding classification, audience, difficulty, intent, and priority metadata to every topic in your DITA corpus. This metadata is what transforms DITA from a structured document format into an **AI-addressable content system**.

In the Extense training guide, enriching 34 sub-topics with full metadata raised the AI-readiness score from 7.2/10 to 9.2/10 and increased metadata coverage from 24% to 100%.

1. Audit every topic and document the current metadata state.

Use `grep` or a script to check each file for the required metadata fields:

```
# Count files with full metadata
grep -rl 'chatbot-priority' dita-topics/ | wc -l

# Count files missing metadata
```

```
grep -rL 'chatbot-priority' dita-topics/ --include="*.dita" | wc -l
```

Create a spreadsheet documenting each file's current metadata state and the values that need to be assigned.

A complete inventory of which topics have metadata and which need enrichment.

2. Read each topic and determine contextually appropriate metadata values.

For each topic, decide:

- **Audience:** Who is this written for? Developer? Technical writer? All?
- **Difficulty:** What knowledge level is required? Beginner? Advanced?
- **Content domain:** What subject area? Publishing? Customization? DevOps?
- **Content intent:** What does this content do? Explain? Instruct? Reference?
- **Chatbot priority:** How likely are users to ask about this? High? Medium? Low?
- **Service line:** Which business service does this relate to?
- **Prerequisites:** What should the reader know before reading this?
- **Duration:** How long does it take to read/complete this content?

All values must come from the controlled vocabulary defined in the subject scheme.

3. Apply metadata to each topic, either manually or using an enrichment script.

Manual approach: Open each file and add the full prolog block. Best for small batches (under 10 topics).

Scripted approach: For larger batches, create a Python script that reads each file, determines the topic type, and replaces the lean prolog with the full AI-ready prolog. Example:

```
import re

def enrich_topic(filepath, metadata):
    with open(filepath, 'r') as f:
        content = f.read()

    # Extract topic ID from root element
    topic_id = re.search(
        r'id="([^\"]+)"', content
    ).group(1)

    # Add xml:lang if missing
    if 'xml:lang=' not in content:
        content = re.sub(
            r'(id="([^\"]+)"',
            r'\1 xml:lang="en-US"',
            content, count=1
        )

    # Build new prolog and replace existing
    new_prolog = build_prolog(topic_id, metadata)
    content = re.sub(
        r'<prolog>.*?</prolog>',
        new_prolog,
        content, flags=re.DOTALL
    )

    with open(filepath, 'w') as f:
        f.write(content)
```

This scripted approach was used to enrich all 34 sub-topics in the Extense training guide in a single batch operation.

Every topic has the complete AI-ready prolog metadata.

4. Verify the enrichment by checking that all required fields are present.

```
# Verify all 7 key fields across all topics
for field in xml:lang chatbot-priority resourceid \
  content-domain critdates category audience; do
  echo "=== $field ==="
  grep -rl "$field" dita-topics/ --include="*.dita" | wc -l
done
```

The count for each field should match the total number of topic files.

Confirmed 100% metadata coverage across the entire corpus.

After enrichment, every topic in the corpus produces **metadata-rich chunks** in the RAG pipeline. The chatbot can now:

- Filter by audience, difficulty, and domain in every query
- Route user intent to the correct content type
- Apply priority weighting to rank the most useful content first
- Suggest prerequisites and next steps in learning paths
- Use freshness dates to prefer newer content

Related concepts

[Topic-Based Information Architecture for Training](#) on page 76

How to design a modular, map-driven information architecture that supports progressive learning paths, role-based content delivery, and AI-powered navigation.

[Taxonomy and Controlled Vocabulary for AI Classification](#) on page 78

How a formal DITA subject scheme taxonomy provides the controlled vocabulary that AI systems need to classify, filter, and route content accurately.

Related reference

[Appendix: Complete AI-Ready Metadata Field Reference](#) on page 204

Comprehensive reference for every metadata field used in the AI-ready DITA prolog — allowed values, purpose, AI pipeline usage, and examples.

[AI-Ready Content Authoring Guide](#) on page 97

Guidelines for writing DITA content that is optimized for the Extense chatbot, RAG retrieval, and AI-driven content delivery.

AI-Ready Content Authoring Guide

Guidelines for writing DITA content that is optimized for the Extense chatbot, RAG retrieval, and AI-driven content delivery.

Why AI-ready content matters

The Extense chatbot uses **Retrieval-Augmented Generation (RAG)** to answer user questions. It works by:

1. **Chunking** your DITA content into sections
2. **Embedding** each chunk as a vector in a database
3. **Retrieving** the most relevant chunks for a user's question
4. **Generating** an answer grounded in those chunks

The **quality of the chatbot's answers depends directly on the quality of your content**. Well-structured, well-metadata'd DITA content produces better retrieval and better answers.

Required prolog metadata

Every topic **must** include these elements in the prolog for chatbot ingestion:

Element	Purpose for chatbot	Example
<code>xml:lang</code>	Language detection for NLP processing	<code>xml:lang="en-US"</code> on root element
<code><shortdesc></code>	Used as chunk summary in retrieval results	One clear sentence describing the topic's purpose
<code><critdates></code>	Content freshness signals for ranking	<code><created date="2026-01-15"/></code>
<code><indexterm></code>	Keyword matching in vector search	3-6 specific terms per topic
<code><data name="difficulty"></code>	Filters results by user level	beginner intermediate advanced expert
<code><data name="content-domain"></code>	Routes questions to the right content area	dita-fundamentals authoring publishing customization
<code><data name="content-intent"></code>	Matches user intent to content type	explain how-to reference troubleshoot tutorial

Element	Purpose for chatbot	Example
<code><data name="service-line"></code>	Links content to Extense service offerings	dita-engineering publishing-engineering training

Writing for retrieval quality

These authoring practices improve chatbot answer quality:

1. **Write clear shortdescs.** The shortdesc is prepended to every chunk from that topic. A vague shortdesc degrades every chunk.
2. **Use section titles as questions.** Instead of "Publishing Overview", use "How does DITA publishing work?" — this matches how users ask the chatbot.
3. **Keep sections focused.** One section = one concept. The chunking pipeline splits at section boundaries.
4. **Front-load key information.** The first 1-2 sentences of each section are the most important for retrieval ranking.
5. **Use DITA terms consistently.** The glossary is used for term resolution — always use the canonical glossterm form.
6. **Include code examples.** Codeblocks with `outputclass="language- *"` are preserved in chunks and are highly valued in retrieval.
7. **Add indexterms for synonyms.** If users might search for "filter" or "conditional processing", include both as indexterms even if the topic title says "DITaval."

Understanding chunking boundaries

The ingestion pipeline chunks content at **section boundaries**. Each chunk:

- Inherits the topic's title, shortdesc, and all prolog metadata
- Gets its own section title and section ID
- Is sized between 50 and 1500 tokens (large sections are split by paragraph)
- Is prefixed with the topic title + section title for embedding context

To optimize chunking:

- Give every `<section>` both an `@id` and a `<title>`
- Keep sections between 200-1200 tokens (roughly 1-6 paragraphs)
- Avoid sections that are just a single sentence
- Put related information in the same section rather than splitting across multiple

Contributing to the glossary

The glossary serves as the chatbot's **term resolution engine**. When adding terms:

- Always include `<glossSurfaceForm>` (the full/formal name)
- Add `<glossAlt><glossAbbreviation>` for common abbreviations
- Write `<glossUsage>` as practical guidance, not just definitions

- Include `<glossScopeNote>` for version applicability and context

Running the ingestion pipeline

After updating content, regenerate the chatbot data:

```
# From the organized-dita-docs directory:
python3 dita-topics/intelligent-content/dita-to-chatbot-json.py \
  --input dita-topics \
  --output dita-topics/intelligent-content/chatbot-data \
  --format jsonl
```

This produces:

- `chunks.jsonl` — Embed into your vector database
- `content-manifest.json` — Topic index for the chatbot navigator
- `glossary-index.json` — Term lookup for the chatbot
- `ingestion-report.json` — Quality metrics and statistics

Related concepts

[Topic-Based Information Architecture for Training](#) on page 76

How to design a modular, map-driven information architecture that supports progressive learning paths, role-based content delivery, and AI-powered navigation.

[Taxonomy and Controlled Vocabulary for AI Classification](#) on page 78

How a formal DITA subject scheme taxonomy provides the controlled vocabulary that AI systems need to classify, filter, and route content accurately.

Related tasks

[Metadata Enrichment: Adding the AI Layer to Every Topic](#) on page 94

Step-by-step process for adding the complete AI-ready prolog metadata to DITA topics — the single highest-impact action for improving chatbot retrieval quality.

[Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step](#) on page 84

A complete, repeatable process for converting unstructured XML files into semantically rich, AI-ready DITA topics with full prolog metadata.

DITA to Schema.org Type Mapping

Reference mapping of DITA topic types, elements, and metadata to schema.org structured data types for AI-readable output and chatbot classification.

Topic type to schema.org mapping

DITA Type	Schema.org Type	Key Properties	Notes
concept	TechArticle	headline, description, author, datePublished, proficiencyLevel, dependencies	Default for explanatory content
task	HowTo	name, description, step[].text, totalTime, tool	Steps extracted from <steps>
reference	TechArticle	headline, description, articleSection	With @type additions for API docs
glossentry	DefinedTerm	name, description, termCode, inDefinedTermSet	Links to DefinedTermSet for the glossary
glossgroup	DefinedTermSet	name, description, hasDefinedTerm[]	Container for all glossary entries
bookmap	Course	name, description, provider, hasCourseInstance, coursePrerequisites	Top-level training guide wrapper
part	CourseInstance	name, description, courseMode	Parts map to course modules

Metadata element mapping

DITA Metadata	Schema.org Property	Transformation
<shortdesc>	description	Direct text extraction
<title>	headline / name	Direct text extraction

DITA Metadata	Schema.org Property	Transformation
<author>	author.name	Wrap in Organization type
<critdates><created>	datePublished	ISO 8601 format
<critdates><revised>	dateModified	ISO 8601 format
<indexterm>	keywords	Comma-separated string
<data name="difficulty">	proficiencyLevel	Map: beginner#Beginner, etc.
<data name="duration">	timeRequired	Direct ISO 8601 duration
<data name="prerequisites">	coursePrerequisites	Resolve to topic titles
<data name="service-line">	provider.name	Map to Extense service name

Content intent to chatbot routing

The `content-intent` metadata drives chatbot answer formatting:

Intent	Chatbot Behavior	Response Format
explain	Provide conceptual explanation with context	Prose paragraph
how-to	Give step-by-step instructions	Numbered steps
reference	Return specific data/values	Table or list
troubleshoot	Diagnose problem and suggest fix	Problem # Cause # Solution
tutorial	Walk through hands-on exercise	Steps with code examples
definition	Look up term in glossary first	Term + definition + usage

Related concepts

[RAG Pipeline Architecture: From DITA to Chatbot Answers](#) on page 170

How a Retrieval-Augmented Generation pipeline transforms structured DITA content into accurate, cited chatbot responses — architecture overview and component roles.

Related tasks

[Section-Aware Chunking and Embedding Strategy](#) on page 172

How DITA's section structure enables semantically coherent chunks that preserve metadata context, and how those chunks are transformed into high-dimensional vectors for similarity search.

Part

VI

Publishing & Automation

Topics:

- [Publishing with DITA-OT](#)
- [Multi-Channel Output Formats](#)
- [DITA Project Files](#)
- [Output Customization Without Plugins](#)
- [CI/CD Publishing Pipelines](#)
- [Troubleshooting DITA Builds](#)

Publishing with DITA-OT

DITA is a source format, not an output. Publishing is the process of:

What publishing means in DITA

DITA is a **source format**, not an output. Publishing is the process of:

- resolving references (conref/keyref),
- applying filtering rules,
- transforming XML into HTML/PDF/other formats,
- applying styling and navigation rules,
- producing deliverables that match your product and brand.

Many DITA programs use the **DITA Open Toolkit (DITA#OT)** as the publishing engine. Others use CCMS publishing services that may internally rely on DITA#OT or custom pipelines.

This topic focuses on practical publishing with DITA#OT because it is widely used and teaches you the core concepts that apply everywhere.

Inputs and outputs: what DITA-OT expects

Input

DITA#OT publishes from a **map** (most common) or a single topic. In professional workflows:

- maps are preferred because they define structure, keys, and context.

Output formats (common)

- **HTML5**: website-like output with navigation
- **PDF**: often XSL#FO based (toolchain-dependent)
- Other outputs via plugins or extensions

The key idea

A map is not just a TOC; it is the **publication definition**. If you publish the same topic under two different maps, you can get different results because:

- key definitions differ,
- filtering differs,
- navigation context differs.

The “build pipeline” steps (what happens during publishing)

Publishing is more than “transform XML to HTML.”

A typical pipeline includes:

1. **Preprocess** — parse the map, resolve topicrefs, collect key definitions, gather referenced topics and resources.
2. **Reference resolution** — apply conref pulls, resolve keyrefs, validate IDs and targets.

3. **Filtering** — include/exclude content based on rules, adjust link targets accordingly.
4. **Transform** — generate intermediate or final formats, apply templates and rules for HTML/PDF.
5. **Postprocess** — create navigation and search index assets, copy images/media, apply final styling.

Understanding these stages helps you debug issues:

- missing content (filtering),
- broken links (IDs, key spaces),
- wrong product name (keys),
- missing images (resource paths).

Basic DITA-OT commands (the 80/20)

Publish HTML5

```
dita -i maps/guides/getting-started.ditamap -f html5 -o out/html
```

Publish PDF (if configured)

```
dita -i maps/guides/getting-started.ditamap -f pdf -o out/pdf
```

Practical tips

- Always use an explicit output directory (`-o`).
- Keep your build artifacts out of your source tree where possible.
- Use a consistent build script so builds are repeatable.

Repeatable builds: project files and build scripts

As soon as you have:

- multiple deliverables,
- filtering variants,
- consistent parameters,

...you should formalize builds.

Why repeatability matters

Repeatable builds ensure:

- anyone can build the same output,
- CI builds match local builds,
- releases are reproducible.

A typical approach

- Create a build script (`build.sh` or `build.ps1`) that runs defined publications.
- Use DITA#OT project files (where supported in your version) or a wrapper configuration.

Even without deep customization, a script with consistent parameters eliminates “works on my machine” issues.

Parameters: controlling output behavior

DITA#OT allows parameters to control:

- toc depth,
- chunking behavior,
- navigation,
- filtering integration,
- output artifacts.

In practice, you'll maintain:

- a baseline parameter set for HTML,
- a baseline parameter set for PDF,
- variant overrides per product/platform.

Common DITA-OT parameters (all transtypes)

These parameters work across most output formats:

Parameter	Values / Example	What it does
<code>-i / --input</code>	<code>maps/book.ditamap</code>	Input map or topic file
<code>-f / --format</code>	<code>html5</code> , <code>pdf</code> , <code>xhtml</code>	Output transtype
<code>-o / --output</code>	<code>out/html5</code>	Output directory
<code>--filter</code>	<code>build/ditaval/web.ditaval</code>	DITaval filter file for conditional processing
<code>--propertyfile</code>	<code>build/html5.properties</code>	Load parameters from a properties file
<code>--processing-mode</code>	<code>strict</code> , <code>lax</code> , <code>skip</code>	How to handle errors (<code>strict</code> = fail on warnings)
<code>--temp</code>	<code>/tmp/dita-temp</code>	Temporary directory (useful for debugging)
<code>-v / --verbose</code>	<i>(flag)</i>	Enable verbose logging
<code>--debug</code>	<i>(flag)</i>	Enable debug logging (very detailed)

HTML5-specific parameters

Parameter	Values / Example	What it does
<code>--nav-toc</code>	<code>none</code> , <code>partial</code> , <code>full</code>	Table of contents generation in each page
<code>--args.copycss</code>	<code>yes</code> , <code>no</code>	Copy custom CSS to output directory
<code>--args.css</code>	<code>brand.css</code>	Name of custom CSS file to link

Parameter	Values / Example	What it does
<code>--args.cssroot</code>	<code>css/</code>	Source directory for CSS files
<code>--args.csspath</code>	<code>css</code>	Path in output where CSS is placed
<code>--args.hdr</code>	<code>header.xml</code>	Custom header file for every page
<code>--args.ftr</code>	<code>footer.xml</code>	Custom footer file for every page
<code>--args.gen.task.lbl</code>	<code>YES</code> , <code>NO</code>	Generate step labels ("Step 1", "Step 2")
<code>--args.rellinks</code>	<code>none</code> , <code>all</code> , <code>nofamily</code>	Which related links to include
<code>--args.outext</code>	<code>.html</code>	Output file extension

PDF-specific parameters

Parameter	Values / Example	What it does
<code>--args.bookmap-order</code>	<code>discard</code> , <code>retain</code>	Keep or discard bookmap element order
<code>--args.chapter.layout</code>	<code>MINITOC</code> , <code>BASIC</code>	Chapter page layout style
<code>--args.fo.userconfig</code>	<code>fop.xconf</code>	FOP configuration file
<code>--custom.xep.config</code>	<code>xep.xml</code>	XEP formatter configuration
<code>--pdf.version</code>	<code>1.5</code> , <code>1.7</code>	PDF version to generate

Using a properties file (recommended for teams)

Instead of long command lines, put stable settings in a `.properties` file:

```
# build/html5-sidebar.properties
nav-toc          = full
args.copycss     = yes
args.cssroot     = css/
args.css         = brand.css
args.gen.task.lbl = YES
args.rellinks    = nofamily
```

Then build with:

```
dita -i maps/book.ditamap -f html5 -o out/html5 \
  --propertyfile=build/html5-sidebar.properties \
  --filter=build/ditaval/web.ditaval
```

Example: passing parameters (Ant-style)

```
dita -i maps/guides/getting-started.ditamap -f html5 -o out/html \
  -Dargs.gen.task.lbl=true \
  -Dargs.rellinks=none
```

Note

Note: The `-D` prefix syntax is the Ant-style parameter passing. The `--args.*` syntax is recommended for DITA-OT 3.x+.

The exact parameters you use depend on your output requirements and your DITA-OT version.

Filtering in the build (DITaval concept)

Filtering is usually applied at build time by providing filtering rules.

Typical workflow:

1. Tag content with attributes (`platform="windows"`).
2. Create a filter file that includes/excludes values.
3. Run separate builds for each variant.

Example build strategy

- `build-windows-html`
- `build-linux-html`
- `build-windows-pdf`
- `build-linux-pdf`

Even if you later automate this, it's valuable to think of variants as explicit publications.

Output customization: where to change what

HTML customization

Common customization areas:

- CSS (theme),
- HTML templates and headers/footers,
- navigation behavior,
- search integration (depending on output).

A practical approach:

- start with CSS and minimal HTML template overrides,
- avoid deep XSLT changes until necessary.

PDF customization

PDF often requires:

- FO styling rules,
- page masters, headers/footers,
- tables and code block formatting,
- numbering and cross-references.

PDF customization can become a deep specialization area. Start small:

- change fonts, margins, headers,
- adjust table styles,

- make code blocks readable.

Then expand:

- custom cover pages,
- running headers and footers,
- index generation and glossaries (if your pipeline supports them).

Plugin customization

DITA#OT supports plugins to:

- add new transforms,
- override templates,
- inject preprocessing steps,
- create reports.

Use plugins when:

- you need consistent custom behavior across builds,
- you want to package customization cleanly,
- parameter overrides aren't enough.

Debugging build problems (a practical checklist)

Problem: Content missing

- Check filtering rules.
- Check map inclusion (topicref points to correct file).
- Check if a topic is excluded by conditional processing attributes.
- Check if content is pulled via conref and the target is missing.

Problem: Broken links or unresolved references

- Validate IDs and references.
- Ensure topic IDs and element IDs exist.
- Check for renames that weren't propagated.

Problem: Wrong product name or wrong URL

- Check keys in the map.
- Ensure you are building the correct map.
- Check key scopes if advanced map structures exist.

Problem: Images missing

- Ensure image paths are correct and copied into output.
- Ensure images are referenced with correct relative paths.
- Check whether filtering excludes topics that provide the images.

Problem: PDF formatting issues

- Inspect FO output if accessible.
- Identify whether the problem is content structure (tables, code blocks) or styling.
- Apply minimal overrides first.

CI and automation: publishing like software

A mature DITA pipeline looks like software CI:

- builds are triggered by commits,
- validation runs automatically,
- outputs are versioned and deployed.

Practical CI steps

1. Validate XML (well-formed + schema/DTD as applicable).
2. Run DITA#OT build for each required publication.
3. Fail if warnings exceed a threshold (ideally fail on unresolved refs).
4. Publish artifacts to:

- documentation portal, - release package, - internal storage.

Output deployment patterns

- Static site hosting for HTML outputs,
- artifact repositories for PDFs,
- CCMS publishing service for enterprise portals.

Professional publishing architecture (how doc teams scale)

As documentation grows, teams typically separate concerns:

- **Authoring repo** (topics, maps, reuse, media)
- **Build system repo** (pipeline scripts, configs)
- **Brand/theme repo** (CSS, templates, PDF themes)
- **Deployment system** (portal, CDN, internal hosting)

This separation:

- keeps authors focused,
- keeps customization controlled,
- supports multiple brands/products.

A practical “starter publishing setup” (copyable blueprint)

Suggested folder layout

```
dita/  
  maps/  
  topics/  
  reuse/  
  media/  
  ditaval/  
  build/  
    build.sh  
    build.ps1  
    params/  
      html.properties  
      pdf.properties
```

What goes where

- `maps/`: deliverable definitions
- `topics/`: content
- `reuse/`: shared snippets
- `ditaval/`: filtering rules per variant
- `build/`: scripts + parameters

Build script responsibilities

- pick input maps,
- pick output formats,
- apply filter rules,
- pass consistent parameters,
- place outputs in a clean directory.

Common publishing mistakes (and how to avoid them)

Mistake: Publishing topics directly instead of maps

Fix: publish maps so key spaces and navigation context are consistent.

Mistake: Customizing too early

Fix: start with minimal overrides; grow customization as requirements become clear.

Mistake: Not treating build warnings as serious

Fix: fail builds on unresolved references and broken conrefs; monitor warning counts.

Mistake: No repeatability

Fix: scripts/project files; document required DITA#OT version and plugins.

Mistake: Mixing build outputs into source

Fix: separate `out/` from source and keep it out of version control.

Analytics and Instrumentation

Instrument publishing outputs to track which formats and outputs are most used. Use analytics to identify popular outputs and troubleshoot publishing issues.

Personalization and Conditional Processing

Use DITA conditional processing to generate branded or generic outputs. Example:

```
<section product="acme">Acme-branded output instructions.</section>
```

Integrating with Conversational AI and NLP

Integrate publishing FAQs and troubleshooting steps with chatbots to provide instant support for publishing issues.

Continuous Improvement and Feedback Loops

Add a feedback widget to publishing documentation. Use analytics and user feedback to improve publishing instructions and resolve common pain points.

Related tasks

[DITA-OT Architecture and DevOps](#) on page 132

This handbook is for engineers who want to develop DITA Open Toolkit—either by contributing to the core toolkit or by building production-grade transformations and pipelines with plug-ins, containers, and CI/CD.

Related reference

[DITA-OT Architecture Diagram](#) on page 188

ASCII reference diagram showing the complete DITA-OT HTML5 processing pipeline from inputs through preprocessing, transformation, and output.

Multi-Channel Output Formats

DITA-OT transforms structured source into HTML5, PDF, EPUB, Markdown, and custom formats — each from the same map with no content duplication.

One source, many outputs

The core value of DITA publishing is **single-source, multi-channel delivery**. You author once in structured XML, then transform the same content into every format your audience needs — without maintaining parallel copies.

DITA-OT ships with built-in transforms (called *transtypes*) and supports additional formats through installable plugins.

Built-in transtypes

DITA-OT includes these transforms out of the box:

Table 1: Built-in DITA-OT transtypes

Transtype	Description	Use case
<code>html5</code>	Modern HTML5 with responsive navigation, search index, and CSS hooks	Documentation portals, internal wikis
<code>pdf</code>	XSL-FO pipeline rendered via Apache FOP — produces print-ready PDFs	Regulatory filings, printed manuals
<code>xhtml</code>	Legacy XHTML 1.0 output for older systems	Embedding in legacy CMS platforms

Transtype	Description	Use case
<code>markdown / markdown_github</code>	Exports DITA content back to Markdown for developer workflows	GitHub READMEs, developer docs
<code>tocjs</code>	HTML with JavaScript-driven table of contents	Standalone doc bundles

Plugin-based formats

The DITA-OT plugin registry and community provide additional transtypes:

- **EPUB** — `org.dita.epub` generates EPUB 3 eBooks for e-readers and mobile reading apps.
- **Normalized DITA** — `org.dita.normalize` resolves all conrefs and keyrefs into flat, standalone topics for CMS import.
- **JSON / YAML** — Community plugins can export topic metadata and content as structured data for API documentation or chatbot ingestion.
- **Custom HTML5** — Branded plugins (like `com.extense.html5.branded`) extend the built-in HTML5 transform with custom CSS, headers, footers, and scripts.
- **Custom PDF** — Branded plugins (like `com.extense.pdf.branded`) override FO templates for corporate fonts, logos, and page layouts.

Install a plugin with:

```
dita --install <plugin-zip-or-url>
```

Choosing the right format

Match the output format to the delivery channel:

- **Web portal** # HTML5 (with branded plugin for navigation and styling)
- **Printed manual or regulatory submission** # PDF
- **E-reader / offline mobile** # EPUB
- **Developer repository** # Markdown (GitHub-flavored)
- **AI / chatbot pipeline** # JSON export or normalized DITA for RAG ingestion
- **CMS migration** # Normalized DITA with all references resolved

You can — and should — build multiple formats from the same map in a single CI pipeline. This is the **write-once-publish-everywhere** promise of DITA.

Format-specific considerations

HTML5

- Use `args.cssroot` and `args.css` to inject custom stylesheets.
- The `args.hdr` and `args.ftr` parameters add header/footer includes.
- Search indexes are generated automatically — enable with `args.indexshow=yes`.

PDF

- PDF generation uses Apache FOP by default. For higher fidelity, consider Antenna House or RenderX XEP.
- Use `args.chapter.layout=MINITOC` to add mini table-of-contents at chapter starts.
- Custom fonts require FOP configuration changes (see the XSLT customization part of this guide).

EPUB

- Cover images: set via `args.epub.coverimage`.
- EPUB 3 supports embedded video and audio — DITA `<object>` elements map automatically.

Markdown

- Markdown export is lossy — DITA metadata, conrefs, and keys do not survive the round-trip.
- Use markdown export for developer consumption, not as an authoring format.

Building multiple formats from one map

Use a DITA-OT project file (`.xml`) to define multiple deliverables in a single build:

```
<project xmlns="https://www.dita-ot.org/project">
  <context name="training-guide">
    <input href="extense-supermap.ditamap"/>
  </context>
  <deliverable name="html5">
    <context idref="training-guide"/>
    <output href="./output/html5"/>
    <publication transtype="html5">
      <param name="args.css" value="brand.css"/>
    </publication>
  </deliverable>
  <deliverable name="pdf">
    <context idref="training-guide"/>
    <output href="./output/pdf"/>
    <publication transtype="pdf">
      <param name="args.chapter.layout" value="MINITOC"/>
    </publication>
  </deliverable>
</project>
```

Run with: `dita --project=build.xml`

Related tasks

[DITA Project Files](#) on page 115

Project files define reusable, shareable build configurations that replace long command-line arguments with declarative XML — enabling consistent, repeatable builds across teams and CI systems.

[CI/CD Publishing Pipelines](#) on page 121

Automate DITA builds with GitHub Actions, Jenkins, and Azure DevOps so every commit produces validated, branded deliverables — no manual builds required.

Related reference

[Troubleshooting DITA Builds](#) on page 125

Diagnose and fix common DITA-OT build failures — broken links, missing content, PDF rendering issues, memory errors, and plugin conflicts — using verbose logging, FOP diagnostics, and systematic debugging.

DITA Project Files

Project files define reusable, shareable build configurations that replace long command-line arguments with declarative XML — enabling consistent, repeatable builds across teams and CI systems.

Why project files?

Without project files, every build requires a long command line:

```
dita -i maps/guide.ditamap -f html5 -o out/html5 \  
  --args.css=brand.css --args.cssroot=css \  
  --args.hdr=includes/header.xml \  
  --args.indexshow=yes -v
```

This is error-prone, hard to share, and impossible to version-control meaningfully. Project files solve all three problems by capturing the entire build definition in XML.

Project file anatomy

A DITA-OT project file uses the <https://www.dita-ot.org/project> namespace and contains three core elements:

<code><context></code>	Names the input map and any shared settings. Multiple deliverables can reference the same context, ensuring consistency.
<code><deliverable></code>	Defines one output — its transtype, output directory, and parameters. A project can contain many deliverables.
<code><publication></code>	Nested inside a deliverable — specifies the transtype and its parameter overrides.

Minimal example

A project file that builds both HTML5 and PDF from the same map:

```
<project xmlns="https://www.dita-ot.org/project">  
  <context name="training-guide" id="ctx-training">  
    <input href="extense-supermap.ditamap"/>  
  </context>  
  <deliverable name="Web Site">  
    <context idref="ctx-training"/>  
    <output href="./output/html5"/>  
    <publication transtype="html5">
```

```

        <param name="args.css"      value="brand.css"/>
        <param name="args.cssroot" value="css"/>
        <param name="nav-toc"      value="full"/>
    </publication>
</deliverable>

<deliverable name="Print Manual">
    <context idref="ctx-training"/>
    <output href="./output/pdf"/>
    <publication transtype="pdf">
        <param name="args.chapter.layout" value="MINITOC"/>
        <param name="args.bookmap-order" value="retain"/>
    </publication>
</deliverable>
</project>

```

Run with a single command:

```
dita --project=build-project.xml
```

DITA-OT builds every deliverable in sequence, reusing the shared context.

Advanced patterns

Profile-specific deliverables

Use DITAVAL profiles inside a context to produce filtered variants:

```

<context name="admin-guide" id="ctx-admin">
    <input href="guide.ditamap"/>
    <profile>
        <ditaval href="filters/admin-only.ditaval"/>
    </profile>
</context>

```

Multiple maps, one project

Define separate contexts for different maps and build them all together:

```

<context name="user-guide" id="ctx-user">
    <input href="maps/user-guide.ditamap"/>
</context>
<context name="api-reference" id="ctx-api">
    <input href="maps/api-ref.ditamap"/>
</context>

```

Including other project files

Project files can include other project files for modular build definitions:

```

<project xmlns="https://www.dita-ot.org/project">
    <include href="html5-deliverables.xml"/>
    <include href="pdf-deliverables.xml"/>
</project>

```

Properties files vs. project files

DITA-OT also supports `.properties` files — simple key=value pairs loaded with `--propertyfile`. Choose the right tool:

Table 2: Properties files vs. project files

Capability	Properties file	Project file
Multiple deliverables in one file	No	Yes
Shared contexts	No	Yes
Include/compose other files	No	Yes
DITaval profiles	Via args.filter	Native <profile>
Simplicity	Very simple	More verbose

Use properties files for quick, single-transtype builds. Use project files when you manage multiple deliverables, filtered variants, or CI pipelines.

Best practices

- **Version-control your project files** alongside your DITA source — they are part of your publication definition.
- **Name deliverables clearly** — `name="Admin PDF (v3.2)"` makes build logs easy to scan.
- **Use relative paths** for input maps and output directories so the project works across machines.
- **Keep one project file per product** — include shared definitions from a common file to avoid duplication.
- **Test locally before CI** — run `dita --project=build.xml` on your machine to catch configuration errors early.

Related concepts

[Multi-Channel Output Formats](#) on page 112

DITA-OT transforms structured source into HTML5, PDF, EPUB, Markdown, and custom formats — each from the same map with no content duplication.

[Output Customization Without Plugins](#) on page 118

Customize HTML5 and PDF output using built-in DITA-OT parameters, properties files, CSS injection, and DITaval filtering — no plugin development required.

Related reference

[Troubleshooting DITA Builds](#) on page 125

Diagnose and fix common DITA-OT build failures — broken links, missing content, PDF rendering issues, memory errors, and plugin conflicts — using verbose logging, FOP diagnostics, and systematic debugging.

Output Customization Without Plugins

Customize HTML5 and PDF output using built-in DITA-OT parameters, properties files, CSS injection, and DITAVAL filtering — no plugin development required.

Customization without code

Many teams assume output customization requires writing XSLT plugins. In practice, DITA-OT provides **dozens of built-in parameters** that control layout, styling, navigation, and content inclusion — all configurable from the command line, properties files, or project files.

This topic covers every customization you can make **without writing any code**.

HTML5 customization parameters

The HTML5 transtype supports extensive customization through parameters:

Table 3: Key HTML5 parameters

Parameter	Effect	Example value
<code>args.css</code>	Custom CSS file injected into every page	<code>brand.css</code>
<code>args.cssroot</code>	Directory containing the CSS file	<code>css/</code>
<code>args.copycss</code>	Copy the CSS file to the output directory	<code>yes</code>
<code>args.hdr</code>	HTML file included as a page header	<code>includes/header.xml</code>
<code>args.ftr</code>	HTML file included as a page footer	<code>includes/footer.xml</code>
<code>nav-toc</code>	Navigation TOC style: none, partial, or full	<code>full</code>
<code>args.indexshow</code>	Show index entries in output	<code>yes</code>
<code>args.gen.task.lbl</code>	Generate labels for task steps (Prerequisites, Steps, etc.)	<code>YES</code>

Example command:

```
dita -i map.ditamap -f html5 -o out/html5 \
  --args.css=brand.css --args.cssroot=css --args.copycss=yes \
```

```
--args.hdr=includes/header.xml --nav-toc=full
```

CSS-only branding

The fastest way to brand HTML5 output is with a custom CSS file. DITA-OT generates semantic HTML with predictable class names, making CSS targeting straightforward:

```
/* Brand colors */
:root {
  --brand-primary: #1a3a5c;
  --brand-accent: #e8721c;
}

/* Page header */
header.brand-header {
  background: var(--brand-primary);
  color: white;
  padding: 1rem 2rem;
}

/* Topic titles */
.topictitle1 {
  color: var(--brand-primary);
  border-bottom: 3px solid var(--brand-accent);
}

/* Code blocks */
pre.codeblock {
  background: #f4f4f4;
  border-left: 4px solid var(--brand-accent);
  padding: 1rem;
}

/* Navigation sidebar */
nav[role="toc"] {
  background: #f9f9f9;
  border-right: 1px solid #ddd;
}
```

Save as `brand.css` and reference via `args.css`.

PDF customization parameters

The PDF transtype also supports customization without XSLT overrides:

Table 4: Key PDF parameters

Parameter	Effect	Example value
<code>args.chapter.layout</code>	Add mini-TOC at the start of each chapter	<code>MINITOC</code>
<code>args.bookmap-order</code>	Preserve the bookmap ordering in output	<code>retain</code>
<code>args.bookmark.style</code>	PDF bookmark (outline) expansion	<code>EXPANDED</code>

Parameter	Effect	Example value
<code>args.fo.userconfig</code>	Custom Apache FOP configuration (fonts, renderers)	<code>fop.xconf</code>
<code>args.gen.task.lbl</code>	Generate section labels in tasks	<code>YES</code>
<code>outputFile.base</code>	Override the output PDF filename	<code>training-guide</code>

Properties files

Instead of passing parameters on the command line, save them in a `.properties` file:

```
# build-html5.properties
args.css = brand.css
args.cssroot = css
args.copycss = yes
args.hdr = includes/header.xml
nav-toc = full
args.indexshow = yes
args.gen.task.lbl = YES
```

Use with:

```
dita -i map.ditamap -f html5 -o out/html5 \
  --propertyfile=build-html5.properties
```

Properties files are version-controllable, shareable, and keep your build commands clean.

DITaval filtering

DITaval files control which content appears in the output based on profiling attributes (`audience`, `platform`, `product`, etc.):

```
<val>
  <!-- Include only admin content -->
  <prop action="include" att="audience" val="admin"/>
  <prop action="exclude" att="audience" val="end-user"/>

  <!-- Flag deprecated content with a warning -->
  <prop action="flag" att="status" val="deprecated">
    <startflag imageref="icons/warning.png">
      <alt-text>Deprecated</alt-text>
    </startflag>
  </prop>

  <!-- Highlight new content -->
  <revprop action="flag" val="2.0"
    changebar="solid" color="green"/>
</val>
```


Apply with:

```
dita -i map.ditamap -f html5 -o out/ \
  --args.filter=filters/admin-only.ditaval
```

One map + multiple DITAVAl files = multiple audience-specific deliverables with zero content duplication.

Combining techniques

The most effective approach combines all three:

1. **Properties file** — captures all build parameters in a reusable, version-controlled file.
2. **CSS file** — handles visual branding for HTML5 without touching the transform.
3. **DITAVAl file** — controls content visibility for audience-specific variants.

Together, these three files give you branded, filtered output with zero plugin development — suitable for 80% of publishing requirements.

Related tasks

[DITA Project Files](#) on page 115

Project files define reusable, shareable build configurations that replace long command-line arguments with declarative XML — enabling consistent, repeatable builds across teams and CI systems.

[CI/CD Publishing Pipelines](#) on page 121

Automate DITA builds with GitHub Actions, Jenkins, and Azure DevOps so every commit produces validated, branded deliverables — no manual builds required.

Related reference

[Troubleshooting DITA Builds](#) on page 125

Diagnose and fix common DITA-OT build failures — broken links, missing content, PDF rendering issues, memory errors, and plugin conflicts — using verbose logging, FOP diagnostics, and systematic debugging.

CI/CD Publishing Pipelines

Automate DITA builds with GitHub Actions, Jenkins, and Azure DevOps so every commit produces validated, branded deliverables — no manual builds required.

Why automate DITA builds?

Manual builds create bottlenecks:

- Writers wait for a build engineer to produce outputs.
- Build parameters drift between machines — "it works on my laptop."
- Broken links and validation errors are discovered late.
- Publishing becomes a ceremony instead of a continuous process.

CI/CD pipelines solve these problems by running DITA-OT automatically on every push, pull request, or release tag — validating content, building outputs, and deploying deliverables without human intervention.

Pipeline architecture

A typical DITA CI/CD pipeline has four stages:

1. **Trigger** — a Git push, PR, or scheduled event starts the pipeline.
2. **Build environment** — a Docker container with DITA-OT and plugins pre-installed provides a clean, repeatable environment.
3. **Build** — DITA-OT runs against the project file, producing HTML5, PDF, or other outputs. Validation errors fail the build.
4. **Deploy** — successful outputs are published to a web server, artifact store, or CDN.

GitHub Actions example

This workflow builds HTML5 and PDF on every push to `main`:

```
name: Build DITA Documentation

on:
  push:
    branches: [main]
    paths: ['docs/**', 'maps/**', 'topics/**']
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    container:
      image: ghcr.io/dita-ot/dita-ot:4.3

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Install plugins
        run: |
          dita --install plugins/com.extense.html5.branded
          dita --install plugins/com.extense.pdf.branded

      - name: Build all deliverables
        run: dita --project=build-project.xml -v

      - name: Upload HTML5 artifact
        uses: actions/upload-artifact@v4
        with:
          name: html5-output
          path: output/html5/

      - name: Upload PDF artifact
        uses: actions/upload-artifact@v4
        with:
          name: pdf-output
          path: output/pdf/

      - name: Deploy to GitHub Pages
        if: github.ref == 'refs/heads/main'
        uses: peaceiris/actions-gh-pages@v3
```

```

with:
  github_token: ${ secrets.GITHUB_TOKEN }
  publish_dir: output/html5

```

Jenkins pipeline example

```

pipeline {
  agent {
    docker { image 'ghcr.io/dita-ot/dita-ot:4.3' }
  }
  triggers {
    pollSCM('H/5 * * * *')
  }
  stages {
    stage('Install Plugins') {
      steps {
        sh 'dita --install plugins/com.extense.html5.branded'
        sh 'dita --install plugins/com.extense.pdf.branded'
      }
    }
    stage('Build') {
      steps {
        sh 'dita --project=build-project.xml -v'
      }
    }
    stage('Archive') {
      steps {
        archiveArtifacts artifacts: 'output/**/*'
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps {
        sh 'rsync -avz output/html5/ docs-server:/var/www/docs/'
      }
    }
  }
  post {
    failure {
      mail to: 'docs-team@extense.com',
          subject: "DITA build failed: ${env.JOB_NAME}",
          body: "Check ${env.BUILD_URL} for details."
    }
  }
}

```

Azure DevOps pipeline

```

trigger:
  branches:
    include: [main]
  paths:
    include: ['docs/*', 'maps/*', 'topics/*']

pool:
  vmImage: 'ubuntu-latest'

container: ghcr.io/dita-ot/dita-ot:4.3

steps:
  - script: |

```

```

    dita --install plugins/com.extense.html5.branded
    dita --install plugins/com.extense.pdf.branded
    displayName: 'Install plugins'

- script: dita --project=build-project.xml -v
  displayName: 'Build deliverables'

- publish: output/html5
  artifact: html5-docs
  displayName: 'Publish HTML5 artifact'

- publish: output/pdf
  artifact: pdf-docs
  displayName: 'Publish PDF artifact'

```

Docker images for CI

The official DITA-OT Docker images are the recommended base for CI:

- ghcr.io/dita-ot/dita-ot:4.3 — standard image with all built-in plugins.
- Build a **custom image** with your plugins pre-installed for faster CI runs:

```

FROM ghcr.io/dita-ot/dita-ot:4.3
COPY plugins/com.extense.html5.branded /opt/app/plugins/com.extense.html5.branded
COPY plugins/com.extense.pdf.branded /opt/app/plugins/com.extense.pdf.branded
RUN dita --install

```

Push to your container registry and reference in CI for 30-60 second faster builds (no plugin install step needed).

Validation gates

Add validation steps before the build to catch problems early:

- **XML well-formedness** — `xmllint --noout topics/**/*.dita`
- **DTD/schema validation** — `xmllint --valid --dtdvalid` or Schematron rules
- **Link checking** — DITA-OT reports broken topicrefs and conrefs in verbose mode
- **Metadata completeness** — custom scripts that check every topic has required `<data>` elements (difficulty, duration, content-domain)

Fail the build on validation errors so broken content never reaches production.

Best practices

- **Use project files** — not inline command arguments — so CI and local builds produce identical results.
- **Pin the DITA-OT version** in your Docker image tag to avoid surprise regressions.
- **Cache dependencies** — cache the DITA-OT home directory between builds if not using Docker.
- **Build on PRs, deploy on merge** — PRs validate and produce preview artifacts; merges to `main` deploy to production.
- **Notify on failure** — send Slack, email, or Teams alerts when builds break.
- **Artifact retention** — keep build artifacts for 30-90 days for audit and rollback.

Related concepts

[Multi-Channel Output Formats](#) on page 112

DITA-OT transforms structured source into HTML5, PDF, EPUB, Markdown, and custom formats — each from the same map with no content duplication.

[Output Customization Without Plugins](#) on page 118

Customize HTML5 and PDF output using built-in DITA-OT parameters, properties files, CSS injection, and DITaval filtering — no plugin development required.

Related reference

[Troubleshooting DITA Builds](#) on page 125

Diagnose and fix common DITA-OT build failures — broken links, missing content, PDF rendering issues, memory errors, and plugin conflicts — using verbose logging, FOP diagnostics, and systematic debugging.

Troubleshooting DITA Builds

Diagnose and fix common DITA-OT build failures — broken links, missing content, PDF rendering issues, memory errors, and plugin conflicts — using verbose logging, FOP diagnostics, and systematic debugging.

The debugging mindset

DITA-OT builds fail for predictable reasons. The key is reading the right logs, understanding which pipeline stage failed, and applying targeted fixes. This topic covers the most common failures and their solutions.

Verbose and debug logging

Always start diagnosis with verbose output:

```
# Verbose mode — shows warnings and processing steps
dita -i map.ditamap -f html5 -o out/ -v

# Debug mode — maximum detail (very verbose)
dita -i map.ditamap -f html5 -o out/ -d

# Save log to file for analysis
dita -i map.ditamap -f html5 -o out/ -v 2>&1 | tee build.log
```

Key log prefixes to watch for:

- **[WARN]** — potential problems that do not stop the build but may cause missing content or broken links in the output.
- **[ERROR]** — problems that prevent successful output generation.
- **[FATAL]** — the build cannot continue at all.

Broken links and missing references

Symptom: Warnings like `DOTJ013W` or `DOTX036W` in the log — "file not found" or "target topic not found."

Common causes:

- **Wrong href path** — relative paths are resolved from the map's directory, not the topic's directory. Double-check your directory structure.
- **Typo in filename or ID** — DITA file references are case-sensitive on Linux/macOS.
- **Moved or renamed files** — update all maps and conrefs that reference the file.
- **Missing keydef** — a `keyref` points to a key not defined in the current map's key space.

Fix checklist:

1. Grep for the broken reference: `grep -r "filename.dita" maps/ submaps/`
2. Verify the file exists at the expected relative path.
3. Check key definitions in the root map and submaps.
4. Re-run with `-v` to confirm the warning is resolved.

Missing content in output

Symptom: Topics or sections that exist in your source do not appear in the output.

Common causes:

- **DITaval filtering** — content is being excluded by a profiling rule. Check your `--args.filter` settings and profiling attributes.
- **Topic not referenced in map** — a topic file exists but is not included in any `<topicref>`.
- **Broken conref** — a conref target does not resolve, so the content silently disappears.
- **processing-role="resource-only"** — the topicref is marked as resource-only (for key definitions) and is intentionally excluded from output.

Diagnosis:

```
# Check which topics were processed
dita -i map.ditamap -f html5 -o out/ -v 2>&1 | grep "Processing"

# Check filtering rules
grep -r "action=\"exclude\"" filters/
```

PDF-specific issues

Blank pages

- FOP inserts blank pages to maintain recto/verso page numbering in double-sided layouts. This is intentional for print output.
- To suppress: set `force-page-count="auto"` in your custom FO templates (requires plugin customization).

Missing or wrong fonts

- FOP only uses fonts it knows about. Register custom fonts in `fop.xconf`:

```
<renderer mime="application/pdf">
  <font>
    <font metrics-url="fonts/OpenSans-Regular.xml"
      embed-url="fonts/OpenSans-Regular.ttf"
      kerning="yes">
      <font-triplet name="Open Sans" style="normal" weight="normal"/>
    </font>
  </font>
</renderer>
```

Apply with: `--args.fo.userconfig=fop.xconf`

Table layout overflow

- Wide tables overflow the page. Use `colspec` with proportional widths (`colwidth="1"`) instead of fixed widths.
- Reduce font size in table cells via custom FO attribute sets.

Image resolution

- FOP renders images at their intrinsic resolution. For print, use images at 300 DPI minimum.
- Set explicit width/height in `<image>` elements to control sizing.

Memory and performance issues

Symptom: java.lang.OutOfMemoryError: Java heap space

Fix: Increase the JVM heap size before running DITA-OT:

```
# Set maximum heap to 4 GB
export ANT_OPTS="-Xmx4096m"
dita -i large-map.ditamap -f pdf -o out/
```

For very large publications (500+ topics), consider:

- Splitting into smaller maps built independently.
- Using map-first preprocessing (`preprocess2`) which is more memory-efficient.
- Running builds in Docker containers with controlled memory limits.

Plugin conflicts

Symptom: Build fails after installing a new plugin, or output looks wrong.

Diagnosis:

```
# List installed plugins
dita --plugins

# Check for duplicate template overrides
grep -r "xsl:template" plugins/*/xsl/ | grep "match="
```

Common issues:

- **Conflicting XSLT imports** — two plugins override the same template. Check `plugin.xml` for `dita.xsl.xhtml` extension points.
- **Missing dependencies** — a plugin requires another plugin that is not installed. Check `<require>` elements in `plugin.xml`.
- **Version mismatch** — plugin was built for an older DITA-OT version. Check the `<require>` for `org.dita.base` version constraints.

Nuclear option: Reinstall DITA-OT from scratch and add plugins one at a time to isolate the conflict.

Common error codes quick reference

Table 5: Frequently seen DITA-OT error codes

Code	Meaning	Action
<code>DOTX036W</code>	Target topic not available	Fix href path or add missing file
<code>DOTJ013W</code>	File not found	Check relative path from map directory
<code>DOTJ031E</code>	Conref target not found	Verify conref path and target ID
<code>DOTX050I</code>	Duplicate key definition	Review key scopes in sub-maps
<code>DOTA069W</code>	Key not resolved	Add keydef to root map or check scope
<code>PDFX004F</code>	FOP formatting error	Check table widths, image paths, fonts

Systematic troubleshooting workflow

1. **Read the log** — run with `-v` and search for `[ERROR]` and `[WARN]`.
2. **Identify the stage** — is the error in preprocessing (references), filtering, transformation, or postprocessing?
3. **Isolate** — build a minimal map with just the problem topic to rule out map-level issues.
4. **Validate XML** — run `xmllint --noout topic.dita` to check well-formedness.
5. **Check temp files** — examine `out/temp/` for intermediate files that show how DITA-OT processed your content.
6. **Compare** — diff a working topic against the broken one to spot differences.
7. **Fix and re-run** — make one change at a time and rebuild to confirm each fix.

Related concepts

[Multi-Channel Output Formats](#) on page 112

DITA-OT transforms structured source into HTML5, PDF, EPUB, Markdown, and custom formats — each from the same map with no content duplication.

[Output Customization Without Plugins](#) on page 118

Customize HTML5 and PDF output using built-in DITA-OT parameters, properties files, CSS injection, and DITAVAL filtering — no plugin development required.

Related tasks

[DITA Project Files](#) on page 115

Project files define reusable, shareable build configurations that replace long command-line arguments with declarative XML — enabling consistent, repeatable builds across teams and CI systems.

[CI/CD Publishing Pipelines](#) on page 121

Automate DITA builds with GitHub Actions, Jenkins, and Azure DevOps so every commit produces validated, branded deliverables — no manual builds required.

Part

VII

DITA-OT Development

Topics:

- [DITA-OT Architecture and DevOps](#)
- [Architecture and Preprocessing](#)
- [Customization and Plug-Ins](#)
- [DevOps and Operations](#)

DITA-OT Architecture and DevOps

This handbook is for engineers who want to develop DITA Open Toolkit—either by contributing to the core toolkit or by building production-grade transformations and pipelines with plug-ins, containers, and CI/CD.

This handbook covers the full DITA-OT development stack, from internal architecture through production deployment. It is based on the official DITA-OT development documentation and references.

Topics in this chapter include:

- **Architecture and Preprocessing** — the multi-stage pipeline, temp directory, preprocessing deep dive, and map-first [preprocess2](#)
- **Customization and Plug-Ins** — extension points, XSLT-import overrides, robust patterns, pipeline hooks, and a complete plug-in template
- **DevOps and Operations** — building from source, Docker workflows, GitHub Actions CI/CD, debugging, and repository layout

Note

Each topic in this chapter can be read independently. Start with the architecture topics if you are new to DITA-OT internals, or jump to the DevOps section for CI/CD recipes.

Related concepts

[Publishing with DITA-OT](#) on page 104

DITA is a source format, not an output. Publishing is the process of:

Related reference

[DITA-OT Architecture Diagram](#) on page 188

ASCII reference diagram showing the complete DITA-OT HTML5 processing pipeline from inputs through preprocessing, transformation, and output.

Architecture and Preprocessing

How DITA-OT Is Built and Its Core Architecture

DITA-OT implements a multi-stage, map-driven architecture where preprocessing dominates the complexity and the temp directory is the single source of truth for debugging.

The mental model

DITA-OT implements a **multi-stage, map-driven architecture**. Each stage examines some or all of the content; some stages generate temporary files used later, while others generate final outputs. Most processing happens in a **temporary working directory**, and **source files are never modified**.

You can think of every transtype as:

```

inputs (map/topics)
  |
  | preprocessing (common)
  v
normalized / resolved intermediates
  |
  | format-specific processing (HTML5, PDF, etc.)
  v
output

```

This is the key to fast debugging: **if the intermediate artifacts are wrong**, downstream output will be wrong.

Stages, temp dir, and why preprocessing dominates

Stages are designed around responsibility boundaries

DITA-OT's processing structure is:

- **map-first** or **mixed** preprocessing (depending on pipeline)
- transformation steps for topics and maps
- resource copying and final assembly

The "hard" part is not HTML or PDF formatting—it's the **DITA model itself**: keys, scoping, references, reuse, filtering, chunking.

The temp directory is your truth source

Because DITA-OT does not modify sources and performs most work in a temp dir, you can debug by comparing:

- preprocessed topic versions (resolved conrefs/keyrefs)
- filtered versions (DITaval applied)
- chunked result lists (how the build decided to split output)

DITA-OT explicitly describes this temp-dir-based processing behavior.

Preprocessing Deep Dive

DITA-OT preprocessing is a set of Ant-target steps that resolve keys, conrefs, chunking, and links before any format-specific transform runs.

Why preprocessing is structured as Ant targets

DITA-OT preprocessing is described as a **set of steps** that typically runs at the beginning of every transformation; each step corresponds to an **Ant target**, and the `preprocess` target calls the complete set.

Ant provides:

- ordering (depends chains)
- incremental injection points for plug-ins

- composable, declarative pipeline wiring (targets invoking Java or XSLT)

Common preprocessing responsibilities

A practical breakdown (you'll see these reflected in extension points and step names):

1. **clean-temp**: initialize working directory
2. **copy-files / copy-image / copy-subsidiary**: ensure resources are staged
3. **gen-list**: create file lists / job metadata
4. **mapref / mappull / topicpull**: map/topic "pull" logic (metadata, navtitle, etc.)
5. **keyref**: resolve keys and references
6. **conref / conrefpush**: resolve content reuse and push patterns
7. **coderef**: resolve code snippets pulled in
8. **chunk**: decide how topics are split (into pages or combined units)
9. **maplink**: generate relationship tables / link metadata
10. **debug-filter**: diagnostics related to filtering
11. plus additional steps depending on pipeline and output

The development docs emphasize that these steps are targets in the Ant pipeline.

Where to hook preprocessing (best practice)

DITA-OT recommends using extension points that run **before or after preprocessing**, and warns against hooking into individual internal steps unless absolutely necessary.

Recommended hooks:

- `depend.preprocess.pre` — run a target *before* preprocessing
- `depend.preprocess.post` — run a target *after* preprocessing

Why:

- your plug-in stays compatible across versions
- you avoid being tied to fragile internal step order (which can change between releases)

Legacy step hooks (use sparingly)

If you must hook inside preprocessing, DITA-OT documents legacy extension points such as:

- `depend.preprocess.conref.pre`
- `depend.preprocess.keyref.pre`
- `depend.preprocess.chunk.pre`
- etc.

But there's a strong caution: internal step order can change, so these should only be used when truly necessary.

Map-First Preprocessing (`preprocess2`) in 4.x

DITA-OT's map-first preprocessing completes map-level resolution before topic processing, simplifying key/scope handling and enabling topic-level filtering optimization.

Why map-first exists

DITA-OT provides a **map-first preprocessing** approach as an alternative to the default `preprocess`. It provides the same functionality but changes the processing model: it completes most map processing first and only begins topic processing after map-level features are resolved.

Later DITA versions added many map-level features (keys, key scopes, etc.) that make it harder to reliably process topics before map features are resolved. The map-first model formalizes "process the map first," simplifies responsibilities, and can process only the topics actually referenced after filtering.

4.x impact: HTML5 uses map-first as of 4.2

DITA-OT notes that map-first preprocessing is enabled by default for:

- PDF and HTML Help as of 3.0
- HTML5 as of **4.2** (after further testing and feedback)

Developer consequence: extension points available only in legacy `preprocess` pipelines might not apply, and step-specific hooks may be unavailable in `preprocess2`.

Calling `preprocess2` in a custom transtype

The docs describe updating a custom transtype's Ant target dependencies to call `preprocess2` instead of `preprocess`, and they show example `depends` chains for HTML5 variants.

Rule for teams: if you maintain production transforms used by many people, consider a "beta" transtype that is identical except it uses `preprocess2` so teams can test safely.

Extension Points and Plug-In Fundamentals

DITA-OT plug-ins use extension points declared in `plugin.xml` to integrate changes into the toolkit without modifying core code.

plugin.xml is the contract

DITA-OT provides a series of **extension points** used to integrate changes into core code. Extension points are defined in `plugin.xml`; when plug-ins are installed, the toolkit makes those extensions visible to the rest of the toolkit.

DITA-OT explains that `plugin.xml` controls all aspects of a plug-in, using predefined extension points to locate changes and integrate them into DITA-OT.

Also important for teams: DITA-OT provides a RELAX NG schema and describes validating `plugin.xml` with an `xml-model` processing instruction.

Plug-in naming and dependencies

- Plug-in IDs must be unique dot-delimited tokens (reverse-domain convention recommended).
- Dependencies are declared with `<require plugin="..." />` (critical for correct load order in XSLT and Ant integration).

Customization and Plug-Ins

Safe Customization Strategy: XSLT-Import Overrides

The recommended customization approach uses XSLT-import extension points so plug-ins override only what they need while remaining upgrade-safe.

Why this is the best default strategy

DITA-OT supports **XSLT-import extension points**, where the plug-in installer adds an `xsl:import` statement to default code so your override becomes part of the normal build.

- No edits to toolkit core
- Minimal overrides (only what you need)
- Upgrade-safe: you import the shipped transform and override specific templates

Minimal HTML5 XSLT override plugin.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-model href="https://www.dita-ot.org/rng/plugin.rnc" type="application/relax-ng-compact-syntax"?>
<plugin id="com.yourco.html5.custom" version="1.0.0">
  <require plugin="org.dita.html5"/>
  <feature extension="dita.xsl.html5" file="xsl/html5/override.xsl"/>
</plugin>
```

- The `plugin.xml` structure and validation method are described in the official docs.
- The XSLT-import mechanism and how it is integrated are described in the XSLT-import extension point docs.

override.xsl pattern

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="3.0">
  <xsl:import href="plugin:org.dita.html5:xsl/dita2html5.xsl"/>
  <!-- your overrides here -->
</xsl:stylesheet>
```


Writing Robust XSLT Customizations

Recommended XSLT patterns for DITA-OT customization: import layers, `@class` ancestry matching, wrapper overrides, and consistent hooks.

Recommended patterns

Even though the full OT pipeline is complex, most real customization work is XSLT. These patterns minimize upgrade pain:

1. **Import the default transform** — `xsl:import` the shipped XSL for your target output.
2. **Override templates by `@class` ancestry** — `match="*[contains(@class,' topic/p ')]"` to work across specializations.
3. **Prefer wrapper overrides** — Use `xsl:next-match` to keep default logic but add wrappers/classes.
4. **Add consistent hooks** — classes and `data-*` attributes for styling and analytics.

Adding Pipeline Steps: Pre/Post Preprocess and Legacy Hooks

How to inject custom Ant targets before or after preprocessing, and when legacy step-specific hooks are appropriate.

The stable approach: run before/after preprocessing

Use:

- `depend.preprocess.pre`
- `depend.preprocess.post`

Example `plugin.xml` snippet:

```
<plugin id="com.yourco.pipeline" version="1.0.0">
  <require plugin="org.dita.base"/>
  <feature extension="depend.preprocess.pre" file="integrator.xml"/>
</plugin>
```

Then in your Ant file (e.g., `integrator.xml`), define a target that runs before preprocess.

Legacy step hooks: when you must

The official docs list many `depend.preprocess.<step>.pre` extension points for legacy preprocess stages.

Use them only for scenarios like:

- injecting a normalization step before conref expansion
- generating intermediate metadata files consumed by later steps
- strict validations that must happen before chunk decisions

Map-first (`preprocess2`) caveat

Step-specific legacy hooks are not available in `preprocess2`. The map-first documentation and preprocessing extension point docs both highlight this constraint.

A Minimal Developer Plug-In Template

A complete starter template for an HTML5 customization plug-in including `plugin.xml`, topic override, TOC override, and build commands.

Directory structure

```
plugins/
  com.yourco.html5.custom/
    plugin.xml
    xsl/
      html5/
        override.xsl
      toc/
        toc-override.xsl
    css/
      brand.css
    js/
      site.js
```

plugin.xml (minimal)

```
<?xml version="1.0" encoding="UTF-8"?>
<plugin id="com.yourco.html5.custom">
  <require plugin="org.dita.html5"/>
  <feature extension="dita.xsl.html5" file="xsl/html5/override.xsl"/>
  <feature extension="dita.xsl.html5.toc" file="xsl/toc/toc-override.xsl"/>
</plugin>
```

Key points:

- `id` must be unique and follow reverse-domain naming.
- `require` ensures DITA-OT installs the HTML5 transtype first.
- Two extension points: `dita.xsl.html5` (topic transform) and `dita.xsl.html5.toc` (navigation).

override.xsl (skeleton)

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="3.0">
  <!-- Import the default HTML5 transform -->
  <xsl:import href="plugin:org.dita.html5:xsl/dita2html5.xsl"/>

  <!-- Override paragraphs: add data-dita-class for CSS/JS hooks -->
  <xsl:template match="*[contains(@class,' topic/p ')]">
    <p data-dita-class="{normalize-space(@class)}">
      <xsl:apply-templates/>
    </p>
  </xsl:template>
```

```
<!-- Override notes: style as callout boxes -->
<xsl:template match="*[contains(@class,' topic/note ')]">
  <div class="callout callout-{@type}" data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </div>
</xsl:template>
</xsl:stylesheet>
```

toc-override.xsl (skeleton)

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="3.0">
  <xsl:import href="plugin:org.dita.html5:xsl/toc.xsl"/>

  <xsl:template match="nav">
    <nav class="sidebar-toc">
      <xsl:apply-templates select="@* | node()"/>
    </nav>
  </xsl:template>

  <xsl:template match="li">
    <li data-level="{count(ancestor::li)+1}">
      <xsl:apply-templates select="@* | node()"/>
    </li>
  </xsl:template>
</xsl:stylesheet>
```

Install and build

```
# Copy plug-in into DITA-OT
cp -r plugins/com.yourco.html5.custom DITA-OT/plugins/

# Integrate
dita --install

# Build
dita -i maps/book.ditamap -f html5 -o out/html5 \
  --nav-toc=full \
  --args.copycss=yes \
  --args.cssroot=css/ \
  --args.css=brand.css
```

DevOps and Operations

Developing DITA-OT from Source

How to clone, build, test, and package DITA-OT from the official GitHub repository using Gradle and Java 17+.

You need JDK 17 or newer installed on your system.

1. Clone the repository with submodules.

```
git clone --recurse-submodules git://github.com/dita-ot/dita-ot.git
cd dita-ot
```

2. Build with Gradle.

```
./gradlew
```

This compiles Java and installs plug-ins as part of the build.

3. Run the test suite.

```
./gradlew check
```

The project runs tests in GitHub Actions on pushes and PRs.

4. Apply code formatting (Spotless / Prettier).

```
./gradlew spotlessApply
```

5. Build distributable packages.

```
./gradlew dist
```

Distributions are built under `build/distributions`.

Note

If Gradle runs out of memory, increase heap via `GRADLE_OPTS` (e.g., `-Xmx1024m`).

Docker for DITA-OT: Local Builds and Reproducible Toolchains

Using the official DITA-OT Docker image for consistent, reproducible builds that eliminate environment differences across developer machines and CI.

Canonical "run a build in a container" pattern

The official docs show how to run the `dita` command from a Docker image with prerequisites (Docker installed and access to GitHub Container Registry).

```
docker run --rm \
-v "$PWD":/src \
ghcr.io/dita-ot/dita-ot:4.4 \
-i /src/dita/maps/book.ditamap \
-o /src/out/html5 \
-f html5 -v
```

Why Docker improves OT development

- consistent Java/toolchain and OT version

- predictable plug-in install behavior
- easy CI parity (local build == CI build)

Build a Custom Docker Image with Plug-Ins Pre-Installed

Create a Dockerfile that extends the official DITA-OT image and pre-installs your plug-ins for faster CI builds and consistent toolchains.

DITA-OT explicitly documents creating a Dockerfile that uses the official image as a parent and runs `dita --install` to install plug-ins (from URL or registry).

1. Create a Dockerfile that installs plug-ins during image build.

```
FROM ghcr.io/dita-ot/dita-ot:4.4

# Option A: install from a remote ZIP
RUN dita --install https://github.com/infotexture/dita-bootstrap/archive/master.zip

# Option B: install from dita-ot.org plugin registry
# RUN dita --install org.dita-community.pdf-page-break
```

2. For private plug-ins, copy them into the image.

- `COPY` your plug-in ZIP into the image
- `RUN dita --install /path/to/plugin.zip`

This keeps CI fast and avoids downloading from private registries at runtime.

3. Build and tag the image.

```
docker build -t my-dita-ot:latest .
```

docker-compose for Doc Builds and Local Preview

Set up a docker-compose configuration that builds HTML output and serves it locally with nginx for instant preview.

A good "developer experience" setup builds HTML output into `out/html5` and serves it locally with a lightweight web server container.

1. Create a `docker-compose.yml` file.

```
services:
  build:
    image: ghcr.io/dita-ot/dita-ot:4.4
    working_dir: /src
    volumes:
      - ./:/src
    command: >
      -i /src/dita/maps/book.ditamap
      -o /src/out/html5
      -f html5
      --propertyfile=/src/build/html5.properties

  serve:
    image: nginx:alpine
    ports:
```

```
- "8080:80"
volumes:
- ./out/html5:/usr/share/nginx/html:ro
```

2. Run the build service.

```
docker compose run --rm build
```

3. Start the preview server and open <http://localhost:8080>.

```
docker compose up serve
```

GitHub Actions: CI/CD Publishing and Workflow Examples

Production-ready GitHub Actions workflows for PR checks, multi-variant builds, GitHub Pages deployment, and release packaging using the official dita-ot-action.

The official dita-ot-action

DITA-OT documents using GitHub Actions and notes that the project provides an official action (as of 3.6.1) to build docs as part of CI/CD. Steps run on the same runner, so build outputs can be archived or published by later steps.

The [dita-ot-action](#) repo README states the action installs DITA-OT and specified plug-ins and builds outputs like PDF or HTML.

PR checks: build HTML and upload as artifact

[.github/workflows/dita-pr.yml](#)

```
name: DITA PR build

on:
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build HTML5
        uses: dita-ot/dita-ot-action@master
        with:
          input: dita/maps/book.ditamap
          transtype: html5
          output-path: out/html5

      - name: Upload HTML artifact
        uses: actions/upload-artifact@v4
        with:
          name: html5
          path: out/html5
```

Multi-variant builds: audience/platform filters as matrix

`.github/workflows/dita-variants.yml`

```
name: DITA variants

on:
  push:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        variant:
          - { name: web,      ditaval: build/ditaval/web.ditaval,    out: out/html5-web }
          - { name: admin,   ditaval: build/ditaval/admin.ditaval,  out: out/html5-admin }
    steps:
      - uses: actions/checkout@v4

      - name: Build ${ matrix.variant.name }
        uses: dita-ot/dita-ot-action@master
        with:
          build: |
            dita -i dita/maps/book.ditamap -f html5 -o ${ matrix.variant.out } \
              --propertyfile=build/html5.properties \
              --args.filter=${ matrix.variant.ditaval }

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: ${ matrix.variant.name }
          path: ${ matrix.variant.out }
```

Deploy HTML to GitHub Pages on main

`.github/workflows/pages.yml`

```
name: Publish docs to Pages

on:
  push:
    branches: [ "main" ]

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build HTML5 site
        uses: dita-ot/dita-ot-action@master
```

```

    with:
      input: dita/maps/book.ditamap
      transtype: html5
      output-path: out/site
  - name: Upload Pages artifact
    uses: actions/upload-pages-artifact@v3
    with:
      path: out/site

deploy:
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: github-pages
    url: ${{ steps.deployment.outputs.page_url }}
  steps:
    - id: deployment
      uses: actions/deploy-pages@v4

```

Release builds: attach outputs to GitHub Releases

At release time you may want:

- HTML artifact zip
- PDF artifact
- source package

Use [actions/upload-artifact](#) for PRs and [softprops/action-gh-release](#) (or similar) for release tagging.

Debugging Playbook

Systematic approach to isolating DITA-OT build failures across the three stages: preprocessing, transformation, and packaging.

Determine where the defect originates

- **Wrong content** (missing steps, wrong links, unresolved keyrefs) # preprocessing issue
- **Correct content, wrong markup** # XSLT transform issue
- **Correct HTML, missing CSS/JS/images** # packaging/copy issue

If you suspect preprocess2 vs preprocess mismatch

Check whether the transtype uses map-first preprocessing:

- HTML5 uses map-first preprocessing as of 4.2
- PDF/HTML Help use it since 3.0

If your extension points aren't firing, it may be because step-specific hooks aren't available in [preprocess2](#).

Note

The official docs explicitly warn that the internal order of preprocessing steps can change between versions. If you're hooking inside preprocess, assume you must maintain it across OT upgrades.

Common preprocessing symptoms

Symptom	Likely cause
Missing or incorrect links	keyref/href resolution or scope
Duplicate IDs or broken xrefs	conref reuse or ID-uniqueness logic
Missing chunks/pages	map structure, <code>@chunk</code> , or chunk decisions
Content that "should be excluded" still appears	filtering config not applied early enough
Titles or nav labels wrong	map processing and/or TOC generation

Reproduce in Docker to isolate environment issues

If a build fails only on CI or only on a coworker's machine, run the same build using a pinned Docker image:

```
docker run --rm -v $(pwd):/src ghcr.io/dita-ot/dita-ot:4.3.5 \
-i /src/dita/maps/book.ditamap -f html5 -o /src/out/html5
```

This eliminates OS-level differences, Java version mismatches, and locally installed plugin conflicts.

Use strict processing mode for quality gates

If your builds silently succeed with missing references, you ship broken docs. Add strict mode to CI:

```
dita -i map.ditamap -f html5 -o out/html5 --processing-mode=strict
```

Repository Layout and Build Variants

Recommended real-world repository structure for DITA projects and how to use DITAVAL filtering to produce audience/platform build variants.

Recommended repository layout

A layout that scales to real teams:

```
repo/
  dita/
    maps/
    topics/
  plugins/
    com.yourco.html5.custom/
      plugin.xml
    xsl/
```

```
resources/  
build/  
  ditaval/  
    *.properties  
.github/  
  workflows/  
out/ (gitignored)
```

This supports:

- versioned build settings
- filter variants
- portable plug-ins
- CI pipelines

Build variants with DITAVAL filtering

DITAVAL files define conditional processing rules for attributes like `audience`, `platform`, or custom props.

Example `build/ditaval/web.ditaval`:

```
<val>  
  <prop action="exclude" att="platform" val="print"/>  
</val>
```

Build:

```
dita -i dita/maps/book.ditamap -f html5 -o out/html5-web --args.filter=build/ditaval/  
web.ditaval
```

Store common settings in `build/html5.properties` and override only filter + output per variant.

Key References

Authoritative DITA-OT documentation sources referenced throughout this handbook.

Official documentation sources

- Processing structure: temp dir, stages, intermediate artifacts
- Pre-processing modules: preprocess as Ant targets
- Pre-processing extension points: recommended pre/post hooks + cautions
- XSLT-import extension points: installer adds imports into default code
- Map-first preprocessing: preprocess2 model, HTML5 default as of 4.2, limitations
- Docker usage and creating images: run builds and install plugins in images
- GitHub Actions guidance and dita-ot-action repo README
- Extension point reference and plug-in descriptor file docs

Part

VIII

Advanced XSLT Customization

Topics:

- [XSLT Customization Tutorial](#)
- [Concepts & Foundations](#)
- [Plugin Development](#)
- [Operations & Reference](#)
- [Technical Reference: XSLT Patterns and Glossary](#)

XSLT Customization Tutorial

This tutorial teaches a practical, production-style way to customize HTML output from DITA using XSLT, primarily through DITA Open Toolkit DITA-OT 4.x plug-ins and official extension points.

Core goals covered:

- **Sidebar TOC** embedded in every topic page (`nav-toc=full`)
- **Chunking** (map-driven split into multiple HTML pages)
- **Filtering and build variants** using **DITaval** (examples for `audience` and `platform`)
- Safe, upgrade-friendly **XSLT override patterns** that work across **all topic types** and common elements

Note

Assumption: You publish with **DITA-OT HTML5** (`-f html5`). If you use a CCMS, the same patterns apply; packaging and invocation may differ.

Analytics and Instrumentation

Track which customization steps are most challenging for users. Use analytics to improve the tutorial and add clarifications where needed.

Personalization and Conditional Processing

Show or hide advanced steps based on user role using DITA conditional processing.

Integrating with Conversational AI and NLP

Enable users to ask a chatbot for help with XSLT customization and receive step-by-step guidance from this tutorial.

Continuous Improvement and Feedback Loops

Collect feedback on tutorial clarity and pain points. Use analytics and user suggestions to refine the tutorial.

Related reference

[Technical Reference: XSLT Patterns and Glossary](#) on page 166

Most real DITA-OT customization work is XSLT. These patterns minimize upgrade pain and maximize maintainability.

[Complete Plugin Reference](#) on page 202

Full source of the v2 starter plugin (`com.acme.html5.sidebar`) demonstrating production-ready DITA-OT HTML5 customization.

Concepts & Foundations

What you're customizing (HTML5 transform architecture)

DITA-OT publishing is a pipeline. Understanding its three phases — preprocessing, XSLT transformation, and post-processing — is essential before writing any override.

In DITA-OT 4.x, HTML5 output involves:

Preprocessing

Typical preprocessing steps include:

- map parsing (reading the map + topicrefs)
- key collection (keydefs # key space)
- conref/keyref resolution (pulling referenced content)
- filtering / conditional processing (DITAVAL / props / audience/platform/product)
- chunking decisions (map-driven chunk strategy)
- copying/transient file preparation in the temp folder

Why this matters: Many "mysterious" HTML issues are not XSLT bugs; they come from preprocessing (e.g., conref not resolved, chunking not applied, keys not in scope).

Transformation to HTML (XSLT)

After preprocessing, the HTML5 pipeline runs XSLT to generate:

- **topic pages** (chunked or not)
- **navigation artifacts** such as TOC pages
- HTML element mappings for DITA structures (p, section, note, steps, tables, xref, etc.)

Your XSLT customizations live here. This is where you:

- modify HTML structure
- add CSS classes and `data-*` attributes
- change output for specific DITA elements
- inject JS/CSS references
- create wrappers for responsive layouts

Post-processing

After HTML is generated, DITA-OT typically:

- copies CSS and other assets
- writes support files (images, JS if configured/added)
- finalizes output folder structure

Why this matters: Your output may look wrong simply because your CSS/JS wasn't copied into output.

Extension points: the supported integration points

DITA-OT exposes official extension points that let you plug into the pipeline without editing core files.

The most common for HTML5:

- `dita.xsl.html5` — override or extend the main HTML5 XSLT step
- `dita.xsl.html5.toc` — override or extend TOC generation XSLT
- `dita.conductor.html5.param` and `dita.conductor.html5.toc.param` — pass parameters into the main HTML5 and TOC steps
- general plug-in XSLT import extension points (import-based override mechanism)

The key idea: create a plug-in that **imports the default** HTML5 XSLT and overrides only the templates you need.

Two safe strategies for HTML5 customization

There are two reliable approaches to customizing DITA-OT HTML5 output: a plug-in with XSLT import overrides (recommended) or build-time custom XSL via `args.xsl`.

Strategy A (recommended): DITA-OT plug-in + XSLT import overrides

You create a plug-in that:

- depends on the HTML5 plug-in (`org.dita.html5`)
- provides an override XSL file via `dita.xsl.html5` and/or `dita.xsl.html5.toc`
- imports the default transform and overrides specific templates

Why it's best long-term:

- clean separation (your code is isolated)
- controlled load order (dependencies)
- easier upgrades (you don't edit toolkit core)

Strategy B: Build-time custom XSL via `args.xsl`

You supply a single XSL file at build time (parameter-driven). Good for:

- experiments
- temporary branches
- customer-specific builds that should not live in your core plugin

Risk: unmanaged sprawl (multiple XSLs floating around, unclear precedence, harder upgrades).

Rule of thumb:

- customization is part of your product pipeline # **plug-in**
- one-off/experimental # **`args.xsl`**

Prerequisites: what you need to know before overriding

Before writing XSLT overrides you need fluency in template matching, modes, import precedence, and how DITA's `@class` attribute drives element processing.

XSLT basics you must understand

You should be comfortable with:

- template matching (`match=`)
- modes (`mode=`)
- named templates (`name=`)
- `xsl:import` vs `xsl:include` (import precedence matters)
- how to build/copy attributes and preserve content
- how to layer changes safely:
 - `xsl:next-match` (often the best)
 - `xsl:apply-imports` (useful in some stylesheets)
 - overriding named templates (when the default is structured that way)

DITA basics that affect HTML transforms

DITA content is specialized. The `@class` attribute encodes ancestry such as:

- `topic/p` (paragraph)
- `topic/note` (note)
- `task/step` (task step)

DITA-OT templates often match using ancestry patterns:

```
match="*[contains(@class, ' topic/p ')]"
```

Why it matters: matching by `@class` supports both base elements and specializations that inherit those semantics.

DITA-OT 4.x setup: sidebar TOC + chunking baseline

Before writing XSLT, establish a baseline build that gives you chunked pages, a sidebar TOC on every page, and repeatable builds via properties files.

Sidebar TOC embedded in every page: `nav-toc=full`

In DITA-OT HTML5 builds, `nav-toc=full` embeds a TOC `<nav>` in each topic page:

```
dita -i path/to/map.ditamap -f html5 -o out/html5 --nav-toc=full
```

Repeatable builds: use a `.properties` file

Create `build/html5-sidebar.properties`:

```
# Sidebar TOC embedded in every topic page
nav-toc=full

# Generate an index TOC file
html5.toc.generate=yes

# Include DITA @class in output to aid debugging and CSS hooks
args.html5.classattr=yes

# Brand CSS
args.copycss=yes
args.cssroot=css/
args.css=brand.css
```

Build with:

```
dita -i path/to/map.ditamap -f html5 -o out/html5 --propertyfile=build/html5-
sidebar.properties
```

Chunking: map-driven, scalable

Chunking is usually controlled in the map using `@chunk`. A common pattern:

```
<topicref href="topics/intro.dita" chunk="to-content">
  <topicref href="topics/install.dita"/>
  <topicref href="topics/configure.dita"/>
</topicref>
```

That yields separate HTML pages per child topic, with a TOC that links to those pages.

Filtering with DITAVAL and build variants (audience/platform)

DITA supports conditional processing using attributes such as audience and platform. DITA-OT applies filtering using a DITAVAL file referenced by args.filter.

Example DITAVAL files

Exclude print-only content for a WEB build:

```
<?xml version="1.0" encoding="UTF-8"?>
<val>
  <prop action="exclude" att="platform" val="print"/>
</val>
```

Exclude novice-only content for an ADMIN build:

```
<?xml version="1.0" encoding="UTF-8"?>
<val>
  <prop action="exclude" att="audience" val="novice"/>
</val>
```

Using filters in builds

```
dita -i map.ditamap -f html5 -o out/html5-web --args.filter=build/ditaval/web.ditaval
dita -i map.ditamap -f html5 -o out/html5-print --args.filter=build/ditaval/print.ditaval

dita -i map.ditamap -f html5 -o out/html5-admin --args.filter=build/ditaval/audience-
admin.ditaval
dita -i map.ditamap -f html5 -o out/html5-novice --args.filter=build/ditaval/audience-
novice.ditaval
```

Best practice: properties files per variant

Store stable settings in properties files:

- `build/html5-sidebar.properties` (base)
- `build/html5-sidebar-web.properties` (base + `args.filter=...`)
- `build/html5-sidebar-print.properties` (base + `args.filter=...`)
- `build/html5-sidebar-admin.properties` (base + `args.filter=...`)

- `build/html5-sidebar-novice.properties` (base + `args.filter=...`)

This prevents "mystery builds" and makes CI reproducible.

Common variant patterns you'll use in real projects

- **Platform split:** web vs print output styles, image sizes, navigation density
- **Audience split:** admin vs end-user content, warnings, extra setup steps
- **Feature tiers:** community vs enterprise (often driven by `product` or custom props)
- **Region/language:** often via map roots + language settings; DITAVAL can help when content is shared

TOC grouping with `topichead`/`topicgroup` (non-clickable headings)

Use `topichead` and `topicgroup` elements to create non-clickable navigation headings that organize the sidebar TOC without requiring landing pages.

For clean navigation headings that are **non-clickable**, use:

- `<topichead navtitle="..."> ... </topichead>` for headings
- `<topicgroup navtitle="..."> ... </topicgroup>` for grouping containers

Example:

```
<map>
  <title>Docs</title>

  <topichead navtitle="Getting Started">
    <topicref href="topics/intro.dita"/>
    <topicref href="topics/install.dita"/>
  </topichead>

  <topicgroup navtitle="Reference">
    <topicref href="topics/cli.dita"/>
    <topicref href="topics/api.dita"/>
  </topicgroup>
</map>
```

This keeps the sidebar TOC tidy without forcing landing pages for every group.

Plugin Development

Create a minimal HTML5 customization plug-in

Build a minimal DITA-OT plug-in that depends on `org.dita.html5` and overrides the HTML5 XSLT via the `dita.xsl.html5` extension point.

Folder structure (recommended layout)

Inside your DITA-OT installation:

```
dita-ot/
  plugins/
```

```
com.acme.html5.custom/
  plugin.xml
  xsl/
    html5/
      override.xsl
```

1. Create `plugin.xml` with HTML5 XSL override and dependency:

```
<?xml version="1.0" encoding="UTF-8"?>
<plugin id="com.acme.html5.custom" version="1.0.0">
  <require plugin="org.dita.html5"/>
  <feature extension="dita.xsl.html5" file="xsl/html5/override.xsl"/>
</plugin>
```

2. Install and integrate the plug-in from the DITA-OT directory:

```
dita --install
```

3. Re-run `dita --install` whenever you edit plug-in files that affect catalogs or import maps.

Write your first override XSLT correctly

Your override stylesheet imports the default HTML5 transform and overrides only the templates you need, using `xsl:next-match` for safe layering.

Import the default HTML5 implementation

Your override stylesheet should start by importing the default HTML5 transform:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  version="3.0">

  <xsl:import href="plugin:org.dita.html5:xsl/dita2html5.xsl"/>

  <!-- overrides below -->

</xsl:stylesheet>
```

Precedence rules (do not skip)

- templates in your override file win over imported templates
- within the same precedence level, more specific matches win
- import order matters if you have multiple imports

Practical implication: keep overrides in a single file (or layered imports you control) and avoid rewriting entire default templates unless necessary.

Prefer `xsl:next-match` for safe layering

When overriding a template but wanting default behavior, wrap with `xsl:next-match`:

```
<xsl:template match="*[contains(@class, ' topic/table ')]">
  <div class="table-wrap">
    <xsl:next-match/>
  </div>
</xsl:template>
```

This is safer than copy-pasting large default templates.

Production patterns for all topic types + common elements

A core set of XSLT override templates that scale across all DITA topic types and common elements using @class-based matching and `xsl:next-match` wrappers.

Scaling approach

To scale to "all topic types + common elements" without creating a fragile customization layer:

1. match topic roots using `topic/topic` ancestry
2. match common elements using `@class` ancestry
3. wrap default behavior with `xsl:next-match` wherever possible
4. apply consistent classes and `data-*` attributes for styling and instrumentation

Standardize the topic wrapper (applies to all topic types)

```
<xsl:template match="*[contains(@class,' topic/topic ')]">
  <article class="dita-topic" data-topic-type="{local-name()}">
    <xsl:apply-templates select="*[contains(@class,' topic/title ')]"/>
    <xsl:apply-templates select="*[contains(@class,' topic/shortdesc ')]"/>

    <main class="dita-body">
      <xsl:apply-templates
        select="*[contains(@class,' topic/body ')]
              |[contains(@class,' concept/conbody ')]
              |[contains(@class,' task/taskbody ')]
              |[contains(@class,' reference/refbody ')]"/>
    </main>

    <xsl:apply-templates select="*[contains(@class,' topic/related-links ')]"/>
  </article>
</xsl:template>
```

Example patterns (core set)

Paragraph hooks:

```
<xsl:template match="*[contains(@class,' topic/p ')]">
  <p class="dita-p" data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </p>
</xsl:template>
```

Notes as callouts:

```
<xsl:template match="*[contains(@class,' topic/note ')]">
  <aside class="callout{if (@type) then concat(' callout-', @type) else ''}">
    <xsl:if test="@type">
      <div class="callout-title"><xsl:value-of select="upper-case(@type)"/></div>
    </xsl:if>
    <div class="callout-body"><xsl:apply-templates/></div>
```

```
</aside>
</xsl:template>
```

Task steps (anchors + step index):

```
<xsl:template match="*[contains(@class, ' task/step ')]">
  <li class="task-step"
      data-step="{count(preceding-sibling::*[contains(@class, ' task/step ')] + 1}">
    <xsl:if test="@id">
      <xsl:attribute name="id" select="@id"/>
    </xsl:if>
    <xsl:apply-templates/>
  </li>
</xsl:template>
```

Tables (responsive wrappers using next-match):

```
<xsl:template match="*[contains(@class, ' topic/table ')]">
  <div class="table-wrap"><xsl:next-match/></div>
</xsl:template>

<xsl:template match="*[contains(@class, ' topic/simpletable ')]">
  <div class="simpletable-wrap"><xsl:next-match/></div>
</xsl:template>
```

Extended "common element" XSLT overrides (hooks + data-attributes)

Override templates for *ph*, *xref*, *fig*, *image*, *li*, *dl/dt/dd*, *draft-comment*, and *hazardstatement* — adding consistent CSS hooks and data-attributes while preserving default behavior.

ph — stable phrasing hook

```
<xsl:template match="*[contains(@class, ' topic/ph ')]">
  <span class="dita-ph{if (@outputclass) then concat(' ', @outputclass) else ''}"
      data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </span>
</xsl:template>
```

xref — preserve default link generation, then add hooks

This preserves DITA-OT link resolution while adding consistent classes and `data-*` attributes.

```
<xsl:template match="*[contains(@class, ' topic/xref ')]">
  <xsl:variable name="out" as="node(*)">
    <xsl:next-match/>
  </xsl:variable>

  <xsl:for-each select="$out">
    <xsl:choose>
      <xsl:when test="self::a">
        <xsl:variable name="isExternal"
            select="starts-with(@href, 'http') or @target='_blank'"/>
```

```

<a>
  <xsl:copy-of select="@*" />

  <xsl:if test="$isExternal">
    <xsl:attribute name="target">_blank</xsl:attribute>
    <xsl:attribute name="rel">noopener noreferrer</xsl:attribute>
  </xsl:if>

  <xsl:attribute name="class"
    select="normalize-space(string-join((@class, 'dita-xref', if ($isExternal)
then 'external' else ()), ' '))" />
  <xsl:attribute name="data-dita-class"
    select="normalize-space(../@class)" />

  <xsl:apply-templates select="node()" />
</a>
</xsl:when>
<xsl:otherwise>
  <xsl:sequence select="." />
</xsl:otherwise>
</xsl:choose>
</xsl:for-each>
</xsl:template>

```

Figures and images

```

<xsl:template match="*[contains(@class, ' topic/fig ')]">
  <figure class="dita-fig" data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </figure>
</xsl:template>

<xsl:template match="*[contains(@class, ' topic/fig/title ')]">
  <figcaption class="dita-figcaption"><xsl:apply-templates/></figcaption>
</xsl:template>

```

Conservative wrapper for images (lets default logic compute `src/alt`):

```

<xsl:template match="*[contains(@class, ' topic/image ')]">
  <span class="dita-image-wrap" data-dita-class="{normalize-space(@class)}">
    <xsl:next-match/>
  </span>
</xsl:template>

```

List items

```

<xsl:template match="*[contains(@class, ' topic/li ')]">
  <li class="dita-li" data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </li>
</xsl:template>

```

Definition lists (`dl`, `dt`, `dd`)

```

<xsl:template match="*[contains(@class, ' topic/dl ')]">
  <dl class="dita-dl" data-dita-class="{normalize-space(@class)}">

```

```

        <xsl:apply-templates/>
      </dl>
    </xsl:template>

    <xsl:template match="*[contains(@class, ' topic/dt ')]">
      <dt class="dita-dt" data-dita-class="{normalize-space(@class)}">
        <xsl:apply-templates/>
      </dt>
    </xsl:template>

    <xsl:template match="*[contains(@class, ' topic/dd ')]">
      <dd class="dita-dd" data-dita-class="{normalize-space(@class)}">
        <xsl:apply-templates/>
      </dd>
    </xsl:template>

```

Draft comments

```

<xsl:template match="*[contains(@class, ' topic/draft-comment ')]">
  <aside class="draft-comment" data-dita-class="{normalize-space(@class)}" aria-
label="Draft comment">
    <div class="draft-comment-label">DRAFT</div>
    <div class="draft-comment-body"><xsl:apply-templates/></div>
  </aside>
</xsl:template>

```

Hazard statements (hazard domain)

```

<xsl:template match="*[contains(@class, ' hazard-d/hazardstatement ')]">
  <section class="hazardstatement" data-dita-class="{normalize-space(@class)}" aria-
label="Hazard statement">
    <xsl:apply-templates/>
  </section>
</xsl:template>

<xsl:template match="*[contains(@class, ' hazard-d/hazardtype ')]">
  <div class="hazardtype"><xsl:apply-templates/></div>
</xsl:template>

<xsl:template match="*[contains(@class, ' hazard-d/consequence ')]">
  <div class="hazard-consequence"><strong>Consequence:</strong> <xsl:apply-templates/></div>
</xsl:template>

<xsl:template match="*[contains(@class, ' hazard-d/howtoavoid ')]">
  <div class="hazard-howtoavoid"><strong>How to avoid:</strong> <xsl:apply-templates/></div>
</xsl:template>

```

Sidebar TOC customization

Customize the sidebar TOC at two levels: CSS-only styling of the default nav element, or XSLT markup overrides via the `dita.xsl.html5.toc` extension point.

Level 1: CSS-only sidebar TOC

If you just need layout/styling:

- set `nav-toc=full`
- style the `<nav>` as a sidebar in CSS

Level 2: XSLT markup customization

Override TOC generation via `dita.xsl.html5.toc` and import the default TOC stylesheet.

Example `plugin.xml`:

```
<feature extension="dita.xsl.html5.toc" file="xsl/toc/toc-override.xsl"/>
```

Example `toc-override.xsl`:

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  version="3.0">
  <xsl:import href="plugin:org.dita.html5:xsl/toc.xsl"/>

  <xsl:template match="nav">
    <nav class="sidebar-toc">
      <xsl:apply-templates select="@* | node()"/>
    </nav>
  </xsl:template>

  <xsl:template match="li">
    <li data-level="{count(ancestor::li)+1}">
      <xsl:apply-templates select="@* | node()"/>
    </li>
  </xsl:template>
</xsl:stylesheet>
```

Passing parameters cleanly

Three ways to pass parameters to DITA-OT builds: command-line flags, properties files (recommended for teams), and plug-in parameter injection for advanced setups.

Command line parameters (quick)

```
dita -i map.ditamap -f html5 -o out/html --nav-toc=full --args.copycss=yes --
args.cssroot=css/ --args.css=brand.css
```

Properties file (recommended for teams)

Put stable settings in a properties file and check it into source control.

Plug-in parameter injection (advanced)

For complex plug-ins, define parameter bindings via conductor param extension points (main HTML5 step and TOC step).

Operations & Reference

Debugging workflow (iterate fast, safely)

A systematic debugging workflow: inspect the temp directory, locate the template you need to override, keep overrides minimal, and use strict processing mode to fail fast.

Inspect the temp directory

Inspect preprocessed files to confirm:

- filtering applied as expected
 - conrefs and keys resolved
 - chunking decisions applied
1. Find the template you need to override:
 - a) Locate HTML5 XSL files in the toolkit (or use `plugin:` URIs).
 - b) Search for the `@class` fragment (e.g., `topic/note`, `task/step`).
 - c) Copy only what you need into your override file.
 - d) Change it minimally.
 2. Keep overrides minimal. Prefer:
 - wrappers + `xsl:next-match`
 - small, well-commented templates
 - one purpose per template
 3. Fail fast for broken content — run in strict processing mode:

```
dita -i map.ditamap -f html5 -o out/html5 --processing-mode=strict
```

Hardening for production (CI, quality gates, upgrade plan)

Pin your toolkit version, set up CI pipelines for all build variants, and follow a deliberate upgrade strategy when moving to new DITA-OT releases.

Version pinning

Pin a toolkit version per release line and upgrade deliberately.

CI pipeline (minimum)

- install plug-ins (`dita --install`)
- run builds (all variants)
- run in strict mode where appropriate
- publish build artifacts for review

Upgrade strategy

When upgrading DITA-OT:

1. run builds with no customization changes

2. compare outputs
3. adjust imports (TOC imports are common)
4. minimize template rewrites

Maintain a [CHANGELOG.md](#) in your plug-in repo.

Exercises and labs (hands-on)

Five progressive labs that build on each other: callout note styling, step anchors with highlighting, custom meta injection, collapsible sidebar TOC, and full plug-in packaging with CI.

Before starting these labs, set up the exercise environment:

1. Navigate to the [exercises/](#) folder in this training repository (see [exercises/README.md](#) for full details).
2. Build the baseline output (no plug-in) to see default DITA-OT rendering:

```
dita -i exercises/lab-sample.ditamap -f html5 -o exercises/out/html5 --nav-toc=full
```

3. Copy the starter plug-in into your DITA-OT install:

```
cp -r exercises/plugins/com.extense.html5.labs DITA-OT/plugins/
cd DITA-OT && dita --install
```

Sample files included:

- [exercises/lab-sample.ditamap](#) — sample map with 5 topics
- [exercises/topics/what-is-acmedoc.dita](#) — concept with all 4 note types (Lab 1)
- [exercises/topics/install-acmedoc.dita](#) — task with 5 steps and IDs (Lab 2)
- [exercises/plugins/com.extense.html5.labs/](#) — starter plug-in skeleton with commented templates
- [exercises/build/html5.properties](#) — build properties for repeatable builds
- [exercises/build/ditaval/web.ditaval](#) — DITAVAL filter sample
- [exercises/build/sample-github-workflow.yml](#) — CI workflow reference (Lab 5)

Each lab builds on the previous one. Complete them in order to assemble a full production plug-in. All XSLT templates and CSS in the starter files are commented out — uncomment each section as you reach that lab.

• Lab 1: Callout note styling

Goal: Style `<note>` elements as colored callout boxes with icons.

Steps:

1. Open [exercises/plugins/com.extense.html5.labs/xsl/html5/override.xsl](#) and uncomment the Lab 1 section — the template matching `*[contains(@class, ' topic/note ')]`:

```
<xsl:template match="*[contains(@class, ' topic/note ')]">
  <div class="callout callout-{@type}" data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </div>
</xsl:template>
```

2. Open `exercises/plugins/com.extense.html5.labs/css/brand.css` and uncomment the Lab 1 callout styles (`.callout-note`, `.callout-warning`, `.callout-caution`, `.callout-tip`).
3. Install the plug-in and build the sample map:

```
dita --install
dita -i exercises/lab-sample.ditamap -f html5 -o exercises/out/html5 --nav-toc=full
```

4. Open `exercises/out/html5/what-is-acmedoc.html` — this topic contains all 4 note types.

Expected result: Notes appear as styled boxes with color-coded left borders instead of plain paragraphs.

- **Lab 2: Step anchors with URL hash highlighting**

Goal: Add `id` anchors to task steps so users can link directly to a step, with JS highlight on load.

Steps:

1. In `exercises/plugins/com.extense.html5.labs/xsl/html5/override.xsl`, uncomment the Lab 2 section — the template for `task/step`:

```
<xsl:template match="*[contains(@class,' task/step ')]">
  <li class="task-step" id="step-{position()}" data-step="{position()}">
    <xsl:apply-templates/>
  </li>
</xsl:template>
```

2. In `exercises/plugins/com.extense.html5.labs/js/step-highlight.js`, uncomment the entire script. It detects `#step-N` URL hashes, scrolls to the matching element, and adds a `.highlighted` CSS class.
3. In `exercises/plugins/com.extense.html5.labs/css/brand.css`, uncomment the Lab 2 step-highlight styles (`.highlighted` with yellow background and transition).
4. Rebuild and open `exercises/out/html5/install-acmedoc.html#step-3` — this task has 5 steps, each with unique IDs like `step-download`, `step-extract`, etc.

Expected result: Navigating to `page.html#step-3` scrolls to and highlights step 3.

- **Lab 3: Custom meta tag injection**

Goal: Add a `<meta name="generator" content="DITA-OT + com.extense.html5.labs">` tag to every page.

Steps:

1. In the DITA-OT source, locate the named template `gen-user-head` in `plugins/org.dita.html5/xsl/topic.xsl` — this empty template is called from `chapterHead` specifically for user customizations. Override it to inject meta tags.
2. Uncomment the Lab 3 section in `exercises/plugins/com.extense.html5.labs/xsl/html5/override.xsl`. The template overrides the empty `gen-user-head` and adds your custom meta tags directly — no `xsl:next-match` needed since the default template is empty:

```
<xsl:template name="gen-user-head">
  <meta name="generator" content="DITA-OT + com.extense.html5.labs"/>
  <meta name="dita-class" content="{normalize-space(*/@class)}"/>
</xsl:template>
```

3. Rebuild:

```
dita --install
dita -i exercises/lab-sample.ditamap -f html5 -o exercises/out/html5 --nav-toc=full
```

4. Open any output page and view source — confirm both custom meta tags appear in the `<head>`.

Expected result: Every HTML page contains your custom meta tags in the `<head>`.

• Lab 4: Collapsible sidebar TOC

Goal: Override the TOC to add CSS classes and make it a collapsible sidebar.

Steps:

1. Open `exercises/plugins/com.extense.html5.labs/xsl/toc/toc-override.xsl` and uncomment the template. It wraps the TOC in a `<nav class="sidebar-toc">` with `data-level` attributes on each `<li>` for nested styling.
2. In `exercises/plugins/com.extense.html5.labs/css/brand.css`, uncomment the Lab 4 sidebar TOC styles:

```
.sidebar-toc { position: sticky; top: 0; max-height: 100vh; overflow-y: auto; }
.sidebar-toc li[data-level="2"] { padding-left: 1rem; }
.sidebar-toc li[data-level="3"] { padding-left: 2rem; }
```

3. In `exercises/plugins/com.extense.html5.labs/js/sidebar-toggle.js`, uncomment the toggle script. It adds a hamburger button on viewports narrower than 768px.
4. Rebuild the sample map (which has 3 levels of nesting) and verify the sidebar renders correctly:

```
dita --install
dita -i exercises/lab-sample.ditamap -f html5 -o exercises/out/html5 --nav-toc=full
```

Expected result: A sticky, scrollable sidebar TOC with indented levels and a collapse toggle.

• Lab 5: Full plug-in packaging and CI integration

Goal: Package all previous labs into a single plug-in, pin the toolkit version, and automate builds.

Steps:

1. Verify the complete plug-in structure in `exercises/plugins/com.extense.html5.labs/`:

```
com.extense.html5.labs/
  plugin.xml
  xsl/html5/override.xsl
  xsl/toc/toc-override.xsl
  css/brand.css
  js/step-highlight.js
  js/sidebar-toggle.js
```

2. Open `exercises/plugins/com.extense.html5.labs/plugin.xml` — confirm it declares both extension points (`dita.xsl.html5` and `dita.xsl.html5.toc`).
3. Ensure all Lab 1–4 code sections are uncommented, then install and build:

```
dita --install
dita -i exercises/lab-sample.ditamap -f html5 -o exercises/out/html5 \
  --args.cssroot=css --args.css=brand.css --args.copycss=yes --nav-toc=full
```

4. Review the CI reference at [exercises/build/sample-github-workflow.yml](#). Adapt it for your own repository (update paths, DITA-OT version pin, and output directory).
5. Test the upgrade path: install a newer DITA-OT version, rebuild, and diff the outputs.

Expected result: A portable, version-pinned plug-in that builds in CI and produces consistent output across upgrades.

Verification checklist — confirm each after completing the corresponding lab:

1. **Lab 1:** Open [what-is-acmedoc.html](#) — all 4 note types (note, tip, warning, caution) render as colored callout boxes with distinct left borders.
2. **Lab 2:** Open [install-acmedoc.html#step-3](#) — the page scrolls to step 3 and highlights it with a yellow background.
3. **Lab 3:** View source on any output page — `<meta name="generator">` and `<meta name="dita-class">` appear in the `<head>`.
4. **Lab 4:** Open any page — a sticky sidebar TOC with indented levels appears. On mobile widths (below 768px), a toggle button shows/hides it.
5. **Lab 5:** Push to a GitHub repo with the sample workflow — the CI build succeeds and publishes to [gh-pages](#).

Troubleshooting

Common issues and solutions for sidebar TOC, chunking, XSL overrides, external links, and post-upgrade breakages.

Sidebar TOC is missing

- confirm `nav-toc=full` is set
- confirm you're building a map (not a single topic)
- confirm CSS is copied to output

Chunking did not produce separate pages

- confirm `@chunk` is set on the correct `topicref`
- inspect preprocessed files in temp directory

XSL overrides do not show up

- confirm your plug-in is installed in the correct DITA-OT instance
- run `dita --install`
- verify `plugin.xml` has `require plugin="org.dita.html5"`
- verify extension points are correct (`dita.xsl.html5`, `dita.xsl.html5.toc`)

External links template doesn't fire

- the final URI may not be stored in `@href`
- locate the default link/href computation logic and override there

Everything broke after upgrade

- re-check import paths (especially TOC)
- reduce overrides to minimal set
- prefer wrappers + next-match rather than large copied templates

Appendix: starter repo integration guide

How to integrate your customization plug-in with the starter repo — copy/symlink, install, build, and where to edit each type of file.

If you are using the starter repo, you typically have:

- sample map with chunking
- sample topics (concept/task/reference)
- plug-in folder under `plugins/`
- CSS + JS assets
- build scripts and properties variants

1. Copy or symlink your plug-in into your DITA-OT install:

```
plugins/<your.plugin.id> # DITA-OT/plugins/
```

2. Integrate:

```
dita --install
```

3. Build:

```
dita -i dita/maps/guides/getting-started.ditamap -f html5 -o out/html5 \
  --propertyfile=build/html5-sidebar.properties
```

4. Open and verify:

- `out/html5/index.html`
- any topic page to see embedded TOC sidebar

Where to edit:

- HTML structure and element hooks: `plugins/<id>/xsl/html5/override.xsl`
- TOC markup: `plugins/<id>/xsl/toc/toc-override.xsl`
- Layout + styling: `css/brand.css`
- Interactivity: `js/site.js`

Appendix: quick checklists

Three checklists for safe overrides, sidebar TOC + chunking, and release-ready builds — use these before every deployment.

Safe override checklist

- Use `@class`-based matches (`contains(@class, ' topic/p ')`)
- Import the default HTML5 XSL via `xsl:import`
- Keep overrides small; use `xsl:next-match` where possible
- Avoid copying large default templates unless absolutely necessary
- Comment intent and expected output per override

Sidebar TOC + chunking checklist

- `nav-toc=full` set
- map has chunking rules (`chunk="to-content"` etc.)
- CSS copied to output
- `<nav>` styled as sidebar (sticky, scrollable)

Release-ready build checklist

- Pin DITA-OT version
- CI builds run `dita --install` and publish output
- strict processing mode enabled for main builds
- plug-in has a changelog and version tag
- upgrade tests performed before changing OT versions

Technical Reference: XSLT Patterns and Glossary

Most real DITA-OT customization work is XSLT. These patterns minimize upgrade pain and maximize maintainability.

Note

This chapter provides a concise technical reference for XSLT customization patterns and developer terminology. For full coverage of source development, debugging, and CI/CD, see the Developer Handbook.

Writing robust XSLT customizations (import layers + next-match)

Most real DITA-OT customization work is XSLT. These patterns minimize upgrade pain and maximize maintainability.

Pattern 1: Import the default transform

Always start by importing the shipped XSL for your target output:

```
<xsl:import href="plugin:org.dita.html5:xsl/dita2html5.xsl"/>
```

This gives you all default behavior, and you only override what you need.

Pattern 2: Override templates by @class ancestry

Use DITA class attributes to match across specializations:

```
<xsl:template match="*[contains(@class, ' topic/p ')]">
  <p data-dita-class="{normalize-space(@class)}">
    <xsl:apply-templates/>
  </p>
</xsl:template>
```

This matches `<p>` elements and any specialized descendants of `topic/p`.

Pattern 3: Prefer wrapper overrides with xsl:next-match

Keep default logic but add wrappers, classes, or data attributes:

```
<xsl:template match="*[contains(@class, ' topic/section ')]">
  <div class="custom-section-wrapper">
    <xsl:next-match/>
  </div>
</xsl:template>
```

Pattern 4: Add consistent hooks

Add classes and `data-*` attributes for CSS/JS targeting:

- `data-dita-class` — enables styling by DITA element type
- `data-step` — enables deep-linking to task steps
- `aria-label` — improves accessibility

Quick glossary (developer-focused)

- **Transtype**: the transform type, e.g., `html5`, `pdf`
- **Preprocess / preprocess2**: legacy vs map-first preprocessing pipelines
- **Plug-in**: extension package that can inject XSLT, Ant targets, resources
- **Extension point**: named hook where a plug-in can contribute behavior
- **Intermediate files**: normalized/filtered content used by later stages
- **DITaval**: XML file defining conditional processing (include/exclude) rules
- **Chunking**: how DITA-OT decides to split or combine topics into output pages/files
- **@class attribute**: inheritance chain used by XSLT to match specialized elements

Related tasks

[XSLT Customization Tutorial](#) on page 148

This tutorial teaches a practical, production-style way to customize HTML output from DITA using XSLT, primarily through DITA Open Toolkit DITA-OT 4.x plug-ins and official extension points.

Part

IX

AI Pipeline & Chatbot Integration

Topics:

- [RAG Pipeline Architecture: From DITA to Chatbot Answers](#)
- [Section-Aware Chunking and Embedding Strategy](#)
- [Retrieval, Reranking, and Answer Generation](#)

RAG Pipeline Architecture: From DITA to Chatbot Answers

How a Retrieval-Augmented Generation pipeline transforms structured DITA content into accurate, cited chatbot responses — architecture overview and component roles.

What is Retrieval-Augmented Generation?

RAG is an AI architecture that combines **information retrieval** (searching a knowledge base) with **text generation** (composing a natural-language answer). Instead of relying solely on the LLM's training data, RAG grounds every answer in your actual documentation.

The pipeline has four stages:

1. **Ingest:** DITA content is chunked, enriched with metadata, and embedded into vector space
2. **Retrieve:** User queries are matched against the vector store to find the most relevant chunks
3. **Rerank:** A cross-encoder model reorders the retrieved chunks by semantic relevance
4. **Generate:** The LLM composes an answer using only the top-ranked chunks as context

Why is DITA the ideal source format for RAG?

DITA provides three things that unstructured content cannot:

Capability	DITA provides	RAG impact
Clean boundaries	Section elements with IDs provide natural chunk boundaries	Chunks are semantically coherent, not arbitrary text splits
Metadata per chunk	Prolog metadata is inherited by every chunk from that topic	Retrieval can filter by audience, difficulty, domain, intent
Type semantics	Concept vs task vs reference typing identifies content purpose	Intent routing: "How do I..." # task chunks; "What is..." # concept chunks
Hierarchy context	Map-defined nesting shows parent-child topic relationships	Chatbot can suggest "See also: parent topic" and prerequisites
Controlled vocabulary	Subject scheme enforces consistent classification	Metadata filters work reliably because values are constrained

End-to-end pipeline architecture

The complete pipeline from DITA source to chatbot answer:

```
#####
# DITA Source ##### Section-Aware ##### Embedding #
```

```

# (.dita XML) # # Chunking # # Model #
#####
#####
# Metadata ##### Vector Store #
# Extraction # # (+ metadata)#
#####
#
#####
# Answer ##### Cross- ##### Semantic #
# Generation # # Encoder # # Search #
# (LLM) # # Reranker # # + Filters #
#####
#
#####
# Formatted #
# Response #
# (with cites) #
#####

```

Each stage is configured in `chatbot-config.json` and can be tuned independently.

Key configuration parameters

The chatbot configuration file controls every stage of the pipeline:

```

{
  "embedding": {
    "model": "text-embedding-3-large",
    "dimensions": 3072,
    "similarity_metric": "cosine"
  },
  "chunking": {
    "strategy": "section-aware",
    "min_tokens": 100,
    "max_tokens": 1500,
    "overlap_sentences": 2
  },
  "retrieval": {
    "top_k": 8,
    "score_threshold": 0.72,
    "reranker": "cross-encoder",
    "metadata_boost": {
      "chatbot-priority-high": 1.3,
      "chatbot-priority-medium": 1.0,
      "chatbot-priority-low": 0.8
    }
  }
}

```

The remaining chapters explain each component in detail.

Related tasks

[Section-Aware Chunking and Embedding Strategy](#) on page 172

How DITA's section structure enables semantically coherent chunks that preserve metadata context, and how those chunks are transformed into high-dimensional vectors for similarity search.

Related reference

[DITA to Schema.org Type Mapping](#) on page 100

Reference mapping of DITA topic types, elements, and metadata to schema.org structured data types for AI-readable output and chatbot classification.

Section-Aware Chunking and Embedding Strategy

How DITA's section structure enables semantically coherent chunks that preserve metadata context, and how those chunks are transformed into high-dimensional vectors for similarity search.

Why does naive text splitting fail?

Most RAG systems split documents by character count — every 1000 characters, start a new chunk. This approach has critical problems:

- **Broken context:** A chunk boundary splits a code example in half, making both halves meaningless
- **Mixed topics:** A chunk contains the end of one section and the beginning of another, conflating two unrelated subjects
- **Lost identity:** Plain text chunks have no metadata — the system cannot tell which chunk is about installation vs. troubleshooting
- **No hierarchy:** All chunks appear equal, with no way to prioritize or filter

DITA solves all four problems because the XML structure explicitly marks section boundaries, topic types, and metadata.

How does section-aware chunking work?

Section-aware chunking uses DITA's XML structure as the chunk boundary instead of arbitrary character counts:

```
DITA Topic: "Content Reuse Strategies"
### shortdesc (inherited by all chunks)
### prolog metadata (inherited by all chunks)
### section[@id="why-reuse"]
#   ### CHUNK 1: title + paragraphs (380 tokens)
### section[@id="conref-mechanism"]
#   ### CHUNK 2: title + paragraphs + codeblock (920 tokens)
### section[@id="keyref-mechanism"]
#   ### CHUNK 3: title + paragraphs + table (1100 tokens)
### section[@id="when-to-use-which"]
#   ### CHUNK 4: title + paragraphs + table (780 tokens)
```

Each chunk is produced by one rule: **one section = one chunk**. The chatbot configuration specifies boundaries:

- **Minimum:** 100 tokens — sections smaller than this are merged with the next section
- **Maximum:** 1500 tokens — sections larger than this are split at paragraph boundaries
- **Overlap:** 2 sentences from the end of the previous chunk are prepended for continuity

How does metadata attach to each chunk?

Every chunk inherits the full prolog metadata from its parent topic. The chunking process creates a JSON payload for each chunk:

```
{
  "chunk_id": "content-reuse__conref-mechanism",
  "topic_id": "content-reuse",
  "topic_type": "concept",
  "section_id": "conref-mechanism",
  "section_title": "How does conref work?",
  "shortdesc": "Techniques for reusing content ...",
  "text": "A conref (content reference) pulls ...",
  "metadata": {
    "audience": "technical-writer",
    "difficulty": "intermediate",
    "content-domain": "content-reuse",
    "content-intent": "explain",
    "chatbot-priority": "high",
    "service-line": "dita-engineering",
    "xml:lang": "en-US",
    "created": "2026-01-15"
  },
  "token_count": 920
}
```

This metadata is stored alongside the vector in the database, enabling filtered search (e.g., "show me only beginner content about publishing").

How are chunks converted to vectors?

Each chunk's text is passed through an embedding model that produces a high-dimensional vector representation:

Parameter	Value	Rationale
Model	text-embedding-3-large	Best performance on technical content retrieval benchmarks
Dimensions	3072	Full dimensionality preserves fine-grained semantic distinctions between technical concepts
Similarity metric	Cosine similarity	Scale-invariant; works well with normalized embeddings
Input format	shortdesc + section title + text	Prepending the shortdesc gives each chunk topic-level context

The embedding input is constructed as: `[shortdesc] [section_title]: [chunk_text]`. This ensures the embedding captures both the topic-level context and the section-level detail.

Before and after: chunk quality comparison

Before (naive chunking of unstructured content):

```
Chunk: "...to install the toolkit. ## Setting Up
You can configure the toolkit by editing the
conf/configuration.properties file. The key settings
are: - temp.dir: The temporary directory..."

Metadata: { source: "guide.xml", page: 12 }
Embedding quality: LOW (mixed topics, no context)
```

After (section-aware chunking of DITA):

```
Chunk: "How to configure the DITA Open Toolkit:
You can configure the toolkit by editing the
conf/configuration.properties file. The key settings
are: temp.dir (the temporary directory)..."

Metadata: {
  topic_type: "task",
  audience: "build-engineer",
  difficulty: "intermediate",
  content-domain: "publishing",
  content-intent: "how-to",
  chatbot-priority: "high"
}
Embedding quality: HIGH (focused, typed, metadata-rich)
```

Related concepts

[RAG Pipeline Architecture: From DITA to Chatbot Answers](#) on page 170

How a Retrieval-Augmented Generation pipeline transforms structured DITA content into accurate, cited chatbot responses — architecture overview and component roles.

Related reference

[DITA to Schema.org Type Mapping](#) on page 100

Reference mapping of DITA topic types, elements, and metadata to schema.org structured data types for AI-readable output and chatbot classification.

Retrieval, Reranking, and Answer Generation

How the chatbot searches the vector store with metadata filters, reranks results with a cross-encoder, and generates cited answers grounded entirely in the training documentation.

How does metadata-filtered retrieval work?

When a user asks a question, the chatbot performs a **two-phase retrieval**:

1. **Intent classification:** The query is classified as a question type (explain, how-to, troubleshoot, reference, definition)

2. Filtered vector search: The query embedding is compared against the vector store, but only chunks matching the metadata filters are considered

Example query flow:

```
User query: "How do I set up a CI/CD pipeline for DITA builds?"

Intent detected: how-to
Metadata filters applied:
  content-intent IN (how-to, tutorial)
  content-domain IN (devops, publishing)

Vector search: top_k=8, score_threshold=0.72

Results before filtering: 3,200 chunks
Results after filtering:   140 chunks
Results after vector search: 8 chunks
```

Without metadata filters, the search would scan all 3,200 chunks and might return concept explanations instead of task instructions. The metadata filter narrows the search space by 96%, dramatically improving both speed and relevance.

How does chatbot-priority boosting affect results?

The `chatbot-priority` metadata field applies a score multiplier to retrieval results:

Priority value	Score multiplier	Effect on ranking
high	1.3×	Core content surfaces first; user-facing tasks and key concepts get top placement
medium	1.0×	Standard ranking; supporting content appears in natural order
low	0.8×	Peripheral content (historical notes, edge cases) is deprioritized

This means a chunk with a vector similarity score of 0.85 and high priority gets an effective score of 1.105, while a chunk at 0.88 with low priority gets 0.704. The high-priority chunk ranks first despite a lower raw similarity score.

Why is cross-encoder reranking necessary?

Vector similarity search is fast but imprecise — it finds chunks that are *topically similar* but may not actually *answer the question*. A cross-encoder reranker is a second model that reads the query and each candidate chunk together and scores how well the chunk answers the query.

The pipeline:

1. Vector search retrieves 8 candidates (`top_k = 8`)
2. Cross-encoder scores each (query, chunk) pair on a 0-1 relevance scale
3. Chunks are re-sorted by cross-encoder score

4. Only chunks above the reranker threshold are passed to the LLM

In testing, cross-encoder reranking improved answer accuracy by 18–24% compared to vector search alone, because it catches cases where surface-level word overlap misleads the embedding model.

How does the LLM generate grounded answers?

The answer generation prompt is constructed from the top-ranked chunks:

```
SYSTEM PROMPT:
You are a DITA training assistant. Answer the user's
question using ONLY the provided context chunks. If the
context does not contain the answer, say so. Cite your
sources using [topic_id § section_id] format.

CONTEXT CHUNKS:
[1] topic_id: github-actions-cicd
    section: pipeline-setup
    type: task
    "To set up a CI/CD pipeline for DITA builds..."

[2] topic_id: docker-basics
    section: container-builds
    type: task
    "Docker containers enable reproducible DITA builds..."

[3] topic_id: dita-publishing
    section: automated-builds
    type: concept
    "Automated publishing pipelines ensure..."

USER QUESTION:
"How do I set up a CI/CD pipeline for DITA builds?"
```

The generated answer cites specific sections, allowing users to verify the information:

```
GENERATED ANSWER:
To set up a CI/CD pipeline for DITA builds, follow these
steps:

1. Create a GitHub Actions workflow that triggers on pushes
   to your main branch [github-actions-cicd § pipeline-setup]

2. Use a Docker container with DITA-OT pre-installed to
   ensure reproducible builds [docker-basics § container-builds]

3. Configure the pipeline to run `dita -i your.ditamap
   -f html5 -o output/` as the build step
   [github-actions-cicd § pipeline-setup]

See also: Automated Publishing (dita-publishing § automated-builds)
```

How does the glossary enhance answers?

When a query or answer contains terms defined in the DITA glossary, the chatbot can:

- **Inline definitions:** Automatically expand abbreviations the first time they appear ("DITA-OT (DITA Open Toolkit)")
- **Link to glossary entries:** Add a "Key terms" section at the bottom of the answer

- **Resolve synonyms:** Map user terminology to the canonical terms in the controlled vocabulary, improving retrieval matching
- **Scope warnings:** Flag when a term applies to a specific version ("Note: keyref requires DITA 1.2+")

This is only possible because DITA glossary entries are structured data with `glossterm`, `glossdef`, `glossSurfaceForm`, and `glossAlt` — not just a flat list of term-definition pairs.

Part

X

Benefits, Utilization & Impact

Topics:

- [Measurable Benefits and ROI of Intelligent Content](#)
- [Utilizing Intelligent Content Features: A Practical Guide](#)
- [Impact on the Final Product: How Intelligence Changes Everything](#)

Measurable Benefits and ROI of Intelligent Content

Quantified improvements in retrieval accuracy, content coverage, maintenance efficiency, and user experience that result from transforming unstructured content into AI-ready DITA.

Before and after: measured improvements

The Extense training guide migration provides concrete metrics at each transformation stage:

Metric	Before (unstructured XML)	After (enriched DITA)	Change
AI-readiness score	2.1 / 10	9.2 / 10	+338%
Topics with full metadata	0%	100%	Complete coverage
Retrievable content chunks	~80 (arbitrary splits)	~210 (section-aware)	+163%
Metadata-filterable chunks	0	210	100% addressable
Content types classified	1 (generic page)	4 (concept, task, reference, glossary)	Intent routing enabled
Reuse instances (conref/keyref)	0	50+ (product names, version strings)	Single-source consistency
Taxonomy-controlled values	0	8 facets × 50 topics = 400 values	Classification consistency
Glossary terms defined	0	41 structured entries	Term resolution enabled

How does retrieval quality improve?

The combination of section-aware chunking, metadata filtering, and intent routing produces measurably better chatbot answers:

- **Precision improvement:** Metadata filters eliminate 95%+ of irrelevant chunks before vector search begins, reducing false positives
- **Intent matching:** "How do I install DITA-OT?" retrieves task topics because intent=how-to maps to topic type=task. Without typing, the chatbot might return concept explanations

- **Audience relevance:** A developer asking about CI/CD gets developer-level content, not beginner-level overviews
- **Priority ranking:** High-priority core content surfaces above peripheral edge cases even at similar similarity scores

What are the long-term maintenance benefits?

- **Single-source updates:** Change a product name via keyref in one place, and it updates everywhere — including all chatbot chunks on the next ingestion cycle
- **Conditional content:** Add audience-specific paragraphs without duplicating entire topics. The chatbot delivers the right version based on user context
- **Version tracking:** The `critdates` metadata tracks when content was created and revised, enabling freshness-based retrieval preferences
- **Scalable architecture:** Adding new topics to the bookmap automatically makes them discoverable by the chatbot — no pipeline reconfiguration required
- **Automated validation:** DITA-OT validation catches broken links, invalid metadata values, and structural errors before they reach the chatbot

What is the business impact?

- **Reduced support costs:** AI-powered documentation answers questions 24/7, deflecting support tickets
- **Faster onboarding:** New team members get personalized learning paths based on role and skill level
- **Competitive differentiation:** Intelligent documentation becomes a product feature, not just a cost center
- **Multi-channel delivery:** The same DITA source powers HTML5, PDF, chatbot, and schema.org-enhanced web pages simultaneously
- **Localization readiness:** `xml:lang` and structured content dramatically reduce translation costs and improve translation quality

Utilizing Intelligent Content Features: A Practical Guide

How to leverage every metadata field, taxonomy facet, and structural element to maximize chatbot accuracy, search precision, and multi-channel delivery.

Feature 1: Metadata-filtered retrieval

What it does: Narrows the vector search space before similarity scoring begins, ensuring only relevant content is considered.

Metadata fields used: audience, difficulty, content-domain, content-intent

How to utilize it:

User scenario	Metadata filter applied	Retrieval result
"I'm a technical writer learning DITA"	audience=technical-writer AND difficulty IN (beginner, intermediate)	Excludes advanced developer content; surfaces authoring guides
"How do I create an XSLT plugin?"	content-intent=how-to AND content-domain=xslt	Returns only step-by-step XSLT task topics
"What is a conkeyref?"	content-intent IN (explain, definition)	Returns concept topics and glossary definitions, not task topics

Feature 2: Intent-to-type routing

What it does: Matches the type of question the user is asking to the type of content most likely to answer it.

Metadata fields used: content-intent, topic type (concept/task/reference)

How to utilize it:

Query pattern	Detected intent	Preferred content type
"What is X?"	explain / definition	Concept topics + glossary entries
"How do I X?"	how-to	Task topics with step-by-step instructions
"X is not working"	troubleshoot	Troubleshooting sections + task topics
"List of X" / "Parameters for X"	reference	Reference topics with tables and parameter lists
"Best way to X"	best-practice	Best practice sections + concept topics

Feature 3: Prerequisite-based learning paths

What it does: Uses the [prerequisites](#) metadata and map hierarchy to suggest a learning sequence when a user encounters unfamiliar concepts.

Metadata fields used: prerequisites, difficulty, map nesting order

How to utilize it:

- When a user asks about an advanced topic, the chatbot checks the prerequisite chain and offers foundational content first
- After answering a question, the chatbot suggests the next topic in the progression
- The difficulty metadata ensures the suggested path matches the user's level

Example: User asks about XSLT customization. Prerequisite chain: DITA fundamentals # publishing # XSLT customization # custom plugins. The chatbot asks: "Would you like to start with DITA publishing fundamentals first?"

Feature 4: Priority-weighted ranking

What it does: Ensures the most important content surfaces first, even when multiple chunks have similar relevance scores.

Metadata field used: chatbot-priority (high / medium / low)

How to utilize it:

- Mark core user-facing content (installation, first project, key concepts) as **high**
- Mark supporting content (best practices, advanced techniques) as **medium**
- Mark peripheral content (history, edge cases, deprecated features) as **low**

Feature 5: Schema.org-enhanced web output

What it does: Transforms DITA type semantics and metadata into schema.org structured data in the HTML5 output, improving search engine understanding.

Schema.org mappings:

DITA type	Schema.org type	SEO benefit
concept	TechArticle	Rich snippet with expertise level and dependencies
task	HowTo	Step-by-step rich snippet in search results
glossentry	DefinedTerm	Definition cards in search results
reference	TechArticle	Structured technical reference with proficiency level

Feature 6: Single-source multi-channel delivery

What it does: The same DITA source produces multiple output formats, each inheriting the intelligent content features.

- **Branded HTML5:** Web documentation with responsive design, navigation, and search
- **Branded PDF:** Print-ready documentation with professional formatting
- **Chatbot knowledge base:** Section-aware chunks with metadata for RAG retrieval
- **Schema.org web pages:** SEO-optimized pages with structured data
- **Localized versions:** Translation-ready content with xml:lang and structural consistency

A single source change (fixing a typo, updating a version number, adding a new topic) propagates to all channels on the next build cycle.

Impact on the Final Product: How Intelligence Changes Everything

How the accumulated intelligent content features — metadata, taxonomy, typing, and RAG — transform the end-user experience across chatbot, web, PDF, and search channels.

Impact on the chatbot experience

Without intelligent content, the chatbot is a keyword search engine wrapped in a conversational UI. With intelligent content, it becomes a **context-aware training assistant**:

Aspect	Without intelligent content	With intelligent content
Answer accuracy	Answers may mix installation steps with conceptual explanations. Wrong content type for the question.	Intent routing ensures "How do I...?" returns task steps and "What is...?" returns concept explanations.
Personalization	Same answer for everyone. A developer and a technical writer see identical results.	Audience metadata filters content by role. Developer gets CI/CD content; writer gets authoring guides.
Learning progression	No sense of order. User may receive advanced content before understanding fundamentals.	Prerequisite chains and difficulty levels guide users through a logical progression.
Citation reliability	Citations point to "document.md, page 12" — opaque and unverifiable.	Citations point to "[topic_id § section_id]" — precise, verifiable, and linkable.
Term consistency	User asks about "conditional text" — chatbot doesn't match to "DITaval filtering."	Glossary synonym resolution maps "conditional text" # "DITaval filtering" and returns the right content.

Impact on web search and discoverability

Schema.org structured data generated from DITA metadata produces **rich search results**:

- Task topics generate **HowTo** rich snippets showing numbered steps directly in Google results
- Glossary entries generate **DefinedTerm** cards that appear as definition boxes in search
- Concept topics include proficiency level metadata, helping users assess difficulty before clicking
- The subject scheme taxonomy maps to schema.org [about](#) and [audience](#) properties

This structured data is invisible to human readers but critical for search engines and AI agents that consume your content.

Impact on PDF and print output

The same intelligent content features that power the chatbot also improve traditional outputs:

- **Consistent terminology:** keyrefs ensure identical product names and version strings across all 200+ pages
- **Audience-filtered variants:** Generate a "Developer Guide" PDF that excludes writer-only content, or a "Quick Start" PDF filtered to beginner-difficulty topics only
- **Automatic glossary:** The structured glossary generates a professional formatted glossary section in the PDF backmatter
- **Cross-references:** Map-aware cross-references generate accurate page numbers and section links

Impact on content maintenance and scalability

As the content corpus grows, intelligent content becomes increasingly valuable:

- **Onboarding new authors:** The taxonomy and subject scheme provide guardrails — new authors cannot invent their own classification values. Metadata validation catches errors at build time.
- **Adding new topics:** New topics are added to the bookmap and automatically become chatbot-searchable, PDF-included, and web-published on the next build. Zero pipeline reconfiguration.
- **Content auditing:** The metadata makes it trivial to find stale content (`critdates/revvised` older than 12 months), uncategorized topics, or gaps in coverage.
- **Translation readiness:** `xml:lang` and structured content reduce translation costs by 30–50% compared to unstructured documents, and enable per-section translation memory matching.

The complete transformation journey

From unstructured XML files in a folder to an intelligent content system:

```

STAGE 1: Unstructured XML files
### No structure # AI can't parse intent
### No metadata # chatbot can't filter or rank
### No typing # all content treated the same
### Result: keyword search, generic answers

STAGE 2: Basic DITA conversion
### XML structure # sections are parseable
### Minimal metadata # author + indexterms only
### Type separation # concept/task/reference
### Result: better chunking, no filtering

STAGE 3: Enriched DITA (AI-ready)
### Full metadata # 8 classification fields
### Controlled vocab # taxonomy-validated values
### Section IDs # addressable chunk boundaries
### Glossary # term resolution engine
### Schema.org # search engine optimization
### Result: intelligent retrieval, personalized
        answers, multi-channel delivery, learning
        paths, priority ranking, cited responses
  
```

The investment in each transformation stage compounds. Stage 3 features build on Stage 2 structure, which builds on Stage 1 source content. Skipping stages produces diminished returns.

Appendix

Appendices — Reference Material

Topics:

- [DITA-OT Architecture Diagram](#)
- [Glossary](#)
- [Complete Plugin Reference](#)
- [Appendix: Complete AI-Ready Metadata Field Reference](#)
- [From Unstructured XML to Intelligent DITA: Detailed Comparison and Improvements](#)

DITA-OT Architecture Diagram

ASCII reference diagram showing the complete DITA-OT HTML5 processing pipeline from inputs through preprocessing, transformation, and output.

DITA-OT HTML5 pipeline overview

```
#####
#                               DITA-OT (HTML5) ARCHITECTURE – with preprocess2 option
#                               #####
#
# INPUTS (your repo)
#####
# • DITA map (.ditamap) + topics (.dita)
#
# • Resources (images/media/code)
#
# • DITAVAL filters (audience/platform/product/props)
#
# • Plug-ins (XSLT / Ant / Java) + build properties (.properties / CLI params)
#
#####
#                               |
#                               v
#
# SELECT PREPROCESSING PIPELINE (HTML5)
#####
# Choose ONE (depends on toolkit/version/transtype configuration):
#
#
# #####
# ##### # A) preprocess (legacy/mixed) # OR # B) preprocess2 (map-first)
# # # map + topic steps interleave # # complete map processing first, then
# # # # # # # process topics after map features are
# # # # # # # resolved
# # # # # # #
# # # # # # #
# ##### #
#####
#                               |                               |
#                               v                               v
#
# COMMON PREPROCESSING RESPONSIBILITIES (both pipelines produce the same "normalized" intent)
#####
# 1) Map parsing + key collection
#
# • keydefs, key scopes, key space
#
# 2) Conditional processing / filtering
#
# • DITAVAL/props applied # content included/excluded
#
# 3) Reference resolution
#
# • conref / conkeyref / keyref / coderef
#
# 4) Pull processing
#
```

```

#   • metadata pulls, navtitle/title behavior
#
# 5) Chunking decisions + job file lists
#
#   • "what becomes which HTML file/page"
#
# 6) Relationship/link metadata
#
#   • maplink, rel tables
#
#
# RESULT: normalized + resolved intermediate content
#
#####
      |
      v
TEMP WORKSPACE (work dir / temp dir)
#####
# • normalized / filtered / resolved topics + maps
#
# • job configuration + file lists (what to build)
#
# • logs + diagnostics
#
#
# Debug rule: if intermediates are wrong, HTML5 output will be wrong.
#
#####
      |
      v
HTML5 PROCESSING (format-specific)
#####
# A) HTML5 XSLT transform
#
#   • converts normalized DITA to HTML markup
#
#   • applies your XSLT import overrides (if plug-in installed)
#
#
# B) Navigation / TOC generation
#
#   • index/TOC artifacts
#
#   • optional embedded nav per page (e.g., nav-toc=full)
#
#
# C) Post-processing / packaging
#
#   • copy CSS/JS/images
#
#   • add extra assets (search index, analytics hooks, etc.)
#
#####
      |
      v
HTML5 OUTPUT (deliverable)
#####
# • chunked topic pages (or single-page output)
#
# • sidebar/embedded TOC (if configured)
#
# • CSS/JS/images/fonts (packaged into output)
#
#####

```

Figure 1: DITA-OT (HTML5) Architecture — preprocess vs preprocess2

Where your customization applies

```
#####
# Plug-ins + extension points can affect:
#
#
# • Preprocessing: Ant target hooks (recommended: pre/post preprocess)
#
# - add validation, metadata generation, diagnostics
#
# - step-specific hooks are more fragile and may differ between preprocess/preprocess2
#
#
# • HTML5 processing: XSLT import overrides + TOC overrides + parameters
#
# - safest way to change HTML structure, classes, data-* hooks, nav markup
#
#
# • Build parameters: .properties/CLI (nav-toc, filters, css, chunking strategy, etc.)
#
#####
```

Figure 2: HTML5-focused customization control plane

Related concepts

[Publishing with DITA-OT](#) on page 104

DITA is a source format, not an output. Publishing is the process of:

Related tasks

[DITA-OT Architecture and DevOps](#) on page 132

This handbook is for engineers who want to develop DITA Open Toolkit—either by contributing to the core toolkit or by building production-grade transformations and pipelines with plug-ins, containers, and CI/CD.

Glossary

Comprehensive DITA and DITA-OT terminology used throughout the Extense training documentation. Each term includes definitions, abbreviations, usage guidance, and scope notes for chatbot knowledge retrieval.

Related concepts

[What Is Intelligent Content?](#) on page 10

Intelligent content is structurally rich, semantically categorized, and machine-readable — enabling AI systems to discover, classify, retrieve, and deliver it without human intervention.

[DITA at a Glance](#) on page 28

DITA is designed for teams that need:

[Authoring DITA Topics](#) on page 48

Writing DITA is not just writing in XML. It's writing with:

Related tasks

[Quick Start Guide](#) on page 22

This quick start guide helps you navigate the documentation collection and begin working with DITA immediately.

[Reuse, Keys, and Variant Management](#) on page 66

For many organizations, multi-format output is helpful—but reuse is what changes the economics of documentation. Reuse reduces:

Topic

A modular, self-contained unit of content in DITA. Topics are the fundamental building blocks and come in typed flavors (concept, task, reference) to match reader intent.

DITA topic

Note

Use "topic" when referring to any standalone DITA content unit. Specify the type (concept topic, task topic) when the distinction matters.

Note

Applicable to all DITA versions (1.0+). Topics are the core unit of structured authoring.

Concept

A DITA topic type for explanatory, background, or "what is" content. Concepts answer questions like "What is X?" and "Why does X matter?" They use the <concept> root element and <conbody> for content.

concept topic

Note

Write concepts for introductory, explanatory, or background material. Do not include step-by-step procedures in a concept — use a task topic instead.

Note

One of the three core DITA information types.

Task

A DITA topic type for procedural, step-by-step content. Tasks answer "How do I do X?" and use structured <steps> elements with <cmd> and optional <info>, <stepresult>, and <stepxmp> sub-elements.

task topic

Note

Use task topics for any procedural content. Include `<prereq>` for prerequisites, `<context>` for background, and `<result>` for expected outcomes.

Note

One of the three core DITA information types. DITA also supports "general task" for less-structured procedures.

Reference

A DITA topic type for lookup-oriented content such as API references, parameter tables, CLI options, or specifications. Uses the `<reference>` root element with `<refbody>` containing tables, property lists, or syntax diagrams.

reference topic

Note

Use reference topics for content that users look up rather than read sequentially. Structure with tables and definition lists for scannability.

Note

One of the three core DITA information types.

Map

An assembly structure that organizes topics into a hierarchy for publishing. Maps define the table of contents, navigation, and relationship tables. A map does not contain content directly — it references topics.

DITA map

ditamap

Note

Use maps to assemble topics into deliverables. A single topic can appear in multiple maps. Use bookmaps for book-like outputs with front/back matter.

Note

Maps use the `.ditamap` file extension. Bookmaps are a specialization of maps.

Bookmap

A specialization of DITA map that adds book-oriented structures like parts, chapters, appendices, front matter, and back matter. Used for producing book-like deliverables (PDF, printed manuals).

DITA bookmap

Note

Use bookmaps when your output needs book structure (chapters, parts, appendices, TOC, index). Use plain maps for simpler HTML or help outputs.

Note

Available in DITA 1.1+. Requires the bookmap DTD/schema.

Information Typing

The practice of using different topic structures (concept, task, reference) to match different reader intents and optimize the user experience. This is a core DITA design principle.

information typing

Note

Apply information typing by asking: "Is this content explaining something (concept), showing how to do something (task), or providing lookup data (reference)?"

Note

Foundational to the DITA architecture. Custom specializations can add domain-specific types.

Reuse

Referencing content instead of duplicating it. DITA provides three primary reuse mechanisms: conref (content reference), keyref (key reference), and conkeyref (key-based content reference). Reuse reduces translation cost, editing effort, and inconsistency.

content reuse

Note

Design content for reuse by keeping topics self-contained, using conref for shared blocks, keyref for variable text, and DITaval for variant filtering.

Note

Reuse is often cited as the primary economic justification for adopting DITA.

Conref

Content reference — an attribute that pulls content from another element by matching its @id. The referenced content replaces the referencing element at processing time. This is "copy by reference" at the element level.

content reference (conref)

conref

Note

Use conref to share identical content (warnings, notes, boilerplate paragraphs) across topics. The source and target elements must be the same type.

Note

Available in DITA 1.0+. For key-based content pulling, see conkeyref.

Keyref

Key reference — resolves links and variable text via keys defined in the DITA map. Keys provide late binding: the same key can resolve differently in different maps or key scopes, enabling context-dependent content.

key reference (keyref)

keyref

Note

Use keyrefs for product names, version numbers, and links that may change between deliverables. Define keys in maps using <keydef>.

Note

Available in DITA 1.2+. Key scopes (DITA 1.3) enable multiple definitions of the same key in different contexts.

Conkeyref

Combines keys and content pulling — a key-based content reference that resolves the source element via a key definition, then pulls the content like a conref. Enables flexible multi-context reuse.

key-based content reference (conkeyref)

conkeyref

Note

Use conkeyref when the reuse source varies by context (e.g., different product editions pull from different warehouse topics).

Note

Available in DITA 1.2+. Requires key definitions in the map.

Filtering

Producing content variants by including or excluding elements based on conditional attributes (audience, platform, product, otherprops) and DITAVAL rules. Also called "conditional processing" or "profiling."

conditional filtering

DITAVAL filtering

Note

Apply filtering attributes to elements, then use DITAVAL files to include/exclude/flag content at build time. Plan your filtering strategy before authoring.

Note

DITA provides four built-in filtering attributes: audience, platform, product, and otherprops.

DITAVAL

An XML file that defines conditional processing rules (include, exclude, flag) applied during DITA-OT builds. DITAVAL files control which content variants appear in the output.

DITAVAL file

Note

Create separate DITAVAL files for each product variant or audience. Reference them in build commands with `--filter=` or in DITA-OT project files.

Note

DITAVAL is processed during the filtering stage of DITA-OT preprocessing.

Specialization

Extending DITA's type system to represent domain-specific concepts while maintaining compatibility with standard DITA processing. Specializations inherit from base types and can add constraints or new elements.

DITA specialization

Note

Consider specialization when your domain has recurring structures that don't fit standard topic types. Always specialize from the closest existing base type.

Note

The "Darwin" in DITA refers to this evolutionary inheritance model.

Constraints

Restricting the DITA content model to enforce consistency — for example, removing optional elements or requiring certain metadata. Constraints tighten the model without adding new elements.

DITA constraints

Note

Use constraints to enforce organization-specific authoring rules (e.g., require shortdesc, prohibit nested topics).

Note

Available in DITA 1.2+. Constraints are complementary to specialization.

Key Scope

A named context within a DITA map that limits key definitions to a specific branch. Key scopes enable the same key name to resolve differently in different parts of the same map.

key scope

keyscope

Note

Use key scopes when a single deliverable needs different values for the same key. Define with @keyscope on topicref elements.

Note

Available in DITA 1.3+.

Branch Filtering

Applying DITaval filtering rules to a specific branch (subtree) of a DITA map rather than the entire map. This enables multiple filtered variants within a single build.

branch filtering

Note

Use <ditavalref> elements on map branches to apply variant-specific filtering without separate builds.

Note

Available in DITA 1.3+. Processed during the DITA-OT branch-filter preprocessing step.

Relationship Table

A table structure in DITA maps that defines non-hierarchical relationships between topics. Each row links related concepts, tasks, and references, generating "Related links" sections in the output.

relationship table (reltable)

reltable

Note

Use relationship tables instead of inline cross-references when link relationships should be managed centrally in the map.

Note

Relationship tables are processed during DITA-OT preprocessing to generate related-links.

Subject Scheme

A special DITA map type that defines controlled vocabularies and taxonomies for metadata values. Subject schemes constrain attribute values to a predefined list, enabling consistent classification.

subject scheme map

Note

Create subject scheme maps to define enumerated values for filtering attributes. This prevents inconsistent metadata and enables AI-readable taxonomies.

Note

Available in DITA 1.2+. Uses the subjectScheme DTD/schema.

DITA Open Toolkit

An open-source publishing engine that transforms DITA XML content into deliverable formats (HTML5, PDF, EPUB, etc.). DITA-OT provides a multi-stage processing pipeline with extension points for customization.

DITA Open Toolkit (DITA-OT)

DITA-OT

Note

DITA-OT is the standard publishing tool for DITA content. Most CCMS products use DITA-OT internally. Current version: 4.3.

Note

Open-source project at github.com/dita-ot/dita-ot. Written in Java with XSLT transforms.

Transtype

The output format identifier used by DITA-OT to select a transformation (e.g., html5, pdf, markdown). Each transtype maps to a set of processing modules and XSLT stylesheets.

transformation type (transtype)

Note

Specify the transtype with `-f` or `--format` in DITA-OT commands. Custom plugins can register new transtypes.

Note

Built-in transtypes include `html5`, `pdf`, `xhtml`, `eclipsehelp`, and `markdown`.

Preprocessing

The first stage of DITA-OT processing that resolves references, applies filtering, normalizes content, and produces intermediate files. DITA-OT offers two pipelines: legacy "preprocess" and map-first "preprocess2."

DITA-OT preprocessing (preprocess / preprocess2)

preprocess2

Note

Use `preprocess2` (map-first) for HTML5 output — it is faster and produces cleaner intermediate files.

Note

`preprocess2` is available in DITA-OT 3.0+ and is the default for HTML5 in DITA-OT 4.x.

Plugin

An extension package for DITA-OT that can inject XSLT overrides, Ant targets, custom transtypes, processing modules, and static resources. Plugins are the standard way to customize DITA-OT output.

DITA-OT plugin (plug-in)

plugin

Note

Create plugins for any output customization. Use the `plugin.xml` descriptor to declare dependencies and extension point contributions.

Note

Plugin IDs follow reverse-domain naming (e.g., com.extense.html5.branded).

Extension Point

A named hook in the DITA-OT processing pipeline where plugins can contribute behavior — XSLT overrides, Ant targets, parameters, or resources.

DITA-OT extension point

Note

Key extension points: dita.xsl.html5 (HTML5 XSLT), dita.xsl.html5.toc (TOC), dita.conductor.target (Ant targets).

Note

Extension points are documented in the DITA-OT developer reference.

Intermediate Files

Normalized, filtered XML content produced during preprocessing and used by later processing stages. Found in the DITA-OT temp directory.

DITA-OT intermediate files

Note

Inspect intermediate files in the temp/ directory to debug preprocessing issues.

Note

Intermediate files include resolved conrefs, applied filtering, and normalized metadata.

Chunking

How DITA-OT decides to split or combine topics into output pages or files. Controlled by the @chunk attribute on map elements.

DITA chunking

Note

Set @chunk="to-content" to combine child topics into one file. Use @chunk="by-topic" to split nested topics.

Note

Chunking behavior changed significantly in DITA 1.3 and DITA-OT 4.x.

Outputclass

A DITA attribute that passes styling hints to the output. The value is typically rendered as a CSS class name in HTML output or a formatting attribute in PDF.

outputclass attribute

Note

Use outputclass for visual customization that doesn't change content meaning. Example: `<codeblock outputclass="language-xml">`

Note

Available on all DITA elements. Processed by XSLT transforms during output generation.

Project File

A DITA-OT project file (.xml) that defines multiple deliverables, contexts, and publications in a single build configuration.

DITA-OT project file

Note

Define deliverables in a project file to build multiple outputs from a single command: `dita --project=myproject.xml`

Note

Available in DITA-OT 3.4+.

Component Content Management System

A content management system designed for component-level (topic-level) content. A CCMS manages DITA topics, maps, images, and metadata as individual components with version control, workflow, and publishing integration.

Component Content Management System (CCMS)

CCMS

Note

Consider a CCMS when your team needs component-level version control, workflow management, and multi-author collaboration beyond what file-based systems like Git provide.

Note

Extense provides CCMS implementation and integration services.

Structured Content

Content authored with explicit semantic markup (XML, DITA, etc.) that separates content from presentation, enables reuse, and supports automated processing.

structured content

Note

Structured content is the foundation of scalable documentation programs. DITA is the most widely adopted structured content standard for technical documentation.

Note

Extense specializes in structured content strategy and migration from unstructured to structured formats.

Content Migration

The process of converting existing documentation from unstructured formats (Word, FrameMaker, HTML, flat XML) into structured DITA XML. Includes analysis, information architecture design, conversion, and validation.

content migration

Note

Plan content migration by first analyzing existing content, designing the DITA information architecture, then converting with a combination of automated tools and manual review.

Note

Extense offers end-to-end content migration services.

Intelligent Content

Content that is structurally rich, semantically categorized, and machine-processable — enabling automated discovery, personalization, and AI-driven delivery.

intelligent content

Note

Transform standard DITA content into intelligent content by adding rich metadata, controlled vocabularies, content graphs, and machine-readable annotations.

Note

This training guide demonstrates intelligent content principles through its subject scheme, enriched prologs, and chatbot-ready metadata.

Retrieval-Augmented Generation

An AI architecture that enhances language model responses by retrieving relevant content from a knowledge base before generating answers. RAG combines vector search over document chunks with LLM generation for grounded, accurate responses.

Retrieval-Augmented Generation (RAG)

RAG

Note

Structure DITA content for RAG by ensuring each topic has clear metadata, a good shortdesc, and proper chunking boundaries at section level.

Note

The Extense chatbot uses RAG over DITA-derived content chunks.

Complete Plugin Reference

Full source of the v2 starter plugin (com.acme.html5.sidebar) demonstrating production-ready DITA-OT HTML5 customization.

plugin.xml

The plugin descriptor declares the plugin ID, its dependency on the HTML5 transtype, and two extension points.

```
<?xml version="1.0" encoding="UTF-8"?>
<plugin id="com.acme.html5.sidebar" version="1.0.0">
  <require plugin="org.dita.html5"/>
  <feature extension="dita.xsl.html5" file="xsl/html5/override.xsl"/>
  <feature extension="dita.xsl.html5.toc" file="xsl/toc/toc-override.xsl"/>
</plugin>
```

override.xsl — Topic Transform (key excerpts)

Import + JS injection:

```
<xsl:import href="plugin:org.dita.html5:xsl/dita2html5.xsl"/>

<xsl:template name="gen-user-head">
  <xsl:next-match/>
  <script src="{concat($PATH2PROJ, 'js/site.js')}"/>
</xsl:template>
```

Topic wrapper (standardizes across concept/task/reference):

```
<xsl:template match="*[contains(@class, ' topic/topic ')]">
```

```

<article class="dita-topic" data-topic-type="{local-name()}">
  <xsl:apply-templates select="*[contains(@class,' topic/title ')]"/>
  <xsl:apply-templates select="*[contains(@class,' topic/shortdesc ')]"/>
  <main class="dita-body">
    <xsl:apply-templates
      select="*[contains(@class,' topic/body ')]
        | *[contains(@class,' concept/conbody ')]
        | *[contains(@class,' task/taskbody ')]
        | *[contains(@class,' reference/refbody ')]"/>
  </main>
</article>
</xsl:template>

```

Notes as callout boxes:

```

<xsl:template match="*[contains(@class,' topic/note ')]">
  <aside class="callout" if (@type) then concat(' callout-', @type) else ''">
    <xsl:if test="@type">
      <div class="callout-title"><xsl:value-of select="upper-case(@type)"/></div>
    </xsl:if>
    <div class="callout-body"><xsl:apply-templates/></div>
  </aside>
</xsl:template>

```

Code blocks with Copy button:

```

<xsl:template match="*[contains(@class,' topic/codeblock ')]">
  <div class="codeblock">
    <button class="copy-btn" type="button">Copy</button>
    <pre><code><xsl:apply-templates/></code></pre>
  </div>
</xsl:template>

```

Task steps with anchors:

```

<xsl:template match="*[contains(@class,' task/step ')]">
  <li class="task-step"
    data-step="{count(preceding-sibling::*[contains(@class,' task/step ')] + 1}">
    <xsl:apply-templates/>
  </li>
</xsl:template>

```

toc-override.xsl — Sidebar Navigation

```

<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="3.0">
  <xsl:import href="plugin:org.dita.html5:xsl/toc.xsl"/>
  <xsl:template match="nav">
    <nav class="sidebar-toc">
      <xsl:apply-templates select="@* | node()"/>
    </nav>
  </xsl:template>
  <xsl:template match="li">
    <li data-level="{count(ancestor::li)+1}">
      <xsl:apply-templates select="@* | node()"/>
    </li>
  </xsl:template>
</xsl:stylesheet>

```

Sample DITaval Files

web.ditaval — exclude print-only content:

```
<val>
  <prop action="exclude" att="platform" val="print"/>
</val>
```

print.ditaval — exclude web-only content:

```
<val>
  <prop action="exclude" att="platform" val="web"/>
</val>
```

audience-admin.ditaval — admin content only:

```
<val>
  <prop action="include" att="audience" val="admin"/>
  <prop action="exclude" att="audience" val="novice"/>
</val>
```

Build Command

```
dita -i dita/maps/guides/getting-started.ditamap -f html5 -o out/html5 --nav-toc=full
--args.copycss=yes --args.cssroot=css/ --args.css=brand.css --args.filter=build/
ditaval/web.ditaval
```

Related tasks

[XSLT Customization Tutorial](#) on page 148

This tutorial teaches a practical, production-style way to customize HTML output from DITA using XSLT, primarily through DITA Open Toolkit DITA-OT 4.x plug-ins and official extension points.

Appendix: Complete AI-Ready Metadata Field Reference

Comprehensive reference for every metadata field used in the AI-ready DITA prolog — allowed values, purpose, AI pipeline usage, and examples.

Complete prolog template

Every DITA topic in an AI-ready corpus should contain this prolog structure:

```
<prolog>
  <author>Extense LLC</author>
  <critdates>
    <created date="YYYY-MM-DD"/>
    <revised modified="YYYY-MM-DD"/>
  </critdates>
  <metadata>
    <keywords>
```

```

    <indexterm>Primary term</indexterm>
    <indexterm>Secondary term</indexterm>
  </keywords>
  <category>taxonomy:category-name</category>
  <audience>audience-value</audience>
</metadata>
<resourceid appid="project-id" appname="chatbot"
  id="topic-id"/>
<data name="difficulty" value="level"/>
<data name="duration" value="PT##M"/>
<data name="content-domain" value="domain"/>
<data name="content-intent" value="intent"/>
<data name="chatbot-priority" value="priority"/>
<data name="service-line" value="service"/>
<data name="prerequisites" value="topic-id|none"/>
</prolog>

```

Field-by-field reference

Field	DITA element	Allowed values	Purpose	Pipeline usage
xml:lang	Root element attribute	en-US, de-DE, fr-FR, ja-JP, zh-CN (BCP 47)	Language identification for localization and glossary matching	Language filter, translation routing
author	<code><author></code>	Organization or person name	Content provenance tracking	Citation attribution
created date	<code><critdates><created></code>	YYYY-MM-DD (ISO 8601)	Content creation date for freshness tracking	Freshness-weighted retrieval
revised date	<code><critdates><revised></code>	YYYY-MM-DD (ISO 8601)	Last content update date	Staleness detection, prefer recent content
indexterms	<code><keywords><indexterm></code>	Free text (2–5 terms recommended)	Index generation, keyword matching boost	Keyword expansion for embedding enrichment
category	<code><category></code>	taxonomy:dita-training, taxonomy:intelligent-content	Top-level content classification	Corpus partitioning
audience	<code><audience></code>	technical-writer, content-strategist, developer, build-engineer, editor, localization, manager, all	Role-based content targeting	Audience filter in retrieval queries

Field	DITA element	Allowed values	Purpose	Pipeline usage
resourceid	<code><resourceid></code>	appid=project, appname=chatbot, id=topic-id	Unique topic identifier for cross-system linking	Chunk-to-source traceability, citation linking
difficulty	<code><data name="difficulty"></code>	beginner, intermediate, advanced, expert	Knowledge level required to understand the content	Difficulty filter, learning path progression
duration	<code><data name="duration"></code>	ISO 8601 duration: PT5M, PT10M, PT15M, PT20M, PT30M, PT45M, PT60M	Estimated read/completion time	Session planning, Schema.org timeRequired
content-domain	<code><data name="content-domain"></code>	dita-fundamentals, authoring, content-reuse, publishing, customization, xslt, devops, architecture, ccms, localization	Subject area classification	Domain-scoped retrieval, faceted search
content-intent	<code><data name="content-intent"></code>	explain, how-to, reference, troubleshoot, best-practice, tutorial, overview, definition	What the content does for the reader	Intent routing: maps user question type to content type
chatbot-priority	<code><data name="chatbot-priority"></code>	high (1.3× boost), medium (1.0×), low (0.8×)	Retrieval ranking weight	Score multiplier applied after vector similarity
service-line	<code><data name="service-line"></code>	dita-engineering, publishing-engineering, ccms-services, content-migration, training, xml-engineering, structured-content-strategy	Business service connection	Service recommendation, lead generation routing
prerequisites	<code><data name="prerequisites"></code>	topic-id of prerequisite topics or "none"	Learning path dependency chain	Prerequisite suggestion, ordered learning path generation

Quick validation checklist

Use this checklist to verify each topic has complete AI-ready metadata:

1. Root element has `xml:lang="en-US"` (or appropriate locale)
2. Prolog contains `<author>`
3. Prolog contains `<critdates>` with `<created>`
4. At least 2 `<indexterm>` entries
5. One `<category>` element
6. At least one `<audience>` element
7. One `<resourceid>` with appid, appname, and id
8. All 7 `<data>` elements present: difficulty, duration, content-domain, content-intent, chatbot-priority, service-line, prerequisites
9. All values match the controlled vocabulary from the subject scheme
10. Short description is 25–35 words and describes purpose, not content

Related concepts

[Topic-Based Information Architecture for Training](#) on page 76

How to design a modular, map-driven information architecture that supports progressive learning paths, role-based content delivery, and AI-powered navigation.

[Taxonomy and Controlled Vocabulary for AI Classification](#) on page 78

How a formal DITA subject scheme taxonomy provides the controlled vocabulary that AI systems need to classify, filter, and route content accurately.

Related tasks

[Metadata Enrichment: Adding the AI Layer to Every Topic](#) on page 94

Step-by-step process for adding the complete AI-ready prolog metadata to DITA topics — the single highest-impact action for improving chatbot retrieval quality.

[Migrating Content from Unstructured XML to AI-Ready DITA: Step by Step](#) on page 84

A complete, repeatable process for converting unstructured XML files into semantically rich, AI-ready DITA topics with full prolog metadata.

From Unstructured XML to Intelligent DITA: Detailed Comparison and Improvements

This document provides a comprehensive before-and-after comparison of the training content, showing the transformation from basic unstructured XML files to a progressive, intelligent DITA-based documentation set. It highlights structural, topical, and metadata improvements, with concrete examples and the value each change brings.

1. Starting Point: Unstructured XML Content

The original training content was organized as unstructured XML files, each covering a single topic or concept. Example:

Source File	Sample Content
core-topics/dita-topic-3-reuse-variants.xml	<pre><article> <h1>DITA Reuse and Variants</h1> conref, keyref, conkeyref DITAVAL filtering Governance patterns </ article></pre>
getting-started/dita_intro_guide.xml	<pre><article> <h1>DITA Introduction Guide</h1> What is DITA? Why use DITA? Key concepts </article></pre>

Unstructured XML lacked semantic structure, metadata, and advanced features for reuse, analytics, or personalization.

2. After: Progressive, Intelligent DITA Content

The content was transformed into modular DITA topics, each with rich metadata, semantic structure, and support for analytics, personalization, and AI integration. Example:

DITA Topic	Sample Content
core-topics/dita-reuse-variants.dita	<pre><topic id="dita-reuse-variants"> <title>Reuse, Keys, and Variant Management</title> <prolog> <metadata> <category>taxonomy:content-reuse</ category> <audience>technical-writer</audience> ... </metadata> </prolog> <conbody> ... <section id="analytics-instrumentation"> <title>Analytics and Instrumentation</title> <p>To measure the impact of reuse, instrument your output with analytics...</p> </section> <section id="personalization-conditional- processing"> <title>Personalization and Conditional Processing</title> <p>Use DITA conditional attributes...</p> </section> ... </conbody> </topic></pre>
getting-started/dita-intro-guide.dita	<pre><topic id="dita-intro-guide"> <title>DITA Introduction Guide</title> <prolog> <metadata> <category>taxonomy:introduction</category> <audience>all</audience> ... </metadata> </prolog> <conbody> ... <section id="analytics-instrumentation"> <title>Analytics and Instrumentation</title> <p>Track which introduction topics are most viewed...</p> </ section> ... </conbody> </topic></pre>

3. Key Improvements and Their Value (Expanded)

- **Modular Structure:** DITA topics are modular and reusable, supporting advanced publishing, content assembly, and easier maintenance. For example, a single topic about 'Password Reset' can be reused in multiple manuals, reducing duplication and ensuring consistency.

- **Rich Metadata:** Each topic now includes <category>, <audience>, and <keywords> for taxonomy, discoverability, and semantic search. This enables targeted content delivery, such as showing only developer-related topics in a developer portal.
- **Analytics Integration:** Topics include guidance for adding analytics, enabling data-driven content improvement. By tracking which topics are most accessed or where users struggle, teams can prioritize updates and measure documentation ROI.
- **Personalization:** Use of DITA conditional processing allows content to be tailored for different audiences, products, or outputs. For instance, a topic can display different instructions for Windows and Mac users, improving user experience and reducing confusion.
- **AI/Chatbot Readiness:** Topics are structured for easy integration with chatbots and NLP systems, supporting conversational help and smart search. This allows users to ask questions and receive precise answers, and enables automated support systems to leverage documentation directly.
- **Feedback Loops:** Guidance for collecting and acting on user feedback is now included, supporting continuous improvement. Feedback widgets and analytics help teams quickly identify gaps and respond to user needs, making documentation more responsive and effective.
- **Progressive Learning Path:** The ditamap now supports a step-by-step, role-based learning journey, not just a static list of topics. This helps trainees build skills progressively, from foundational concepts to advanced practices, ensuring a more effective and engaging learning experience.

4. Bigger Picture: Organizational Impact (Expanded)

These improvements make the documentation:

- **More maintainable and scalable:** Modular topics and maps reduce duplication, simplify updates, and support large-scale documentation projects.
- **Ready for analytics, personalization, and AI-driven workflows:** Semantic metadata and structured content enable advanced features like personalized help, smart search, and chatbot integration.
- **More discoverable and useful for diverse audiences:** Targeted content delivery ensures users find relevant information quickly, improving satisfaction and reducing support costs.
- **Continuously improving through feedback and data:** Built-in analytics and feedback loops enable agile, user-centered documentation practices, keeping content relevant and effective.
- **Aligned with modern content maturity models:** The documentation now supports the full maturity model, from content capture and structure to analytics, personalization, and AI-driven improvement, positioning the organization for future growth and innovation.

In practical terms, these changes mean faster onboarding, fewer errors, better user support, and a stronger foundation for digital transformation and intelligent content initiatives.

5. Summary Table: Before and After

Aspect	Before (Unstructured XML)	After (Intelligent DITA)
Structure	Flat, file-based	Modular, topic-based, map-driven
Metadata	None	Rich: category, audience, keywords

Aspect	Before (Unstructured XML)	After (Intelligent DITA)
Reuse	Manual copy-paste	DITA conref, keyref, maps
Analytics	Not possible	Guidance for instrumentation
Personalization	Not possible	Conditional processing, DITaval
AI/Chatbot	Not possible	Structured for integration
Feedback	Not built-in	Guidance for feedback loops
Learning Path	Static list	Progressive, role-based

6. Intelligent Content: Practical DITA and AI Benefits (Expanded)

The transformation to intelligent DITA content brings significant, practical benefits for both documentation teams and AI-driven workflows:

- **Semantic Metadata & Taxonomy:** Enables precise targeting, filtering, and semantic search. AI and users can find the right content faster, and content can be assembled dynamically for different audiences or products. For example, a chatbot can use metadata to recommend the most relevant troubleshooting guide based on user role and context.
- **Modular, Topic-Based Structure:** Supports content reuse, dynamic assembly, and easier updates. AI can extract and repurpose content for chatbots, portals, or custom outputs. This allows for automated content delivery and personalized user experiences.
- **Analytics & Instrumentation:** Lets you track usage, identify gaps, and improve content based on real data. Analytics can also train AI models for better recommendations and search, ensuring documentation evolves with user needs.
- **Personalization & Conditional Processing:** Delivers tailored content automatically using DITA attributes and DITaval. Supports dynamic, user-specific experiences in portals or apps, such as showing advanced options only to power users.
- **AI/Chatbot/NLP Readiness:** Structured topics and metadata make it easy to integrate with chatbots and virtual assistants, enabling conversational help and smart search. This reduces support workload and improves user satisfaction.
- **Feedback Loops & Continuous Improvement:** Built-in prompts and analytics enable rapid, data-driven updates, improving both docs and AI models over time. This ensures documentation remains accurate, relevant, and user-focused.

Example: A DITA topic tagged with `<audience>developer</audience>` and `<category>taxonomy:customization</category>` can be filtered for a developer portal, reused in multiple guides, and surfaced by AI when a developer asks a chatbot for “customization help.” Analytics show which topics are most accessed, guiding further improvements and smarter AI recommendations. Feedback from users is collected and used to refine the content, creating a virtuous cycle of improvement.

These changes make your documentation smarter, more adaptive, and ready for the future—serving both human users and AI systems with the right content, at the right time, in the right context. This is

the essence of intelligent content, enabling organizations to deliver superior user experiences and drive innovation.

7. Practical Examples: Intelligent DITA in Action

- **Example 1: Conditional Instructions**A DITA topic for software installation includes conditional steps for Windows, Mac, and Linux. Using platform attributes, the same topic delivers tailored instructions for each OS. *Benefit:* Reduces confusion, improves user experience, and supports automated content delivery in portals.
- **Example 2: Analytics-Driven Updates**Analytics show that users frequently access the 'Troubleshooting' section but rarely use 'Advanced Customization.' The team prioritizes updates to troubleshooting content and adds more examples, improving documentation ROI.*Benefit:* Data-driven improvement, focused updates, and measurable impact.
- **Example 3: Chatbot Integration**A DITA topic about password reset is tagged with <audience>end-user</audience> and <category>taxonomy:security</category>. When a user asks a chatbot “How do I reset my password?”, the bot surfaces the relevant topic, providing instant help.*Benefit:* Faster support, reduced workload, and improved satisfaction.
- **Example 4: Feedback Loops**A feedback widget is embedded in the HTML output. Users rate topics and leave comments. The team reviews feedback weekly, updating unclear sections and adding requested examples.*Benefit:* Continuous improvement, responsive documentation, and higher quality.
- **Example 5: Modular Reuse**A topic describing API authentication is reused in developer guides, admin manuals, and chatbot responses. Updates to the topic automatically propagate everywhere.*Benefit:* Consistency, reduced maintenance, and scalable content management.

8. Validation Issues and Solutions

During migration and enhancement, several validation issues may arise. Addressing these ensures content quality and readiness for intelligent workflows:

- **Issue: Missing Metadata**Some topics lack <category>, <audience>, or <keywords>. *Solution:* Review and add metadata to all topics for discoverability and semantic search.
- **Issue: Incomplete Conditional Processing**Conditional attributes (e.g., platform, audience) are not consistently applied. *Solution:* Audit topics for conditional content and ensure attributes are used where needed.
- **Issue: Analytics Instrumentation Gaps**Not all topics include guidance for analytics. *Solution:* Add analytics instrumentation sections to each topic, with practical examples.
- **Issue: Feedback Mechanisms Missing**HTML outputs lack feedback widgets or prompts. *Solution:* Embed feedback widgets and provide instructions for collecting user input.
- **Issue: AI/Chatbot Integration Not Ready**Topics are not structured for easy extraction by chatbots/ NLP. *Solution:* Ensure topics have clear titles, metadata, and concise answers for chatbot use.
- **Issue: Validation Automation**Manual review is time-consuming. *Solution:* Use DITA-OT plugins or scripts to automate validation of metadata, conditional processing, and analytics sections.

By systematically addressing these issues, documentation teams can ensure their content is not only migrated but also optimized for intelligent delivery, analytics, and AI integration.

Index

Special Characters

@class attribute [150](#)
@class matching [137](#)

A

AI classification [100](#)
AI metadata [94](#)
AI readiness [iii](#), [15](#)
AI retrieval quality [12](#)
AI-ready content [97](#)
Allowed values [204](#)
Answer generation [174](#)
Ant targets [133](#)
architecture [132](#)
Architecture [132](#), [188](#)
args.xml [150](#)
Assessment [15](#)
Authoring [48](#)
Authoring tools [44](#)
Automation [121](#)
Azure DevOps [121](#)

B

Batch enrichment [94](#)
Before and after [88](#)
Benefits [180](#)
Best Practices [48](#)
Build [104](#)
Build configuration [115](#)
Build errors [125](#)
build variants [145](#)
Build variants [152](#)
Build workflow [165](#)

C

Callout styling [161](#)
Chatbot architecture [170](#)
Chatbot optimization [97](#)
Chatbot quality [184](#)
Checklists [166](#)
chunking [133](#)
Chunking [151](#), [164](#)
Chunking strategy [172](#)
CI integration [161](#)
CI pipeline [160](#)
CI/CD [121](#), [132](#), [142](#)
Citation [174](#)
Classification [78](#)

Command line [159](#)
Comparison [88](#)
Concept topic [60](#), [63](#)
Conditional processing [152](#)
Conductor param [159](#)
Conkeyref [66](#)
conref [133](#)
Conref [66](#)
container [140](#)
Content conversion [84](#)
Content intelligence [10](#)
Content maturity [15](#)
Content migration [16](#)
Content quality [180](#)
Content transformation [iii](#)
Contexts [115](#)
Controlled vocabulary [78](#)
Cross-encoder [174](#)
CSS [118](#)
customization [136](#)
Customization [148](#)
Customization strategies [150](#)

D

debugging [144](#)
Debugging [125](#), [160](#)
Deliverables [115](#)
depend.preprocess.post [137](#)
depend.preprocess.pre [137](#)
DITA [28](#), [36](#)
dita --install [153](#)
DITA maps [76](#)
DITA terms [190](#)
DITA-OT [132](#), [132](#)
DITA-OT setup [42](#)
DITA-OT source [139](#)
dita-ot-action [142](#)
dita.xml.html5 [153](#)
dita.xml.html5.toc [158](#)
DITaval [66](#), [118](#), [145](#), [152](#), [202](#)
Docker [132](#), [140](#), [141](#)
docker-compose [141](#)
Dockerfile [141](#)
documentation [146](#)
draft-comment [156](#)

E

Editor setup [44](#)
Element overrides [156](#)
Embeddings [172](#)

End-to-end [56](#)
EPUB [112](#)
Exercises [161](#)
extension points [135](#)
Extension points [149](#), [166](#), [202](#)

F

Feature utilization [181](#)
fig [156](#)
Filtering [66](#), [152](#)
First project [56](#)
FOP [125](#)

G

Getting started [22](#)
GitHub Actions [121](#), [132](#), [142](#)
GitHub Pages [142](#)
Glossary [190](#)
Gradle [139](#)

H

Hands-on [56](#)
hazardstatement [156](#)
HTML5 [112](#), [135](#)
HTML5 plugin [138](#)
HTML5 processing [188](#)
HTML5 transform [149](#)

I

import layers [137](#)
Import precedence [150](#)
Information architecture [76](#)
Information typing [28](#), [36](#), [60](#), [63](#)
Installation [42](#)
Intelligent content [iii](#), [10](#), [97](#)
Intent routing [181](#)
Introduction [28](#)

J

Java [42](#)
Java 17 [139](#)
Jenkins [121](#)
JSON-LD [100](#)

K

keyref [133](#)
Keyref [66](#)
Knowledge base [190](#)

L

Labs [161](#)
Learning path [22](#)
Learning paths [76](#), [181](#)
legacy hooks [137](#)
Limitations [16](#)
local preview [141](#)

M

Machine-readable [10](#)
map-first [135](#)
Maps [28](#)
Maturity model [15](#)
Metadata enrichment [94](#)
Metadata filtering [181](#)
Metadata reference [204](#)
Metrics [180](#)
Migration [84](#)
Multi-channel [112](#)

N

nav-toc [151](#)
Navigation [153](#)
nginx [141](#)
Note callouts [155](#)

O

Output [104](#)
Output customization [118](#)
Output formats [112](#)
override [136](#)
override.xsl [138](#), [202](#)
Overview [36](#)
Oxygen XML [44](#)
Oxygen XML Author [44](#)
Oxygen XML Editor [44](#)

P

Paragraph hooks [155](#)
Parameters [118](#), [159](#)
Patterns [166](#)
PDF [112](#)
ph [156](#)
Pipeline [149](#), [188](#)
pipeline steps [137](#)
plug-in [135](#)
plug-in install [141](#)
plug-in template [138](#)
Plugin [202](#)
Plugin creation [153](#)

Plugin integration [165](#)
Plugin override [150](#)
plugin.xml [135](#), [153](#), [202](#)
preprocess2 [135](#)
preprocessing [132](#), [133](#), [144](#)
Preprocessing [132](#), [149](#), [188](#)
Product impact [184](#)
Production patterns [155](#)
Project files [115](#)
project structure [145](#)
Prolog fields [204](#)
Prolog metadata [94](#)
Properties file [151](#), [159](#)
Properties files [118](#)
Publishing [104](#)

Q

Quality gates [160](#)
Quick start [22](#)

R

RAG authoring [97](#)
RAG pipeline [170](#)
Reference topic [60](#), [63](#)
references [146](#)
Release-ready [166](#)
repository layout [145](#)
reproducible builds [140](#)
Reranking [174](#)
Retrieval [174](#)
Retrieval-Augmented Generation [170](#)
Reuse [66](#)
ROI [180](#)

S

Safe overrides [166](#)
Schema.org [100](#)
Search precision [184](#)
Section-aware chunking [172](#)
Sidebar TOC [151](#), [158](#)
Sidebar TOC missing [164](#)
Specialization [36](#)
Spotless [139](#)
Starter repo [165](#)
Step anchors [161](#)
Step by step [84](#)
strict mode [144](#)
Strict mode [160](#)
Structured authoring [16](#)
Structured content [10](#)
Structured data [100](#)
Subject scheme [78](#)

T

Task steps [155](#)
Task topic [60](#), [63](#)
Taxonomy [78](#)
temp directory [132](#)
Temp directory [160](#)
Template matching [150](#)
Template override [160](#)
Template precedence [154](#)
Templates [48](#)
Terminology [190](#)
Text embedding [172](#)
TOC customization [158](#)
TOC grouping [153](#)
toc-override.xsl [138](#), [158](#)
Topic wrapper [155](#)
Topic-based authoring [76](#)
topicgroup [153](#)
topichead [153](#)
Topics [28](#)
Transformation examples [88](#)
Transtype [166](#)
troubleshooting [144](#)
Troubleshooting [125](#), [164](#)
Tutorial [56](#), [148](#)

U

Unstructured content [12](#), [16](#)
Unstructured content limitations [12](#)
Unstructured XML to DITA [84](#)
Upgrade strategy [160](#)
upgrade-safe [136](#)
User experience [184](#)

V

Vector representation [172](#)
Vector search [170](#)
Verbose logging [125](#)
Version pinning [160](#)

X

xref [156](#)
XSL overrides [164](#)
xsl:import [154](#)
xsl:next-match [137](#), [154](#)
XSLT [137](#), [148](#), [166](#)
XSLT import [136](#)
XSLT override [154](#)
XSLT prerequisites [150](#)
